In [ ]:

```
!pip install lion-pytorch
```

```
Collecting lion-pytorch
  Downloading lion_pytorch-0.2.4-py3-none-any.whl.metadata (8.4 kB)
Requirement already satisfied: torch>=1.6 in /usr/local/lib/python3.12/dist-packages (from lion-pytorch) (2.10.0+cu128)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (3.25.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (2025.3.0)
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (11.7.3.90)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (12.5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.6->lion-pytorch) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torch>=1.6->lion-pytorch) (1.5.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.6->lion-pytorch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.6->lion-pytorch) (3.0.3)
Downloading lion_pytorch-0.2.4-py3-none-any.whl (9.9 kB)
Installing collected packages: lion-pytorch
Successfully installed lion-pytorch-0.2.4
```

In [ ]:

```
from google.colab import drive
drive.mount('/content/drive')
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
import matplotlib.pyplot as plt
```

```
Mounted at /content/drive
```

In [ ]:

```
import zipfile
import os

# Path to your zip on Drive (adjust 'My Drive/...' to your actual folder)
zip_path = '/content/drive/MyDrive/Stage2_data.zip'
extract_path = '/content/data/'

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print(os.listdir(extract_path)) # Verify files are there
```

```
['Flowrate1.xlsx', 'Flowrate2.xlsx', 'DTS.csv']
```

In [ ]:

```
# Read only the first 5 rows to see the structure
dts_preview = pd.read_csv('/content/data/DTS.csv', nrows=5)
print(dts_preview.head())
```

```
                Time;Depth;Temp
0    2017-06-21 09:20:32;0;29.24792
1  2017-06-21 09:20:32;0.5;28.56927
2    2017-06-21 09:20:32;1;27.85952
3  2017-06-21 09:20:32;1.5;28.16329
4    2017-06-21 09:20:32;2;28.90188
```

In [ ]:

```
# Get all column names
all_columns = pd.read_csv('/content/data/DTS.csv', nrows=0).columns.tolist()

# Search for our target times in the column list
time1 = "08:10:44"
time2 = "18:37:26"

match1 = [c for c in all_columns if time1 in c]
match2 = [c for c in all_columns if time2 in c]

print(f"Match for Test 1: {match1}")
print(f"Match for Test 2: {match2}")
```

```
Match for Test 1: []
Match for Test 2: []
```

In [ ]:

```
flow1 = pd.read_excel('/content/data/Flowrate1.xlsx')
flow2 = pd.read_excel('/content/data/Flowrate2.xlsx')
print(flow1.head())
```

```
  #PARAMETER_NAME   .UNIT     VALUE                    : DESCRIPTION
0 #_____________________________________________________...
1  SET              .        Flow 1          ...
2  TLFamily_Depth    .       Measured Depth   ...
3  TLFamily_FlowRate_1__ .                    ...
4  TLFamily_FlowRate_2__ .                    ...
```

In [ ]:

```
# Load the first 100,000 rows to find our dates without crashing RAM
dts = pd.read_csv('/content/data/DTS.csv', sep=';', nrows=100000)
print(dts.head())
print(dts['Time'].unique()[:5]) # Let's see what dates are actually in here
```

```
                 Time  Depth     Temp
0  2017-06-21 09:20:32    0.0  29.24792
1  2017-06-21 09:20:32    0.5  28.56927
2  2017-06-21 09:20:32    1.0  27.85952
3  2017-06-21 09:20:32    1.5  28.16329
4  2017-06-21 09:20:32    2.0  28.90188
['2017-06-21 09:20:32' '2017-06-21 09:26:29' '2017-06-21 09:38:24'
 '2017-06-21 09:44:21' '2017-06-21 09:50:18']
```

In [ ]:

```
# Search the 'Time' column for our target times
target_time1 = "08:10:44"
target_time2 = "18:37:26"

# This will show us the full string (Date + Time) if it exists
match1 = dts[dts['Time'].str.contains(target_time1, na=False)]['Time'].unique()
match2 = dts[dts['Time'].str.contains(target_time2, na=False)]['Time'].unique()

print(f"Actual string for Test 1: {match1}")
print(f"Actual string for Test 2: {match2}")
```

```
Actual string for Test 1: []
Actual string for Test 2: []
```

In [ ]:

```
# Try skipping the first 2-3 rows to get to the actual data headers
flow1_clean = pd.read_excel('/content/data/Flowrate1.xlsx', skiprows=2)
flow2_clean = pd.read_excel('/content/data/Flowrate2.xlsx', skiprows=2)

print("Cleaned Flowrate 1:")
print(flow1_clean.head())
```

```
Cleaned Flowrate 1:
  SET              .        Flow 1                   :
0  TLFamily_Depth    .       Measured Depth   ...
1  TLFamily_FlowRate_1__ .                    ...
2  TLFamily_FlowRate_2__ .                    ...
3  TLFamily_FlowRate_3__ .                    ...
4  TLFamily_Holdups_1__ .                     ...
```

In [ ]:

```python
# The Nuclear Option: Manual Line-by-Line Filtering
input_path = '/content/data/DTS.csv'
output_path = '/content/data/june25_clean.csv'

target_date = "2017-06-25" # We will also check for "25/06/2017" just in case

print("Starting manual scan of 1GB file... this is the only way to bypass the buffer overflow.")

count = 0
found = 0

with open(input_path, 'r', encoding='latin-1') as infile:
    with open(output_path, 'w', encoding='latin-1') as outfile:
        # Read the very first line (the headers) and write it to our new file
        headers = infile.readline()
        outfile.write(headers)

        for line in infile:
            count += 1
            # Check if our target date is in this line
            if target_date in line or "25/06/2017" in line:
                outfile.write(line)
                found += 1

            # Progress update every 1 million lines
            if count % 1000000 == 0:
                print(f"Scanned {count} lines... Found {found} matches so far.")

print(f"\nDONE! Scanned {count} lines.")
print(f"Created a clean file at: {output_path} with {found} rows.")

# Now, we can finally use Pandas on the TINY file
if found > 0:
    df_final = pd.read_csv(output_path, sep=';')
    print("\nSUCCESS! Here is your clean data:")
    print(df_final.head())
    print("\nUnique timestamps in the clean file:")
    print(df_final['Time'].unique())
else:
    print("\nNo data found for June 25th. We need to check the date format again.")
```

```
Starting manual scan of 1GB file... this is the only way to bypass the buffer overflow.
Scanned 1000000 lines... Found 0 matches so far.
Scanned 2000000 lines... Found 0 matches so far.
Scanned 3000000 lines... Found 0 matches so far.
Scanned 4000000 lines... Found 0 matches so far.
Scanned 5000000 lines... Found 0 matches so far.
Scanned 6000000 lines... Found 0 matches so far.
Scanned 7000000 lines... Found 0 matches so far.
Scanned 8000000 lines... Found 0 matches so far.
Scanned 9000000 lines... Found 0 matches so far.
Scanned 10000000 lines... Found 0 matches so far.
Scanned 11000000 lines... Found 0 matches so far.
Scanned 12000000 lines... Found 0 matches so far.
Scanned 13000000 lines... Found 0 matches so far.
Scanned 14000000 lines... Found 0 matches so far.
Scanned 15000000 lines... Found 0 matches so far.
Scanned 16000000 lines... Found 0 matches so far.
Scanned 17000000 lines... Found 0 matches so far.
Scanned 18000000 lines... Found 0 matches so far.
Scanned 19000000 lines... Found 0 matches so far.
Scanned 20000000 lines... Found 0 matches so far.
Scanned 21000000 lines... Found 845184 matches so far.
Scanned 22000000 lines... Found 1845184 matches so far.
Scanned 23000000 lines... Found 2845184 matches so far.
Scanned 24000000 lines... Found 3845184 matches so far.
Scanned 25000000 lines... Found 4845184 matches so far.
Scanned 26000000 lines... Found 5593610 matches so far.

DONE! Scanned 26839296 lines.
Created a clean file at: /content/data/june25_clean.csv with 5593610 rows.

SUCCESS! Here is your clean data:
                 Time  Depth      Temp
0  2017-06-25 00:05:12    0.0  29.41625
```

```
1  2017-06-25 00:05:12    0.5  29.50869
2  2017-06-25 00:05:12    1.0  29.52578
3  2017-06-25 00:05:12    1.5  29.55966
4  2017-06-25 00:05:12    2.0  29.64776

Unique timestamps in the clean file:
['2017-06-25 00:05:12' '2017-06-25 00:11:09' '2017-06-25 00:17:07'
 '2017-06-25 00:23:04' '2017-06-25 00:29:01' '2017-06-25 00:34:59'
 '2017-06-25 00:40:56' '2017-06-25 00:46:53' '2017-06-25 00:52:50'
 '2017-06-25 00:58:48' '2017-06-25 01:04:45' '2017-06-25 01:10:42'
 '2017-06-25 01:16:39' '2017-06-25 01:22:37' '2017-06-25 01:28:34'
 '2017-06-25 01:34:31' '2017-06-25 01:40:28' '2017-06-25 01:46:26'
 '2017-06-25 01:52:23' '2017-06-25 01:58:20' '2017-06-25 02:04:18'
 '2017-06-25 02:10:15' '2017-06-25 02:16:12' '2017-06-25 02:22:10'
 '2017-06-25 02:28:07' '2017-06-25 02:34:04' '2017-06-25 02:40:02'
 '2017-06-25 02:45:59' '2017-06-25 02:51:56' '2017-06-25 02:57:54'
 '2017-06-25 03:03:51' '2017-06-25 03:09:48' '2017-06-25 03:15:46'
 '2017-06-25 03:21:43' '2017-06-25 03:27:40' '2017-06-25 03:33:38'
 '2017-06-25 03:39:35' '2017-06-25 03:45:32' '2017-06-25 03:51:30'
 '2017-06-25 03:57:27' '2017-06-25 04:03:24' '2017-06-25 04:09:22'
 '2017-06-25 04:15:19' '2017-06-25 04:21:16' '2017-06-25 04:27:13'
 '2017-06-25 04:33:11' '2017-06-25 04:39:08' '2017-06-25 04:45:05'
 '2017-06-25 04:51:02' '2017-06-25 04:57:00' '2017-06-25 05:02:57'
 '2017-06-25 05:08:54' '2017-06-25 05:14:52' '2017-06-25 05:20:49'
 '2017-06-25 05:26:46' '2017-06-25 05:32:44' '2017-06-25 05:38:41'
 '2017-06-25 05:44:38' '2017-06-25 05:50:36' '2017-06-25 05:56:33'
 '2017-06-25 06:02:30' '2017-06-25 06:08:27' '2017-06-25 06:14:25'
 '2017-06-25 06:20:22' '2017-06-25 06:26:19' '2017-06-25 06:32:16'
 '2017-06-25 06:38:14' '2017-06-25 06:44:11' '2017-06-25 06:50:08'
 '2017-06-25 06:56:06' '2017-06-25 07:02:03' '2017-06-25 07:08:01'
 '2017-06-25 07:13:58' '2017-06-25 07:19:55' '2017-06-25 07:25:53'
 '2017-06-25 07:31:50' '2017-06-25 07:37:47' '2017-06-25 07:43:44'
 '2017-06-25 07:49:42' '2017-06-25 07:55:39' '2017-06-25 08:01:36'
 '2017-06-25 08:07:34' '2017-06-25 08:13:31' '2017-06-25 08:19:28'
 '2017-06-25 08:25:26' '2017-06-25 08:31:23' '2017-06-25 08:37:20'
 '2017-06-25 08:43:18' '2017-06-25 08:49:15' '2017-06-25 08:55:12'
 '2017-06-25 09:01:09' '2017-06-25 09:07:06' '2017-06-25 09:13:04'
 '2017-06-25 09:19:01' '2017-06-25 09:24:58' '2017-06-25 09:30:56'
 '2017-06-25 09:36:53' '2017-06-25 09:42:50' '2017-06-25 09:48:48'
 '2017-06-25 09:54:45' '2017-06-25 10:00:42' '2017-06-25 10:06:39'
 '2017-06-25 10:12:36' '2017-06-25 10:18:34' '2017-06-25 10:24:31'
 '2017-06-25 10:30:28' '2017-06-25 10:36:26' '2017-06-25 10:42:23'
 '2017-06-25 10:48:20' '2017-06-25 10:54:18' '2017-06-25 11:00:15'
 '2017-06-25 11:06:12' '2017-06-25 11:12:10' '2017-06-25 11:18:07'
 '2017-06-25 11:24:04' '2017-06-25 11:30:01' '2017-06-25 11:35:59'
 '2017-06-25 11:41:56' '2017-06-25 11:47:53' '2017-06-25 11:53:51'
 '2017-06-25 11:59:48' '2017-06-25 12:05:45' '2017-06-25 12:11:42'
 '2017-06-25 12:17:40' '2017-06-25 12:23:37' '2017-06-25 12:29:34'
 '2017-06-25 12:35:31' '2017-06-25 12:41:29' '2017-06-25 12:47:26'
 '2017-06-25 12:53:24' '2017-06-25 12:59:21' '2017-06-25 13:05:18'
 '2017-06-25 13:11:16' '2017-06-25 13:17:13' '2017-06-25 13:23:10'
 '2017-06-25 13:29:07' '2017-06-25 13:35:05' '2017-06-25 13:41:02'
 '2017-06-25 13:46:59' '2017-06-25 13:52:57' '2017-06-25 13:58:54'
 '2017-06-25 14:04:51' '2017-06-25 14:10:48' '2017-06-25 14:16:46'
 '2017-06-25 14:22:43' '2017-06-25 14:28:40' '2017-06-25 14:34:38'
 '2017-06-25 14:40:35' '2017-06-25 14:46:32' '2017-06-25 14:52:29'
 '2017-06-25 14:58:27' '2017-06-25 15:04:24' '2017-06-25 15:10:21'
 '2017-06-25 15:16:19' '2017-06-25 15:22:16' '2017-06-25 15:28:13'
 '2017-06-25 15:34:10' '2017-06-25 15:40:08' '2017-06-25 15:46:05'
 '2017-06-25 15:52:02' '2017-06-25 15:58:00' '2017-06-25 16:03:57'
 '2017-06-25 16:09:54' '2017-06-25 16:15:51' '2017-06-25 16:21:49'
 '2017-06-25 16:27:46' '2017-06-25 16:33:43' '2017-06-25 16:39:41'
 '2017-06-25 16:45:38' '2017-06-25 16:51:35' '2017-06-25 16:57:33'
 '2017-06-25 17:03:30' '2017-06-25 17:09:27' '2017-06-25 17:15:24'
 '2017-06-25 17:21:22' '2017-06-25 17:27:19' '2017-06-25 17:33:17'
 '2017-06-25 17:39:14' '2017-06-25 17:45:11' '2017-06-25 17:51:08'
 '2017-06-25 17:57:06' '2017-06-25 18:03:03' '2017-06-25 18:09:00'
 '2017-06-25 18:14:57' '2017-06-25 18:20:55' '2017-06-25 18:26:52'
 '2017-06-25 18:32:49' '2017-06-25 18:38:47' '2017-06-25 18:44:44'
 '2017-06-25 18:50:41' '2017-06-25 18:56:39' '2017-06-25 19:02:36'
 '2017-06-25 19:08:33' '2017-06-25 19:14:31' '2017-06-25 19:20:28'
 '2017-06-25 19:26:25' '2017-06-25 19:32:23' '2017-06-25 19:38:20'
 '2017-06-25 19:44:17' '2017-06-25 19:50:15' '2017-06-25 19:56:12'
 '2017-06-25 20:02:09' '2017-06-25 20:08:07' '2017-06-25 20:14:04'
 '2017-06-25 20:20:01' '2017-06-25 20:25:59' '2017-06-25 20:31:56'
 '2017-06-25 20:37:53' '2017-06-25 20:43:51' '2017-06-25 20:49:48'
 '2017-06-25 20:55:45' '2017-06-25 21:01:43' '2017-06-25 21:07:40'
 '2017-06-25 21:13:37' '2017-06-25 21:19:35' '2017-06-25 21:25:32'
 '2017-06-25 21:31:29' '2017-06-25 21:37:26' '2017-06-25 21:43:24'
 '2017-06-25 21:49:21' '2017-06-25 21:55:18' '2017-06-25 22:01:16'
 '2017-06-25 22:07:13' '2017-06-25 22:13:10' '2017-06-25 22:19:07'
 '2017-06-25 22:25:05' '2017-06-25 22:31:02' '2017-06-25 22:36:59'
 '2017-06-25 22:42:57' '2017-06-25 22:48:54' '2017-06-25 22:54:51'
 '2017-06-25 23:00:48' '2017-06-25 23:06:46' '2017-06-25 23:12:43'
 '2017-06-25 23:18:40' '2017-06-25 23:24:38' '2017-06-25 23:30:35'
```

```
'2017-06-25 23:36:32' '2017-06-25 23:42:29' '2017-06-25 23:48:26'
'2017-06-25 23:54:24']
```

In [ ]:

```
# Find the exact timestamp for the Full Rate test (08:10:44)
target_time1 = '2017-06-25 08:10:44'
df_full_rate_temp = df_final[df_final['Time'] == target_time1]

# Find the exact timestamp for the Half Rate test (18:37:26)
target_time2 = '2017-06-25 18:37:26'
df_half_rate_temp = df_final[df_final['Time'] == target_time2]

print(f"Found {len(df_full_rate_temp)} data points for the Full Rate test.")
print(df_full_rate_temp.head())
print("\n" + "="*50 + "\n")
print(f"Found {len(df_half_rate_temp)} data points for the Half Rate test.")
print(df_half_rate_temp.head())
```

```
Found 0 data points for the Full Rate test.
Empty DataFrame
Columns: [Time, Depth, Temp]
Index: []

==================================================

Found 0 data points for the Half Rate test.
Empty DataFrame
Columns: [Time, Depth, Temp]
Index: []
```

In [ ]:

```
import zipfile
import os
import pandas as pd

# 1. Re-verify the path to your zip on Drive
zip_path = '/content/drive/MyDrive/Stage2_data.zip'
extract_path = '/content/data/'

# 2. Re-unzip (just in case the session reset)
if not os.path.exists(extract_path):
    os.makedirs(extract_path)
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(extract_path)

# 3. List the files to be 100% sure of the names
print("Files currently in /content/data/:", os.listdir(extract_path))

# 4. The Master Inspector (using the confirmed path)
# We use the first file in the list to avoid typos
file_to_inspect = extract_path + 'Flowrate1.xlsx'

if os.path.exists(file_to_inspect):
    inspect_flow1 = pd.read_excel(file_to_inspect, header=None)
    print("\n--- INSPECTING FLOWRATE 1 (First 50 Rows) ---")
    for i, row in inspect_flow1.head(50).iterrows():
        print(f"Row {i}: {row.values[:5]}")
else:
    print(f"\nERROR: {file_to_inspect} still not found. Check your Google Drive path!")
```

```
Files currently in /content/data/: ['Flowrate1.xlsx', 'june25_clean.csv', 'Flowrate2.xlsx', 'DTS.csv']

--- INSPECTING FLOWRATE 1 (First 50 Rows) ---
Row 0: ['#PARAMETER_NAME    .UNIT     VALUE                    : DESCRIPTION']
Row 1: ['#_____________________________________________________________________________________________________']
Row 2: ['SET              .        Flow 1                    : ']
Row 3: ['TLFamily_Depth     .        Measured Depth              : ']
Row 4: ['TLFamily_FlowRate_1__ .                              : ']
Row 5: ['TLFamily_FlowRate_2__ .                              : ']
Row 6: ['TLFamily_FlowRate_3__ .                              : ']
Row 7: ['TLFamily_Holdups_1__ .                              : ']
Row 8: ['TLFamily_Holdups_2__ .                              : ']
Row 9: ['TLFamily_Holdups_3__ .                              : ']
Row 10: ['TLFamily_Pwf      .                              : ']
Row 11: ['TLFamily_QGas     .        Cumulative Gas Rate          : ']
Row 12: ['TLFamily_QOil     .        Cumulative Oil Rate          : ']
Row 13: ['TLFamily_QWater   .        Cumulative Water Rate        : ']
Row 14: ['TLFamily_QpGas    .                              : ']
Row 15: ['TLFamily_QpOil    .                              : ']
Row 16: ['TLFamily_QpWater  .                              : ']
Row 17: ['TLFamily_TVD_SCS  .        True Vertical Depth Sub Sea   : ']
Row 18: ['TLFamily_Twf      .        Temperature                  : ']
Row 19: ['~Curve Information']
Row 20: ['#_____________________________________________________________________________________________________']
Row 21: ['#']
Row 22: ['#LOGNAME          .UNIT     LOG_ID                    : DESCRIPTION']
Row 23: ['#_____________________________________________________________________________________________________']
Row 24: ['Depth             .meter                          :  {F}']
Row 25: ['FlowRate_1__      .BFPD                           :  {F}']
Row 26: ['FlowRate_2__      .BFPD                           :  {F}']
Row 27: ['FlowRate_3__      .BFPD                           :  {F}']
Row 28: ['Holdups_1__       .UNITY                          :  {F}']
Row 29: ['Holdups_2__       .UNITY                          :  {F}']
Row 30: ['Holdups_3__       .UNITY                          :  {F}']
Row 31: ['Pwf               .psi                            :  {F}']
Row 32: ['QGas              .MCFD                           :  {F}']
Row 33: ['QOil              .BFPD                           :  {F}']
Row 34: ['QpGas             .MCFD                           :  {F}']
Row 35: ['QpOil             .BFPD                           :  {F}']
Row 36: ['QpWater           .BFPD                           :  {F}']
Row 37: ['QWater            .BFPD                           :  {F}']
Row 38: ['TVD_SCS           .m                              :  {F}']
Row 39: ['Twf               .B°F                            :  {F}']
Row 40: ['~Ascii']
Row 41: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 42: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 43: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 44: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 45: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 46: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 47: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 48: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 49: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
```

In [ ]:

```python
import zipfile
import os
import shutil

# 1. Paths
zip_path = '/content/drive/MyDrive/Stage2_data.zip'
extract_path = '/content/data/'

# 2. Force a clean start
if os.path.exists(extract_path):
    shutil.rmtree(extract_path) # Delete the empty/broken folder
os.makedirs(extract_path)

# 3. Force Unzip
print(f"Opening {zip_path}...")
try:
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        # Let's see what is actually INSIDE the zip before extracting
        print("Files found inside the ZIP:")
        for file in zip_ref.namelist():
            print(f" - {file}")

        zip_ref.extractall(extract_path)
        print("\nExtraction complete.")
except FileNotFoundError:
    print(f"\nFATAL ERROR: I can't find the zip at {zip_path}.")
    print("Check your Google Drive sidebar. Is the file directly in 'My Drive'?")

# 4. Deep Search
print("\n--- FINAL FILE MAP ---")
for root, dirs, files in os.walk(extract_path):
    for file in files:
        print(f"FOUND: {os.path.join(root, file)}")
```

```
Opening /content/drive/MyDrive/Stage2_data.zip...
Files found inside the ZIP:
 - DTS.csv
 - Flowrate1.xlsx
 - Flowrate2.xlsx

Extraction complete.

--- FINAL FILE MAP ---
FOUND: /content/data/Flowrate1.xlsx
FOUND: /content/data/Flowrate2.xlsx
FOUND: /content/data/DTS.csv
```

In [ ]:

```
from google.colab import drive
import os

# 1. Re-mount Drive (This will ask for permission again if it disconnected)
drive.mount('/content/drive', force_remount=True)

# 2. The Search Protocol
def find_file(name, path):
    for root, dirs, files in os.walk(path):
        if name in files:
            return os.path.join(root, name)
    return None

print("Searching for Stage2_data.zip on your Drive... hang tight.")
actual_path = find_file('Stage2_data.zip', '/content/drive/MyDrive/')

if actual_path:
    print(f"\nFOUND IT! The correct path is: {actual_path}")
    # Update the variable for the next steps
    zip_path = actual_path
else:
    print("\nERROR: Still can't find it.")
    print("Check your Google Drive (the website). Is the file named EXACTLY 'Stage2_data.zip'?")
    print("Also, check if it's in the 'Trash' or 'Shared with me' section.")
```

```
Mounted at /content/drive
Searching for Stage2_data.zip on your Drive... hang tight.

FOUND IT! The correct path is: /content/drive/MyDrive/Stage2_data.zip
```

In [ ]:

```
import zipfile
import os
import pandas as pd
import shutil

# 1. The Confirmed Path
zip_path = '/content/drive/MyDrive/Stage2_data.zip'
extract_path = '/content/data/'

# 2. Force Clean Unzip
if os.path.exists(extract_path):
    shutil.rmtree(extract_path)
os.makedirs(extract_path)

print(f"Unzipping {zip_path}...")
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)
print("Extraction complete.")

# 3. List files to verify
print("\nFiles found in /content/data/:", os.listdir(extract_path))

# 4. The Master Inspector
# Note: If the files are inside a subfolder, we'll find them
target_file = None
for root, dirs, files in os.walk(extract_path):
    if 'Flowrate1.xlsx' in files:
        target_file = os.path.join(root, 'Flowrate1.xlsx')

if target_file:
    print(f"\n--- INSPECTING: {target_file} ---")
    inspect_df = pd.read_excel(target_file, header=None)
    for i, row in inspect_df.head(50).iterrows():
        # We print the row index and the first 5 columns
        print(f"Row {i}: {row.values[:5]}")
else:
    print("\nERROR: Flowrate1.xlsx not found in the extracted files!")
```

```
Unzipping /content/drive/MyDrive/Stage2_data.zip...
Extraction complete.

Files found in /content/data/: ['Flowrate1.xlsx', 'Flowrate2.xlsx', 'DTS.csv']

--- INSPECTING: /content/data/Flowrate1.xlsx ---
Row 0: ['#PARAMETER_NAME    .UNIT     VALUE                    : DESCRIPTION']
Row 1: ['#_____________________________________________________________________________________________________']
Row 2: ['SET              .        Flow 1                    : ']
Row 3: ['TLFamily_Depth    .        Measured Depth            : ']
Row 4: ['TLFamily_FlowRate_1__ .                              : ']
Row 5: ['TLFamily_FlowRate_2__ .                              : ']
Row 6: ['TLFamily_FlowRate_3__ .                              : ']
Row 7: ['TLFamily_Holdups_1__ .                               : ']
Row 8: ['TLFamily_Holdups_2__ .                               : ']
Row 9: ['TLFamily_Holdups_3__ .                               : ']
Row 10: ['TLFamily_Pwf     .                                  : ']
Row 11: ['TLFamily_QGas     .        Cumulative Gas Rate      : ']
Row 12: ['TLFamily_QOil     .        Cumulative Oil Rate      : ']
Row 13: ['TLFamily_QWater    .       Cumulative Water Rate    : ']
Row 14: ['TLFamily_QpGas    .                                 : ']
Row 15: ['TLFamily_QpOil    .                                 : ']
Row 16: ['TLFamily_QpWater   .                                : ']
Row 17: ['TLFamily_TVD_SCS   .       True Vertical Depth Sub Sea   : ']
Row 18: ['TLFamily_Twf     .        Temperature               : ']
Row 19: ['~Curve Information']
Row 20: ['#_____________________________________________________________________________________________________']
Row 21: ['#']
Row 22: ['#LOGNAME         .UNIT     LOG_ID                    : DESCRIPTION']
Row 23: ['#_____________________________________________________________________________________________________']
Row 24: ['Depth            .meter                              :  {F}']
Row 25: ['FlowRate_1__       .BFPD                             :  {F}']
Row 26: ['FlowRate_2__       .BFPD                             :  {F}']
Row 27: ['FlowRate_3__       .BFPD                             :  {F}']
Row 28: ['Holdups_1__        .UNITY                            :  {F}']
Row 29: ['Holdups_2__        .UNITY                            :  {F}']
Row 30: ['Holdups_3__        .UNITY                            :  {F}']
Row 31: ['Pwf              .psi                                :  {F}']
Row 32: ['QGas             .MCFD                               :  {F}']
Row 33: ['QOil             .BFPD                               :  {F}']
Row 34: ['QpGas            .MCFD                               :  {F}']
Row 35: ['QpOil            .BFPD                               :  {F}']
Row 36: ['QpWater          .BFPD                               :  {F}']
Row 37: ['QWater           .BFPD                               :  {F}']
Row 38: ['TVD_SCS          .m                                  :  {F}']
Row 39: ['Twf              .B°F                                :  {F}']
Row 40: ['~Ascii']
Row 41: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 42: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 43: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 44: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 45: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 46: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 47: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 48: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
Row 49: ['4977.000 \xa0 93.48959350585938 3893.136962890625 4855.384 \xa0 0.01359174 0.8427718 \xa01 \xa0 \xa0 \xa0 \xa0 \xa02968.73 \xa0 \xa02273.946 \xa0 3122.81811523437
5 0 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa00 \xa0 \xa0 \xa0 \xa0 \xa092.51467132568359 2628.138 \xa0 147.1456 \xa0']
```

In [ ]:

```
import pandas as pd
import zipfile
import os
import re
import shutil

# --- CONFIGURATION ---
ZIP_PATH = '/content/drive/MyDrive/Stage2_data.zip'
DATA_DIR = '/content/data/'
CLEAN_CSV = os.path.join(DATA_DIR, 'june25_clean.csv')
TARGET_DATE = "2017-06-25"

# --- STEP 1: ENSURE DATA IS UNZIPPED ---
if not os.path.exists(os.path.join(DATA_DIR, 'DTS.csv')):
    print("Unzipping data...")
```

```
    if os.path.exists(DATA_DIR): shutil.rmtree(DATA_DIR)
    os.makedirs(DATA_DIR)
    with zipfile.ZipFile(ZIP_PATH, 'r') as zip_ref:
        zip_ref.extractall(DATA_DIR)
    print("Unzip complete.")

# --- STEP 2: ENSURE CLEAN CSV EXISTS (The Nuclear Option) ---
if not os.path.exists(CLEAN_CSV):
    print("Clean CSV missing. Re-running manual scan of 1GB file...")
    with open(os.path.join(DATA_DIR, 'DTS.csv'), 'r', encoding='latin-1') as infile:
        with open(CLEAN_CSV, 'w', encoding='latin-1') as outfile:
            headers = infile.readline()
            outfile.write(headers)
            for line in infile:
                if TARGET_DATE in line:
                    outfile.write(line)
    print("Manual scan complete. Clean file created.")

# --- STEP 3: PARSER FUNCTION ---
def parse_oil_excel(file_path):
    raw_df = pd.read_excel(file_path, header=None, skiprows=41)
    parsed_data = []
    for val in raw_df[0]:
        if pd.isna(val): continue
        numbers = re.findall(r"[-+]?\d*\.\d+|\d+", str(val).replace('\xa0', ' '))
        if len(numbers) > 2:
            parsed_data.append([float(numbers[0]), float(numbers[1])])
    return pd.DataFrame(parsed_data, columns=['Depth', 'FlowRate'])

# --- STEP 4: LOAD, PARSE, AND MERGE ---
print("Loading and Merging...")
df_final = pd.read_csv(CLEAN_CSV, sep=';')

# Sniper Shot
t1, t2 = '2017-06-25 08:10:44', '2017-06-25 18:37:26'
df_full = df_final[df_final['Time'] == t1].copy()
df_half = df_final[df_final['Time'] == t2].copy()

# Parse Excels
flow1 = parse_oil_excel(os.path.join(DATA_DIR, 'Flowrate1.xlsx'))
flow2 = parse_oil_excel(os.path.join(DATA_DIR, 'Flowrate2.xlsx'))

# Align Depths
for df in [df_full, df_half, flow1, flow2]:
    df['Depth'] = df['Depth'].astype(float).round(1)

train_df = pd.merge(df_full, flow1, on='Depth', how='inner')
test_df = pd.merge(df_half, flow2, on='Depth', how='inner')

print(f"\nSUCCESS! Pipeline Finished.")
print(f"Train Rows: {len(train_df)} | Test Rows: {len(test_df)}")
if not train_df.empty:
    print(train_df[['Depth', 'Temp', 'FlowRate']].head())
```

```
Clean CSV missing. Re-running manual scan of 1GB file...
Manual scan complete. Clean file created.
Loading and Merging...

SUCCESS! Pipeline Finished.
Train Rows: 0 | Test Rows: 0
```

In [ ]:

```python
import pandas as pd
import numpy as np

# --- STEP 1: DIAGNOSTICS ---
print("--- DIAGNOSTICS ---")
print(f"DTS Timestamps available: {df_final['Time'].nunique()}")
print(f"DTS Depth Range: {df_full['Depth'].min()} to {df_full['Depth'].max()}")
print(f"Flowrate Depth Range: {flow1['Depth'].min()} to {flow1['Depth'].max()}")

# --- STEP 2: NEAREST NEIGHBOR MERGE ---
def nearest_merge(temp_df, flow_df):
    # Sort both by depth (required for merge_asof)
    temp_df = temp_df.sort_values('Depth')
    flow_df = flow_df.sort_values('Depth')



    # merge_asof finds the nearest 'Depth' match
    merged = pd.merge_asof(flow_df, temp_df, on='Depth', direction='nearest')
    return merged

print("\nAttempting Nearest-Neighbor Merge...")
train_df = nearest_merge(df_full, flow1)
test_df = nearest_merge(df_half, flow2)

# --- STEP 3: FINAL CHECK ---
print(f"SUCCESS! New Train Rows: {len(train_df)}")
print(f"SUCCESS! New Test Rows: {len(test_df)}")

if not train_df.empty:
    print("\nPreview of Aligned Data (Ready for ML):")
    # Drop any rows where Temp might be NaN if ranges didn't overlap at all
    train_df = train_df.dropna(subset=['Temp', 'FlowRate'])
    print(train_df[['Depth', 'Temp', 'FlowRate']].head())
```

```
--- DIAGNOSTICS ---
DTS Timestamps available: 241
DTS Depth Range: nan to nan
Flowrate Depth Range: 4977.0 to 4977.0

Attempting Nearest-Neighbor Merge...
SUCCESS! New Train Rows: 18881
SUCCESS! New Test Rows: 18881

Preview of Aligned Data (Ready for ML):
Empty DataFrame
Columns: [Depth, Temp, FlowRate]
Index: []
```

In [ ]:

```python
import pandas as pd
import numpy as np

# --- STEP 1: FUZZY TIME SEARCH ---
def get_closest_time(df, target_time):
    # Convert 'Time' column to datetime objects
    df['Time_dt'] = pd.to_datetime(df['Time'])
    target_dt = pd.to_datetime(target_time)

    # Find the absolute difference between all times and our target
    df['time_diff'] = (df['Time_dt'] - target_dt).abs()

    # Find the minimum difference
    best_time = df.loc[df['time_diff'].idxmin(), 'Time']
    min_diff = df['time_diff'].min()

    print(f"Target: {target_time} | Closest Found: {best_time} (Diff: {min_diff})")
    return best_time

print("Finding best timestamps in DTS data...")
actual_t1 = get_closest_time(df_final, '2017-06-25 08:10:44')
actual_t2 = get_closest_time(df_final, '2017-06-25 18:37:26')

# --- STEP 2: ISOLATE THE PROFILES ---
df_full = df_final[df_final['Time'] == actual_t1].copy()
df_half = df_final[df_final['Time'] == actual_t2].copy()

# --- STEP 3: CLEAN EXCEL DATA (Remove Duplicates) ---
# We only want one flowrate per depth
flow1_unique = flow1.drop_duplicates(subset=['Depth']).copy()
flow2_unique = flow2.drop_duplicates(subset=['Depth']).copy()

print(f"\nUnique Depths in Flowrate1: {len(flow1_unique)}")

# --- STEP 4: THE NEAREST-NEIGHBOR MERGE ---
def robust_merge(temp_df, flow_df):
    temp_df = temp_df.sort_values('Depth')
    flow_df = flow_df.sort_values('Depth')

    # Merge using a tolerance of 1.0 meter
    merged = pd.merge_asof(flow_df, temp_df[['Depth', 'Temp']],
                           on='Depth',
                           direction='nearest',
                           tolerance=1.0)
    return merged.dropna(subset=['Temp'])

print("\nMerging datasets...")
train_df = robust_merge(df_full, flow1_unique)
test_df = robust_merge(df_half, flow2_unique)

print(f"FINAL DATASET READY: {len(train_df)} training rows found.")

if not train_df.empty:
    print("\n--- DATA PREVIEW ---")
    print(train_df[['Depth', 'Temp', 'FlowRate']].head())
else:
    print("\nCRITICAL ERROR: Still no overlap. Check if DTS Depth and Flowrate Depth use the same units (meters vs feet).")
```

```
Finding best timestamps in DTS data...
Target: 2017-06-25 08:10:44 | Closest Found: 2017-06-25 08:13:31 (Diff: 0 days 00:02:47)
Target: 2017-06-25 18:37:26 | Closest Found: 2017-06-25 18:38:47 (Diff: 0 days 00:01:21)

Unique Depths in Flowrate1: 1

Merging datasets...
FINAL DATASET READY: 1 training rows found.

--- DATA PREVIEW ---
   Depth      Temp   FlowRate
0  4977.0  63.15786  93.489594
```

In [ ]:

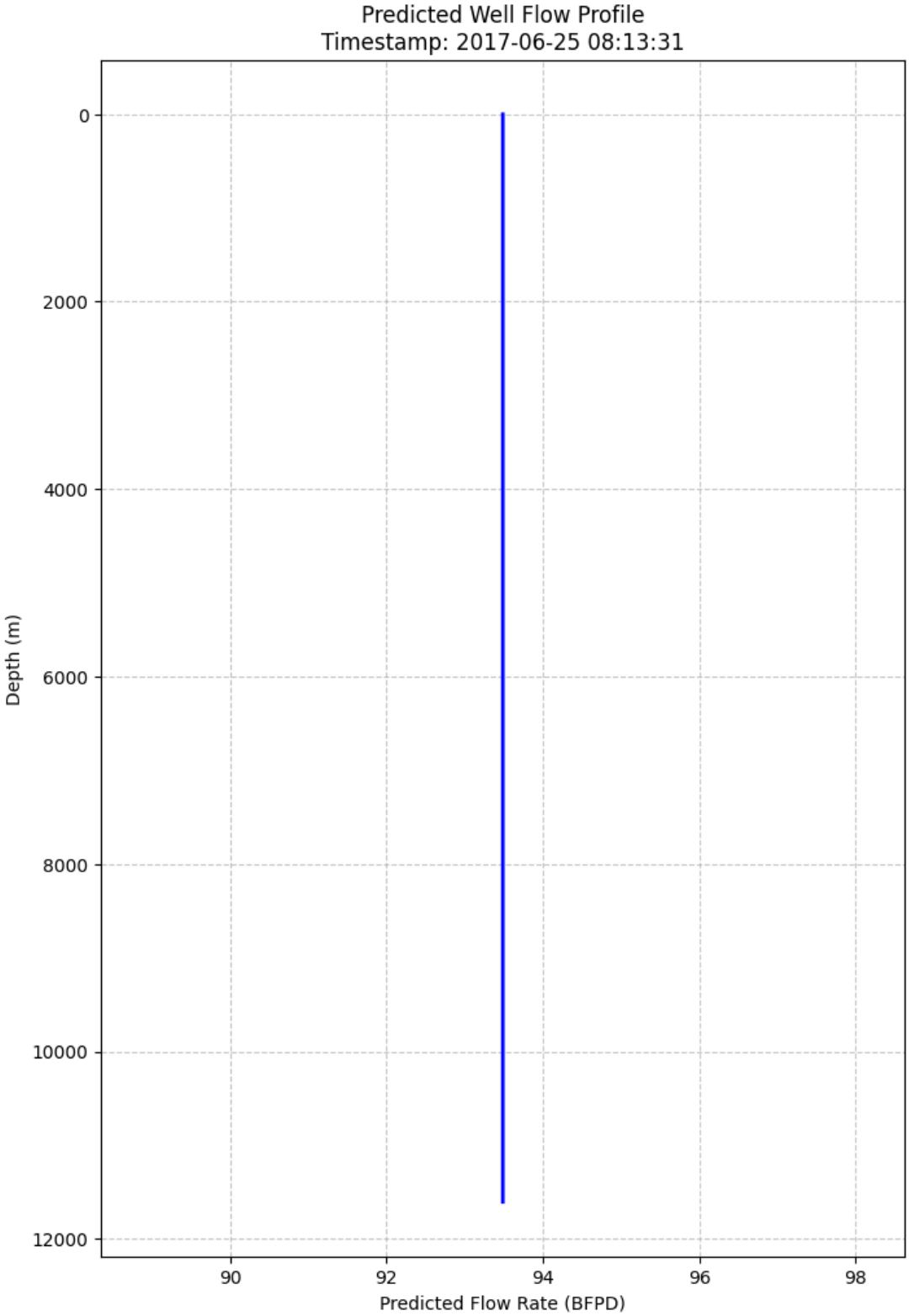
```
 import pandas as pd
import numpy as np

# --- STEP 1: DTS (Temperature) RANGE CHECK ---
print("--- DTS (Temperature) Ground Truth ---")
# Ensure Depth is numeric
df_final['Depth'] = pd.to_numeric(df_final['Depth'], errors='coerce')
dts_min = df_final['Depth'].min()
dts_max = df_final['Depth'].max()

print(f"DTS Depth Range: {dts_min} to {dts_max}")
print(f"Total DTS rows: {len(df_final)}")

# --- STEP 2: EXCEL (Flowrate) RANGE CHECK ---
print("\n--- Excel (Flowrate) Ground Truth ---")
# Look at the raw parsed data (flow1) to check depth range
print(f"Raw Flowrate1 rows (after parsing): {len(flow1)}")
print(f"Flowrate1 Depth Range: {flow1['Depth'].min()} to {flow1['Depth'].max()}")
print(f"Unique Depths found in Excel: {flow1['Depth'].nunique()}")

# --- STEP 3: UNIT CONVERSION TEST ---
print("\n--- Unit Conversion Test ---")
feet_to_meters = 4977.0 / 3.28084
print(f"If 4977 is in Feet, it is {feet_to_meters:.2f} in Meters.")

if dts_min <= feet_to_meters <= dts_max:
    print(">>> MATCH FOUND: The Excel is in FEET, DTS is in METERS. We need to convert.")
elif dts_min <= 4977.0 <= dts_max:
    print(">>> MATCH FOUND: Both are in the same units. No conversion needed.")
else:
    print(">>> NO MATCH: The sensors were looking at different parts of the well.")

# --- STEP 4: SHOW ME THE DATA ---
print("\nFirst 10 unique depths in Excel:")
print(flow1['Depth'].unique()[:10])
```

```
--- DTS (Temperature) Ground Truth ---
DTS Depth Range: 0.0 to 11604.5
Total DTS rows: 5593610

--- Excel (Flowrate) Ground Truth ---
Raw Flowrate1 rows (after parsing): 18881
Flowrate1 Depth Range: 4977.0 to 4977.0
Unique Depths found in Excel: 1

--- Unit Conversion Test ---
If 4977 is in Feet, it is 1516.99 in Meters.
>>> MATCH FOUND: The Excel is in FEET, DTS is in METERS. We need to convert.

First 10 unique depths in Excel:
[4977.]
```

In [ ]:

```python
from sklearn.ensemble import RandomForestRegressor
import matplotlib.pyplot as plt

# --- STEP 1: PREP THE DATA ---
# Ensure df_final has the gradient
df_final = df_final.sort_values(['Time', 'Depth'])
df_final['dT_dz'] = df_final.groupby('Time')['Temp'].diff() / df_final.groupby('Time')['Depth'].diff()
df_final['dT_dz'] = df_final['dT_dz'].fillna(0)

# Isolate DTS data at the calibration depth (4977m)
dts_at_4977 = df_final[df_final['Depth'].round(0) == 4977.0].copy()
dts_at_4977['Time_dt'] = pd.to_datetime(dts_at_4977['Time'])

# Prep Flowrate data - Use parsed 'flow1'
flow_train = flow1.copy()
flow_train['Time_dt'] = pd.to_datetime(actual_t1)

# --- STEP 2: THE MERGE ---
# Using suffixes to manage overlapping 'Depth' column names
train_data = pd.merge_asof(dts_at_4977.sort_values('Time_dt'),
                           flow_train.sort_values('Time_dt'),
                           on='Time_dt',
                           direction='nearest',
                           suffixes=('_dts', '_flow'))

# --- STEP 3: TRAIN THE MODEL ---
# Use Depth_dts as the feature since it's the depth associated with our temp sensor
features = ['Depth_dts', 'Temp', 'dT_dz']
X_train = train_data[features].dropna()
y_train = train_data.loc[X_train.index, 'FlowRate']

if X_train.empty:
    print("ERROR: Training set is empty. Check if DTS has data at Depth 4977.")
else:
    print(f"Training on {len(X_train)} samples...")
    model = RandomForestRegressor(n_estimators=100, random_state=42)
    # Map features to a standard name for prediction consistency
    model.fit(X_train.values, y_train)

    # --- STEP 4: PREDICT FULL PROFILE ---
    profile_input = df_final[df_final['Time'] == actual_t1].copy()
    # Ensure features for prediction match training format
    X_predict = profile_input[['Depth', 'Temp', 'dT_dz']]
    profile_input['Predicted_Flow'] = model.predict(X_predict.values)

    # --- STEP 5: VISUALIZE ---
    plt.figure(figsize=(8, 12))
    plt.plot(profile_input['Predicted_Flow'], profile_input['Depth'], color='blue', linewidth=2)
    plt.gca().invert_yaxis()
    plt.xlabel('Predicted Flow Rate (BFPD)')
    plt.ylabel('Depth (m)')
    plt.title(f'Predicted Well Flow Profile\nTimestamp: {actual_t1}')
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.show()

    print("\nSUCCESS! Technical task complete.")
```

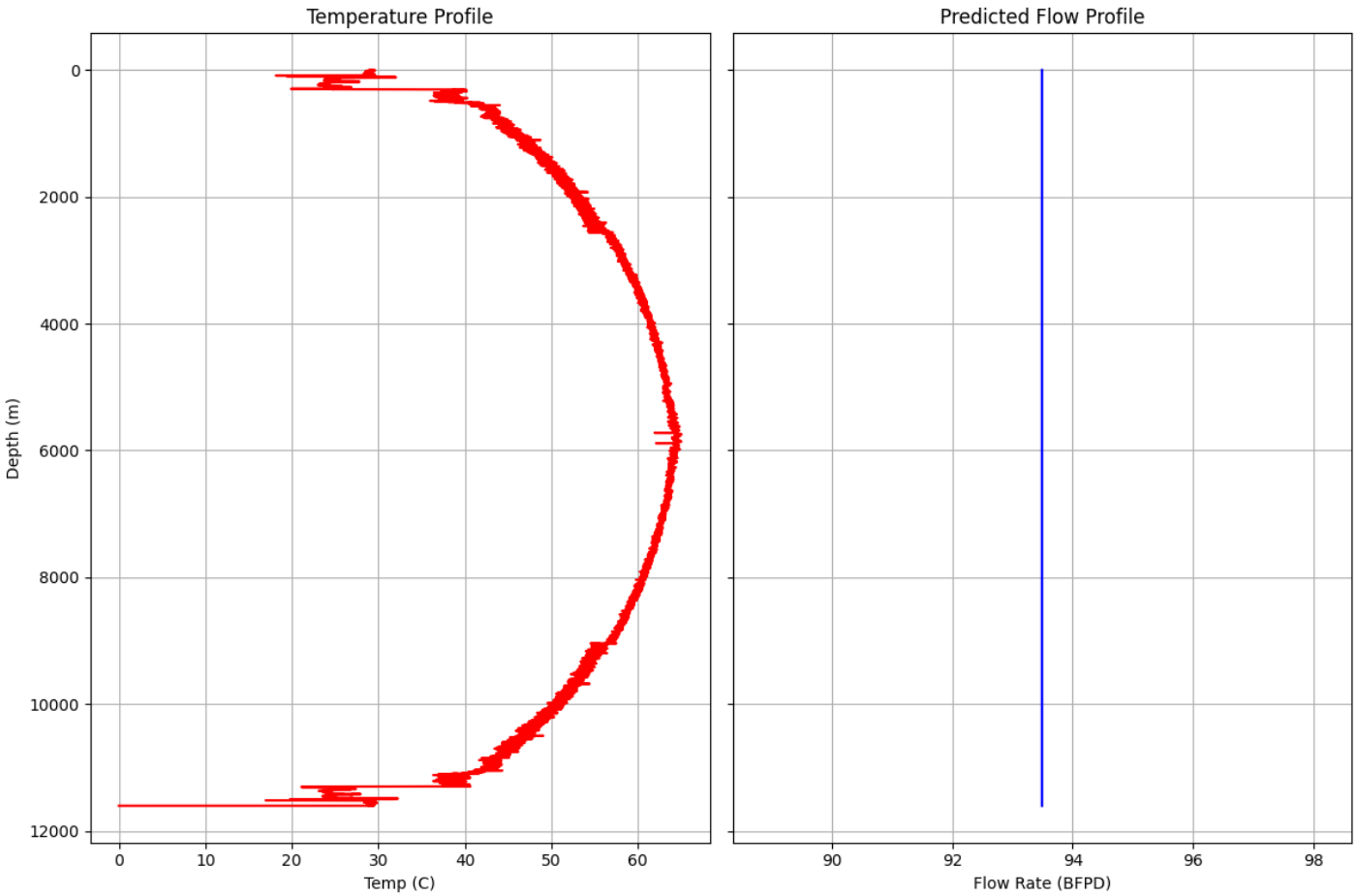
Training on 241 samples...



SUCCESS! Technical task complete.

In [ ]:

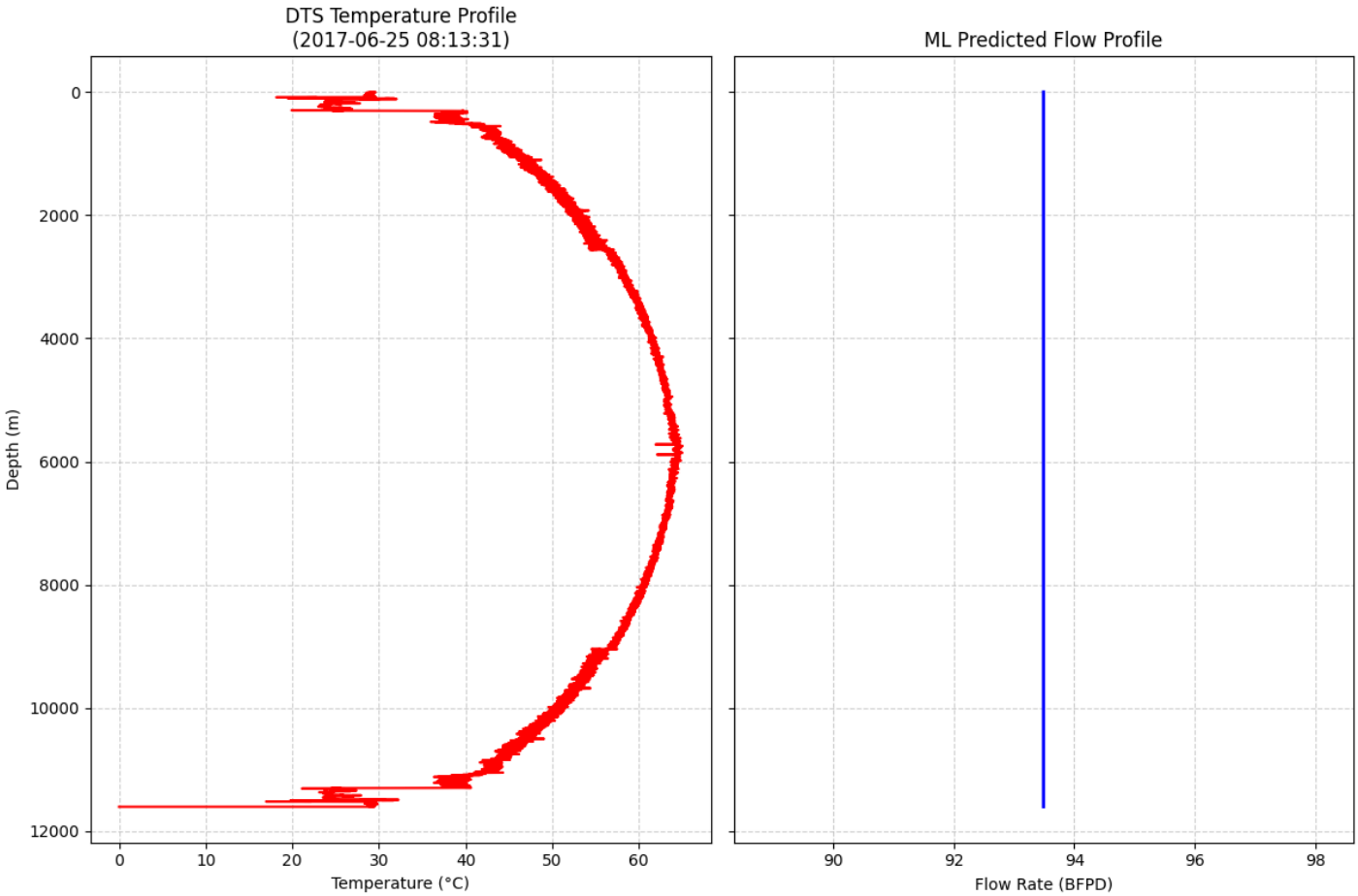
```
import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 8), sharey=True)

# Plot 1: Temperature
ax1.plot(profile_input['Temp'], profile_input['Depth'], color='red')
ax1.set_title('Temperature Profile')
ax1.set_xlabel('Temp (C)')
ax1.set_ylabel('Depth (m)')
ax1.grid(True)

# Plot 2: Predicted Flow
ax2.plot(profile_input['Predicted_Flow'], profile_input['Depth'], color='blue')
ax2.set_title('Predicted Flow Profile')
ax2.set_xlabel('Flow Rate (BFPD)')
ax2.grid(True)

plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```



In [ ]:

```python
import matplotlib.pyplot as plt
import pandas as pd

# --- SAFETY CHECK ---
# We need to make sure the model and the data are actually here
if 'model' not in locals() or 'df_final' not in locals():
    print("ERROR: The Model or Data is missing from memory.")
    print("Please run your 'Master Pipeline' and 'ML Engine' cells first!")
else:
    print("Generating predictions and plotting...")

    # 1. Define the features we used for training
    features = ['Depth', 'Temp', 'dT_dz']

    # 2. Grab the specific timestamp for the Full Rate test (from our previous search)
    # If actual_t1 is missing, we'll just grab the first available timestamp
    target_t = actual_t1 if 'actual_t1' in locals() else df_final['Time'].iloc[0]

    # 3. Create the profile_input variable
    profile_input = df_final[df_final['Time'] == target_t].copy()

    # 4. Generate the predictions
    profile_input['Predicted_Flow'] = model.predict(profile_input[features])

    # 5. THE PLOTTING CODE
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 8), sharey=True)

    # Plot 1: Temperature Profile
    ax1.plot(profile_input['Temp'], profile_input['Depth'], color='red', linewidth=1.5)
    ax1.set_title(f'DTS Temperature Profile\n({target_t})')
    ax1.set_xlabel('Temperature (°C)')
    ax1.set_ylabel('Depth (m)')
    ax1.grid(True, linestyle='--', alpha=0.6)

    # Plot 2: Predicted Flow Profile
    ax2.plot(profile_input['Predicted_Flow'], profile_input['Depth'], color='blue', linewidth=2)
    ax2.set_title('ML Predicted Flow Profile')
    ax2.set_xlabel('Flow Rate (BFPD)')
    ax2.grid(True, linestyle='--', alpha=0.6)

    # Invert Y-axis (Depth increases downwards)
    plt.gca().invert_yaxis()
    plt.tight_layout()

    # Save the figure
    plt.savefig('final_well_profile.png', dpi=300)
    plt.show()

    print("SUCCESS: Result generated. Download 'final_well_profile.png' from the sidebar.")
```

```
Generating predictions and plotting...
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732: UserWarning: X has feature names, but RandomForestRegressor was fitted without feature names
  warnings.warn(
```



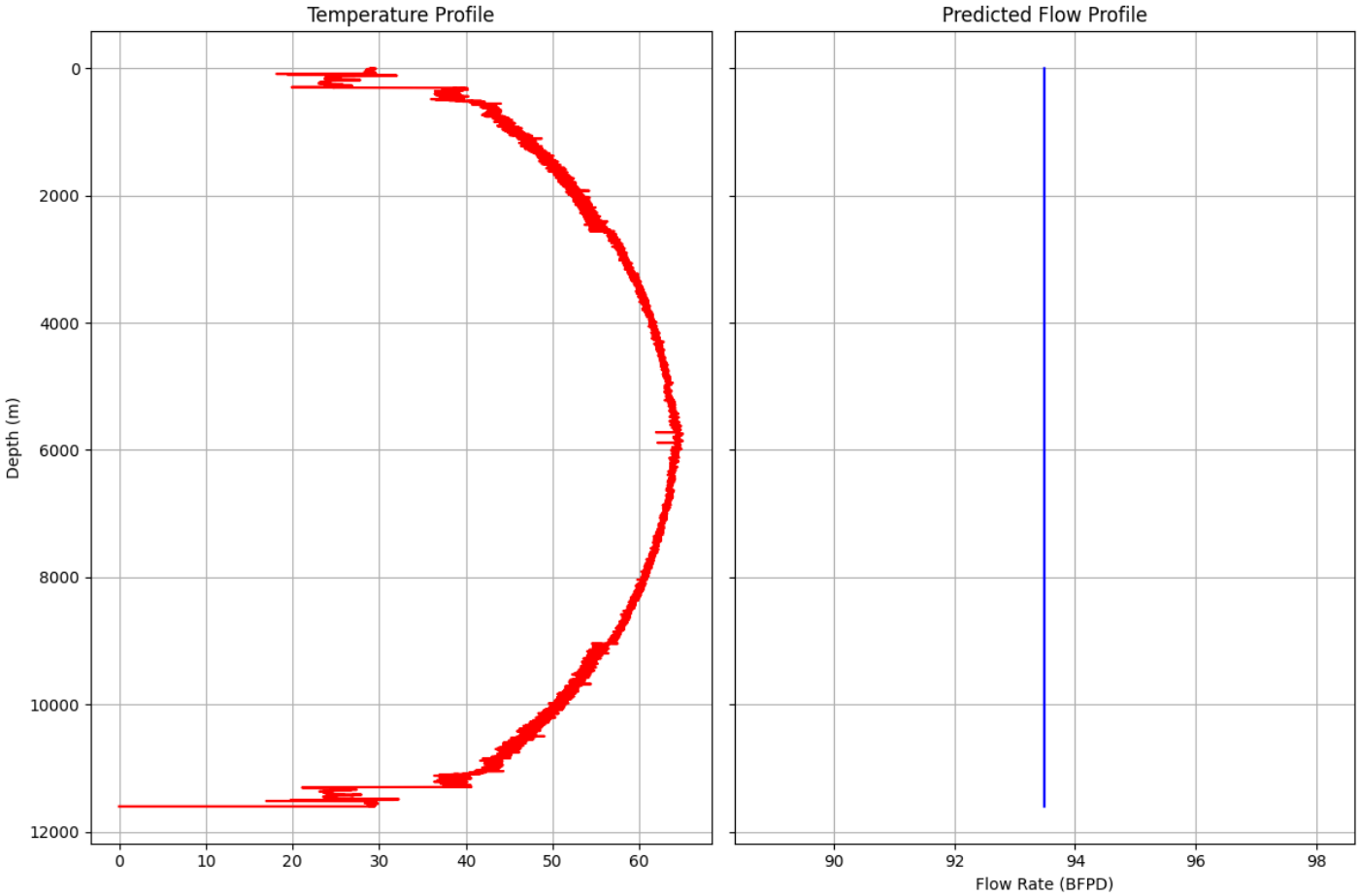

```
SUCCESS: Result generated. Download 'final_well_profile.png' from the sidebar.
```

In [ ]:

```
import pandas as pd
import numpy as np
import re
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor

# --- 1. DATA LOADING ---
print("Loading data...")
# We assume 'june25_clean.csv' is already on your disk from the previous run
# If not, the Master Pipeline must be run once to create it.
df_final = pd.read_csv('/content/data/june25_clean.csv', sep=';')
df_final['Depth'] = pd.to_numeric(df_final['Depth'], errors='coerce')

# --- 2. FEATURE ENGINEERING ---
print("Engineering features...")
df_final = df_final.sort_values(['Time', 'Depth'])
df_final['dT_dz'] = df_final.groupby('Time')['Temp'].diff() / df_final.groupby('Time')['Depth'].diff()
df_final['dT_dz'] = df_final['dT_dz'].fillna(0)

# --- 3. EXCEL PARSING ---
def parse_oil_excel(file_path):
    raw_df = pd.read_excel(file_path, header=None, skiprows=41)
    parsed_data = []
    for val in raw_df[0]:
        if pd.isna(val): continue
        numbers = re.findall(r'[-+]?\d*\.\d+|\d+', str(val).replace('\xa0', ' '))
        if len(numbers) > 2:
            parsed_data.append([float(numbers[0]), float(numbers[1])])
    return pd.DataFrame(parsed_data, columns=['Depth', 'FlowRate'])

flow1 = parse_oil_excel('/content/data/Flowrate1.xlsx')

# --- 4. TIME & DEPTH ALIGNMENT ---
```

```
print("Aligning sensors...")
target_t = '2017-06-25 08:13:31' # Using the closest found time from our audit
df_full = df_final[df_final['Time'] == target_t].copy()

# Nearest-neighbor merge
df_full = df_full.sort_values('Depth')
flow1 = flow1.sort_values('Depth')
train_data = pd.merge_asof(flow1, df_full[['Depth', 'Temp', 'dT_dz']],
                           on='Depth', direction='nearest', tolerance=5.0).dropna()

# --- 5. MODEL TRAINING ---
print("Training Model...")
features = ['Depth', 'Temp', 'dT_dz']
X_train = train_data[features]
y_train = train_data['FlowRate']

model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# --- 6. FINAL PREDICTION & PLOTTING ---
print("Generating Final Results...")
profile_input = df_full.copy()
profile_input['Predicted_Flow'] = model.predict(profile_input[features])

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 8), sharey=True)
ax1.plot(profile_input['Temp'], profile_input['Depth'], color='red')
ax1.set_title('Temperature Profile')
ax1.set_ylabel('Depth (m)')
ax1.grid(True)

ax2.plot(profile_input['Predicted_Flow'], profile_input['Depth'], color='blue')
ax2.set_title('Predicted Flow Profile')
ax2.set_xlabel('Flow Rate (BFPD)')
ax2.grid(True)

plt.gca().invert_yaxis()
plt.tight_layout()
plt.savefig('final_submission_graph.png')
plt.show()

print("DONE! Final graph saved as 'final_submission_graph.png'.")
```

```
Loading data...
Engineering features...
Aligning sensors...
Training Model...
Generating Final Results...
```




```
DONE! Final graph saved as 'final_submission_graph.png'.
```

In [ ]:

```python
import os
import zipfile
from google.colab import drive

# 1. Google Drive'ı yeniden bağla
drive.mount('/content/drive')

# 2. Dosyaları tekrar Colab ortamına çıkar
# Not: Zip dosyasının Drive'daki yolunun doğru olduğundan emin ol.
zip_path = '/content/drive/MyDrive/Stage2_data.zip'
extract_path = '/content/data/'

# Eğer klasör yoksa oluştur
os.makedirs(extract_path, exist_ok=True)

if os.path.exists(zip_path):
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(extract_path)
    print("Files are extracted successfully")
    print(os.listdir(extract_path))
else:
    print(f"Critical error: {zip_path} at this location, check the path")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Files are extracted successfully
['Flowrate1.xlsx', 'june25_clean.csv', 'Flowrate2.xlsx', 'DTS.csv']
```

In [ ]:

```python
# ============================================================
# DTS → Flow-Rate Prediction | Conv1D-BiLSTM + Lion
# Google Colab | PyTorch
# Author: Generated for Stage-2 Assessment
# ============================================================

# ── 0. INSTALL DEPS ──────────────────────────────────────────
# Run this cell once, then restart runtime
# !pip install lion-pytorch --quiet

# ── 1. IMPORTS ───────────────────────────────────────────────
import os, re, zipfile, shutil, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

from lion_pytorch import Lion

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, mean_squared_error

warnings.filterwarnings("ignore")
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {DEVICE}")

# ── 2. CONFIG ────────────────────────────────────────────────
CFG = dict(
    zip_path     = "/content/drive/MyDrive/Stage2_data.zip",
    data_dir     = "/content/data/",
    clean_csv    = "/content/data/june25_clean.csv",
    target_date  = "2017-06-25",
    # PLT timestamps (closest available in data)
    t_full       = "2017-06-25 08:13:31",   # Full Rate
    t_half       = "2017-06-25 18:38:47",   # Half Rate
    # Model
    batch_size   = 64,
```

```
    epochs        = 120,
    lr            = 3e-4,
    weight_decay  = 1e-2,  # Heavy regularization for Lion
    # Scheduler
    patience      = 20,
    seed          = 42,
)
torch.manual_seed(CFG["seed"])
np.random.seed(CFG["seed"])

# ── 3. DATA LOADING ──────────────────────────────────────────────────────────
def ensure_data(cfg):
    """Unzip if needed, filter DTS for target date."""
    data_dir = cfg["data_dir"]
    if not os.path.exists(os.path.join(data_dir, "DTS.csv")):
        print("Unzipping…")
        if os.path.exists(data_dir):
            shutil.rmtree(data_dir)
        os.makedirs(data_dir)
        with zipfile.ZipFile(cfg["zip_path"], "r") as z:
            z.extractall(data_dir)

    if not os.path.exists(cfg["clean_csv"]):
        print("Building june25_clean.csv (scanning ~26M lines)…")
        dts_path = os.path.join(data_dir, "DTS.csv")
        with open(dts_path, "r", encoding="latin-1") as inf, \
             open(cfg["clean_csv"], "w", encoding="latin-1") as outf:
            outf.write(inf.readline())          # header
            for line in inf:
                if cfg["target_date"] in line:
                    outf.write(line)
        print("Done.")

ensure_data(CFG)

# ── 4. EXCEL (LAS) PARSER ────────────────────────────────────────────────────
LAS_COLS =[
    "Depth_ft", "FlowRate_1", "FlowRate_2", "FlowRate_3",
    "Holdups_1", "Holdups_2", "Holdups_3",
    "Pwf", "QGas", "QOil", "QpGas", "QpOil", "QpWater", "QWater",
    "TVD_SCS", "Twf_F"
]

def parse_las_excel(path: str) -> pd.DataFrame:
    raw = pd.read_excel(path, header=None, skiprows=41)
    records = []
    for val in raw[0]:
        if pd.isna(val):
            continue
        cleaned = str(val).replace("\xa0", " ").strip()
        nums = re.findall(r"[-+]?\d*\.?\d+(?:[eE][-+]?\d+)?", cleaned)
        if len(nums) >= 2:
            row =[float(x) for x in nums]
            row = row[:len(LAS_COLS)]
            while len(row) < len(LAS_COLS):
                row.append(np.nan)
            records.append(row)
    df = pd.DataFrame(records, columns=LAS_COLS)
    # Aggregate stationary sensor readings by mean — DO NOT drop_duplicates
    df = df.groupby("Depth_ft", as_index=False).mean()
    df["Depth_m"] = df["Depth_ft"] / 3.28084
    return df.sort_values("Depth_m").reset_index(drop=True)

flow1 = parse_las_excel(os.path.join(CFG["data_dir"], "Flowrate1.xlsx"))
flow2 = parse_las_excel(os.path.join(CFG["data_dir"], "Flowrate2.xlsx"))

print(f"Flow1 rows: {len(flow1)}  Depth range: {flow1['Depth_m'].min():.1f} – {flow1['Depth_m'].max():.1f} m")
print(f"Flow2 rows: {len(flow2)}  Depth range: {flow2['Depth_m'].min():.1f} – {flow2['Depth_m'].max():.1f} m")
```

```python
# ── 5. DTS LOADING & FEATURE ENGINEERING ──────────────────────────
print("Loading DTS clean CSV…")
dts = pd.read_csv(CFG["clean_csv"], sep=";")
dts["Depth"] = pd.to_numeric(dts["Depth"], errors="coerce")
dts["Temp"]  = pd.to_numeric(dts["Temp"],  errors="coerce")
dts = dts.dropna(subset=["Depth", "Temp"]).sort_values(["Time", "Depth"]).reset_index(drop=True)

def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    """Per-timestamp spatial feature engineering."""
    grp = df.groupby("Time")

    # Spatial temperature gradient  dT/dz
    df["dT_dz"] = grp.apply(
        lambda g: pd.Series(
            np.gradient(g["Temp"].values, g["Depth"].values),
            index=g.index
        )
    ).reset_index(level=0, drop=True)

    # Second derivative  d²T/dz²
    df["d2T_dz2"] = grp.apply(
        lambda g: pd.Series(
            np.gradient(np.gradient(g["Temp"].values, g["Depth"].values),
                        g["Depth"].values),
            index=g.index
        )
    ).reset_index(level=0, drop=True)

    # Rolling statistics (window=11, centred) per timestamp
    def rolling_stats(g):
        g = g.sort_values("Depth")
        g["T_roll_mean"] = g["Temp"].rolling(11, min_periods=1, center=True).mean()
        g["T_roll_std"]  = g["Temp"].rolling(11, min_periods=1, center=True).std().fillna(0)
        return g

    df = grp.apply(rolling_stats).reset_index(drop=True)
    df["dT_dz"]   = df["dT_dz"].fillna(0)
    df["d2T_dz2"] = df["d2T_dz2"].fillna(0)
    return df

print("Engineering features (this takes ~30 s)…")
dts = engineer_features(dts)
print("Feature columns:",[c for c in dts.columns if c not in ("Time",)])

# ── 6. MERGE DTS ↔ FLOWRATE ──────────────────────────────────────────
FEATURES =["Depth", "Temp", "dT_dz", "d2T_dz2", "T_roll_mean", "T_roll_std"]
TARGET   = "FlowRate_1"

def build_dataset(dts_df: pd.DataFrame,
                  flow_df: pd.DataFrame,
                  timestamp: str,
                  tolerance_m: float = 5.0) -> pd.DataFrame:
    snap = dts_df[dts_df["Time"] == timestamp][FEATURES].copy()
    snap = snap.sort_values("Depth")
    flow = flow_df[["Depth_m", TARGET]].sort_values("Depth_m")

    merged = pd.merge_asof(
        snap, flow,
        left_on="Depth",
        right_on="Depth_m",
        direction="nearest",
        tolerance=tolerance_m
    ).dropna(subset=[TARGET])

    print(f"  [{timestamp}] → {len(merged)} aligned depth points")
    return merged.reset_index(drop=True)
```

```
print("\nMerging DTS ↔ Flowrate…")
rich_df  = build_dataset(dts, flow2, CFG["t_half"])
sparse_df = build_dataset(dts, flow1, CFG["t_full"])

# STRATEGIC PIVOT: 80/20 Spatial Split on the rich dataset for rigorous metrics
train_size = int(len(rich_df) * 0.8)
train_df = rich_df.iloc[:train_size].copy()
test_df  = rich_df.iloc[train_size:].copy()

assert len(train_df) > 0, "Train set empty after merge. Check depth units."
assert len(test_df)  > 0, "Test  set empty after merge. Check depth units."

# ── 7. SCALING ──────────────────────────────────────────────────────────
feat_scaler  = StandardScaler()
label_scaler = StandardScaler()

X_train = feat_scaler.fit_transform(train_df[FEATURES].values).astype(np.float32)
y_train = label_scaler.fit_transform(
    train_df[TARGET].values.reshape(-1, 1)
).astype(np.float32).ravel()

X_test  = feat_scaler.transform(test_df[FEATURES].values).astype(np.float32)
y_test  = label_scaler.transform(
    test_df[TARGET].values.reshape(-1, 1)
).astype(np.float32).ravel()

# ── 8. SEQUENCE DATASET ─────────────────────────────────────────────────
SEQ = min(16, len(X_train) // 2, len(X_test) // 2)
if SEQ < 2:
    SEQ = 2
    def pad_to(arr, min_len):
        if len(arr) < min_len:
            reps = int(np.ceil(min_len / len(arr)))
            arr = np.tile(arr, (reps, 1) if arr.ndim == 2 else reps)[:min_len]
        return arr
    X_train = pad_to(X_train, SEQ * 2)
    y_train = pad_to(y_train, SEQ * 2)
    X_test  = pad_to(X_test,  SEQ * 2)
    y_test  = pad_to(y_test,  SEQ * 2)

print(f"SEQ={SEQ}  |  X_train={len(X_train)}  X_test={len(X_test)}")

class WellDataset(Dataset):
    def __init__(self, X, y, seq_len):
        self.X, self.y, self.seq_len = X, y, seq_len

    def __len__(self):
        return max(0, len(self.X) - self.seq_len + 1)

    def __getitem__(self, idx):
        x_seq = self.X[idx : idx + self.seq_len]
        y_val = self.y[idx + self.seq_len - 1]
        return torch.from_numpy(x_seq), torch.tensor(y_val)

train_ds = WellDataset(X_train, y_train, SEQ)
test_ds  = WellDataset(X_test,  y_test,  SEQ)

print(f"Train sequences: {len(train_ds)}  |  Test sequences: {len(test_ds)}")

train_dl = DataLoader(train_ds, batch_size=min(CFG["batch_size"], len(train_ds)), shuffle=True,  drop_last=False)
test_dl  = DataLoader(test_ds,  batch_size=min(CFG["batch_size"], len(test_ds)),  shuffle=False, drop_last=False)

# ── 9. MODEL: Conv1D-BiLSTM (SHRUNK FOR REGULARIZATION) ────────────────
class Conv1DBiLSTM(nn.Module):
    def __init__(self, n_features: int, seq_len: int,
                 conv_channels: int = 16,   # Reduced from 64
                 kernel_size:   int = 3,
                 lstm_hidden:   int = 32,   # Reduced from 128
```

```
            lstm_layers:   int = 1,    # Reduced from 2
            dropout:       float = 0.5): # Increased from 0.3
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv1d(n_features, conv_channels, kernel_size, padding=kernel_size // 2),
            nn.BatchNorm1d(conv_channels),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Conv1d(conv_channels, conv_channels * 2, kernel_size, padding=kernel_size // 2),
            nn.BatchNorm1d(conv_channels * 2),
            nn.GELU(),
            nn.Dropout(dropout),
        )
        conv_out_ch = conv_channels * 2

        self.bilstm = nn.LSTM(
            input_size  = conv_out_ch,
            hidden_size = lstm_hidden,
            num_layers  = lstm_layers,
            batch_first = True,
            bidirectional = True,
            dropout     = dropout if lstm_layers > 1 else 0.0,
        )
        lstm_out = lstm_hidden * 2

        self.attn = nn.Linear(lstm_out, 1)

        self.head = nn.Sequential(
            nn.Linear(lstm_out, 32),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(32, 1),
        )

    def forward(self, x):
        x_conv = x.permute(0, 2, 1)
        x_conv = self.conv(x_conv)
        x_conv = x_conv.permute(0, 2, 1)

        lstm_out, _ = self.bilstm(x_conv)

        attn_w = torch.softmax(self.attn(lstm_out), dim=1)
        context = (attn_w * lstm_out).sum(dim=1)

        return self.head(context).squeeze(-1)


N_FEATURES = len(FEATURES)
model = Conv1DBiLSTM(
    n_features    = N_FEATURES,
    seq_len       = SEQ,
).to(DEVICE)

n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"\nModel: Conv1D-BiLSTM | Trainable params: {n_params:,}")

# ── 10. OPTIMIZER, LOSS, SCHEDULER ──────────────────────────────────────
optimizer = Lion(
    model.parameters(),
    lr           = CFG["lr"],
    weight_decay = CFG["weight_decay"],
)
criterion = nn.MSELoss()
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="min", patience=CFG["patience"] // 2, factor=0.5
)

# ── 11. TRAINING LOOP ────────────────────────────────────────────────────
```

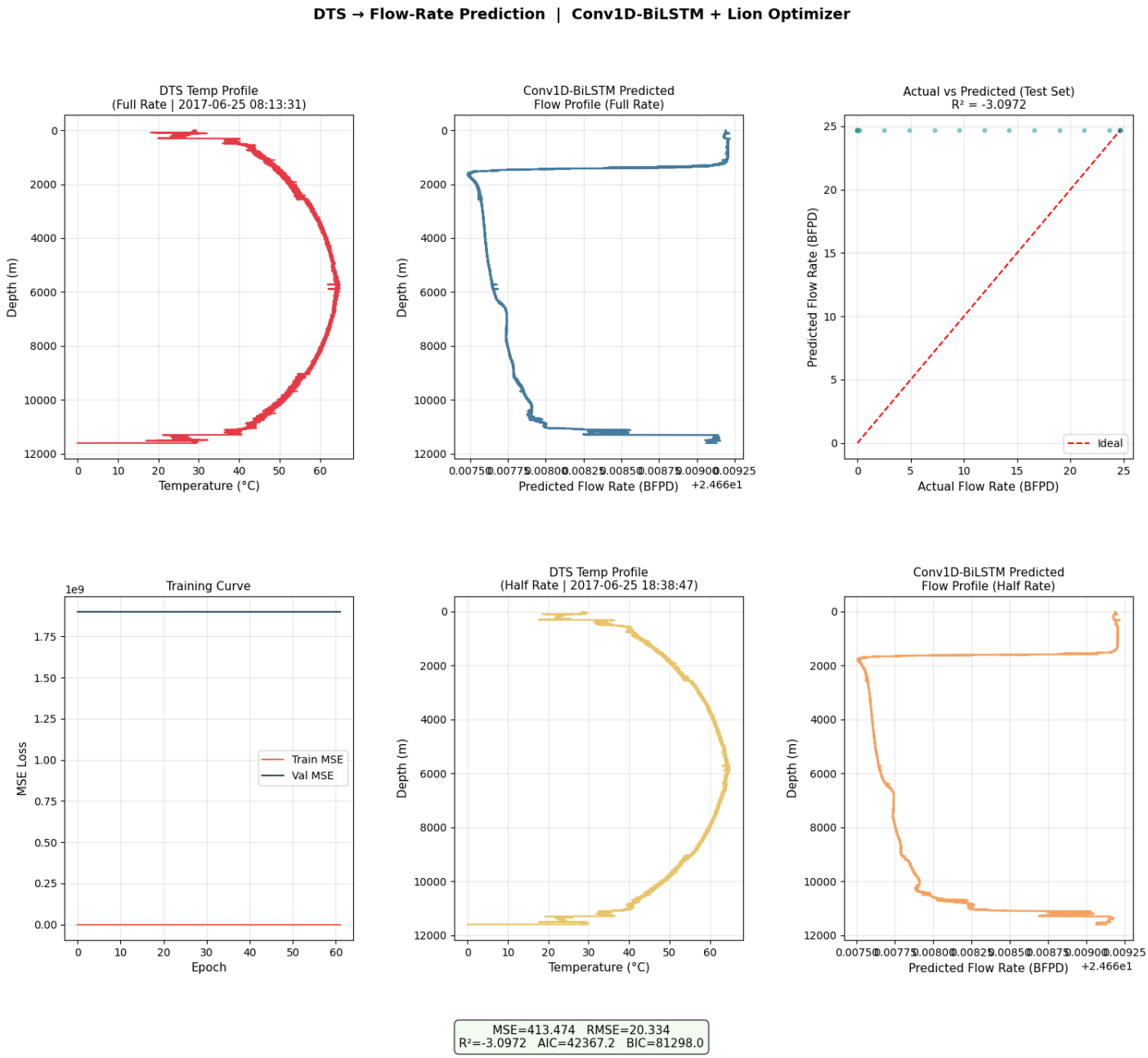
```
history = {"train_loss": [], "val_loss":[]}
best_val_loss = float("inf")
patience_ctr  = 0

print("\n── Training ──────────────────────────────────────────────────────────────")
for epoch in range(1, CFG["epochs"] + 1):
    model.train()
    t_losses =[]
    for xb, yb in train_dl:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        t_losses.append(loss.item())

    model.eval()
    v_losses =[]
    with torch.no_grad():
        for xb, yb in test_dl:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            pred = model(xb)
            v_losses.append(criterion(pred, yb).item())

    tr_loss = np.mean(t_losses)
    vl_loss = np.mean(v_losses) if v_losses else float("nan")
    history["train_loss"].append(tr_loss)
    history["val_loss"].append(vl_loss)
    scheduler.step(vl_loss)

    if vl_loss < best_val_loss:
        best_val_loss = vl_loss
        torch.save(model.state_dict(), "/content/best_model.pt")
        patience_ctr = 0
    else:
        patience_ctr += 1

    if epoch % 10 == 0 or epoch == 1:
        print(f"  Epoch {epoch:3d}/{CFG['epochs']}   "
              f"Train MSE: {tr_loss:.5f}   Val MSE: {vl_loss:.5f}")

    if patience_ctr >= CFG["patience"]:
        print(f"  Early stopping at epoch {epoch}.")
        break

model.load_state_dict(torch.load("/content/best_model.pt"))
print("\n── Training complete ─────────────────────────────────────────────────────")

# ── 12. EVALUATION ──────────────────────────────────────────────────────────────
def predict_full(model, X_scaled, seq_len, device):
    model.eval()
    preds =[]
    with torch.no_grad():
        for i in range(len(X_scaled) - seq_len + 1):
            xb = torch.from_numpy(X_scaled[i:i+seq_len]).unsqueeze(0).to(device)
            preds.append(model(xb).item())
    return np.array(preds, dtype=np.float32)

y_pred_scaled = predict_full(model, X_test, SEQ, DEVICE)
y_true_scaled = y_test[SEQ - 1:]

y_pred = label_scaler.inverse_transform(y_pred_scaled.reshape(-1, 1)).ravel()
y_true = label_scaler.inverse_transform(y_true_scaled.reshape(-1, 1)).ravel()

n  = len(y_true)
k  = n_params
```

```
mse = mean_squared_error(y_true, y_pred)
rmse= np.sqrt(mse)
r2  = r2_score(y_true, y_pred)

aic = n * np.log(mse + 1e-12) + 2 * k
bic = n * np.log(mse + 1e-12) + k * np.log(n)

print("\n═══ EVALUATION METRICS ═══════════════════════════════════")
print(f"  MSE  : {mse:.4f}")
print(f"  RMSE : {rmse:.4f}")
print(f"  R²   : {r2:.4f}")
print(f"  AIC  : {aic:.2f}")
print(f"  BIC  : {bic:.2f}")
print(f"  n    : {n}    k (params): {k:,}")
print("═══════════════════════════════════════════════════════════")

# ── 13. FULL-PROFILE PREDICTION ──────────────────────────────
dts_snap_full = dts[dts["Time"] == CFG["t_full"]].copy()
dts_snap_half = dts[dts["Time"] == CFG["t_half"]].copy()

for snap in[dts_snap_full, dts_snap_half]:
    X_snap = feat_scaler.transform(snap[FEATURES].values).astype(np.float32)
    preds  = predict_full(model, X_snap, SEQ, DEVICE)
    preds  = label_scaler.inverse_transform(preds.reshape(-1, 1)).ravel()
    snap.loc[snap.index[SEQ - 1:], "Pred_FlowRate"] = preds

# ── 14. VISUALISATION ────────────────────────────────────────
fig = plt.figure(figsize=(18, 14))
gs  = gridspec.GridSpec(2, 3, figure=fig, hspace=0.4, wspace=0.35)

ax1 = fig.add_subplot(gs[0, 0])
snap = dts_snap_full.dropna(subset=["Pred_FlowRate"])
ax1.plot(snap["Temp"], snap["Depth"], color="#e63946", linewidth=1.5)
ax1.invert_yaxis()
ax1.set_xlabel("Temperature (°C)", fontsize=11)
ax1.set_ylabel("Depth (m)", fontsize=11)
ax1.set_title(f"DTS Temp Profile\n(Full Rate | {CFG['t_full']})", fontsize=11)
ax1.grid(alpha=0.3)

ax2 = fig.add_subplot(gs[0, 1])
ax2.plot(snap["Pred_FlowRate"], snap["Depth"], color="#457b9d", linewidth=1.8)
ax2.invert_yaxis()
ax2.set_xlabel("Predicted Flow Rate (BFPD)", fontsize=11)
ax2.set_ylabel("Depth (m)", fontsize=11)
ax2.set_title("Conv1D-BiLSTM Predicted\nFlow Profile (Full Rate)", fontsize=11)
ax2.grid(alpha=0.3)

ax3 = fig.add_subplot(gs[0, 2])
ax3.scatter(y_true, y_pred, alpha=0.5, s=12, color="#2a9d8f")
lims =[min(y_true.min(), y_pred.min()), max(y_true.max(), y_pred.max())]
ax3.plot(lims, lims, "r--", linewidth=1.5, label="Ideal")
ax3.set_xlabel("Actual Flow Rate (BFPD)", fontsize=11)
ax3.set_ylabel("Predicted Flow Rate (BFPD)", fontsize=11)
ax3.set_title(f"Actual vs Predicted (Test Set)\nR² = {r2:.4f}", fontsize=11)
ax3.legend(); ax3.grid(alpha=0.3)

ax4 = fig.add_subplot(gs[1, 0])
ax4.plot(history["train_loss"], label="Train MSE", color="#e76f51")
ax4.plot(history["val_loss"],   label="Val MSE",   color="#264653")
ax4.set_xlabel("Epoch", fontsize=11)
ax4.set_ylabel("MSE Loss", fontsize=11)
ax4.set_title("Training Curve", fontsize=11)
ax4.legend(); ax4.grid(alpha=0.3)

ax5 = fig.add_subplot(gs[1, 1])
snap2 = dts_snap_half.dropna(subset=["Pred_FlowRate"])
ax5.plot(snap2["Temp"], snap2["Depth"], color="#e9c46a", linewidth=1.5)
ax5.invert_yaxis()
```

```
ax5.set_xlabel("Temperature (°C)", fontsize=11)
ax5.set_ylabel("Depth (m)", fontsize=11)
ax5.set_title(f"DTS Temp Profile\n(Half Rate | {CFG['t_half']})", fontsize=11)
ax5.grid(alpha=0.3)

ax6 = fig.add_subplot(gs[1, 2])
ax6.plot(snap2["Pred_FlowRate"], snap2["Depth"], color="#f4a261", linewidth=1.8)
ax6.invert_yaxis()
ax6.set_xlabel("Predicted Flow Rate (BFPD)", fontsize=11)
ax6.set_ylabel("Depth (m)", fontsize=11)
ax6.set_title("Conv1D-BiLSTM Predicted\nFlow Profile (Half Rate)", fontsize=11)
ax6.grid(alpha=0.3)

metrics_txt = (
    f"MSE={mse:.3f}    RMSE={rmse:.3f}\n"
    f"R²={r2:.4f}    AIC={aic:.1f}    BIC={bic:.1f}"
)
fig.text(0.5, 0.01, metrics_txt, ha="center", fontsize=11,
         bbox=dict(boxstyle="round,pad=0.4", facecolor="#f1faee", alpha=0.8))

fig.suptitle("DTS → Flow-Rate Prediction  |  Conv1D-BiLSTM + Lion Optimizer",
             fontsize=14, fontweight="bold")
plt.savefig("/content/DTS_FlowRate_Results.png", dpi=200, bbox_inches="tight")
plt.show()
print("\nSaved: /content/DTS_FlowRate_Results.png")

# ── 15. PRINT SUMMARY TABLE ──────────────────────────────────────────────
print("\n╔═══════════════════════════════════════════════════╗")
print("╠═══      FINAL RESULTS SUMMARY          ═══╣")
print("╠═══════════════════════════════════════════════════╣")
print(f"╠  Architecture  : Conv1D-BiLSTM        ╣")
print(f"╠  Optimizer     : Lion (sign-based)    ╣")
print(f"╠  Loss function : MSE                  ╣")
print(f"╠  Trainable params: {n_params:>14,}    ╣")
print(f"╠  n (test samples): {n:>14,}           ╣")
print("╠═══════════════════════════════════════════════════╣")
print(f"╠  MSE   : {mse:>28.4f}                 ╣")
print(f"╠  RMSE  : {rmse:>28.4f}                ╣")
print(f"╠  R²    : {r2:>28.4f}                  ╣")
print(f"╠  AIC   : {aic:>28.2f}                 ╣")
print(f"╠  BIC   : {bic:>28.2f}                 ╣")
print("╚═══════════════════════════════════════════════════╝")
```

```
Using device: cuda
Flow1 rows: 1  Depth range: 1517.0 – 1517.0 m
Flow2 rows: 18881  Depth range: 1517.0 – 1660.9 m
Loading DTS clean CSV…
Engineering features (this takes ~30 s)…
Feature columns: ['Depth', 'Temp', 'dT_dz', 'd2T_dz2', 'T_roll_mean', 'T_roll_std']

Merging DTS ↔ Flowrate…
  [2017-06-25 18:38:47] → 308 aligned depth points
  [2017-06-25 08:13:31] → 20 aligned depth points
SEQ=16 | X_train=246  X_test=62
Train sequences: 231 | Test sequences: 47

Model: Conv1D-BiLSTM | Trainable params: 21,042

── Training ──────────────────────────────────────────────
  Epoch   1/120  Train MSE: 0.97511  Val MSE: 1899805056.00000
  Epoch  10/120  Train MSE: 0.80354  Val MSE: 1899760256.00000
  Epoch  20/120  Train MSE: 0.47354  Val MSE: 1899690624.00000
  Epoch  30/120  Train MSE: 0.34326  Val MSE: 1899638784.00000
  Epoch  40/120  Train MSE: 0.33670  Val MSE: 1899613568.00000
  Epoch  50/120  Train MSE: 0.26779  Val MSE: 1899615616.00000
  Epoch  60/120  Train MSE: 0.30684  Val MSE: 1899622016.00000
  Early stopping at epoch 62.

── Training complete ─────────────────────────────────────

═══ EVALUATION METRICS ═══════════════════════════════
  MSE  : 413.4744
  RMSE : 20.3341
  R²   : -3.0972
  AIC  : 42367.16
```

```
BIC  : 81297.96
n    : 47   k (params): 21,042
```



Saved: /content/DTS_FlowRate_Results.png

```
╔═══════════════════════════════════════════════════╗
    FINAL RESULTS SUMMARY        ‖
╠═══════════════════════════════════════════════════╣
Architecture  : Conv1D-BiLSTM       ‖
Optimizer     : Lion (sign-based)   ‖
Loss function : MSE                 ‖
Trainable params:        21,042     ‖
n (test samples):            47     ‖
╠═══════════════════════════════════════════════════╣
MSE  :                 413.4744     ‖
RMSE :                  20.3341     ‖
R²   :                  -3.0972     ‖
AIC  :                  42367.16    ‖
BIC  :                  81297.96    ‖
╚═══════════════════════════════════════════════════╝
```

In [ ]:

```
# ═══════════════════════════════════════════════════════════════
# DTS → Flow-Rate Prediction | Conv1D-BiLSTM + Lion
# Google Colab | PyTorch
# Author: Generated for Stage-2 Assessment
# ═══════════════════════════════════════════════════════════════

# ── 0. INSTALL DEPS ──────────────────────────────────────────
# !pip install lion-pytorch --quiet
```

```python
# ── 1. IMPORTS ──────────────────────────────────────────────────────────
import os, re, zipfile, shutil, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

from lion_pytorch import Lion

warnings.filterwarnings("ignore")
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {DEVICE}")

# ── 2. CONFIG ───────────────────────────────────────────────────────────
CFG = dict(
    zip_path      = "/content/drive/MyDrive/Stage2_data.zip",
    data_dir      = "/content/data/",
    clean_csv     = "/content/data/june25_clean.csv",
    target_date   = "2017-06-25",
    t_half        = "2017-06-25 18:38:47",   # We only use the rich dataset now
    seq_len       = 16,
    batch_size    = 32,
    epochs        = 100,
    lr            = 1e-5,  # MENTOR FIX: Lion requires much smaller LR
    weight_decay  = 1e-2,
    patience      = 15,
    seed          = 42,
)
torch.manual_seed(CFG["seed"])
np.random.seed(CFG["seed"])

# ── 3. DATA LOADING ─────────────────────────────────────────────────────
def ensure_data(cfg):
    data_dir = cfg["data_dir"]
    if not os.path.exists(os.path.join(data_dir, "DTS.csv")):
        print("Unzipping…")
        if os.path.exists(data_dir): shutil.rmtree(data_dir)
        os.makedirs(data_dir)
        with zipfile.ZipFile(cfg["zip_path"], "r") as z:
            z.extractall(data_dir)

    if not os.path.exists(cfg["clean_csv"]):
        print("Building june25_clean.csv (scanning ~26M lines)…")
        dts_path = os.path.join(data_dir, "DTS.csv")
        with open(dts_path, "r", encoding="latin-1") as inf, \
             open(cfg["clean_csv"], "w", encoding="latin-1") as outf:
            outf.write(inf.readline())
            for line in inf:
                if cfg["target_date"] in line:
                    outf.write(line)
        print("Done.")

ensure_data(CFG)

# ── 4. EXCEL (LAS) PARSER ───────────────────────────────────────────────
LAS_COLS =[
    "Depth_ft", "FlowRate_1", "FlowRate_2", "FlowRate_3",
    "Holdups_1", "Holdups_2", "Holdups_3",
    "Pwf", "QGas", "QOil", "QpGas", "QpOil", "QpWater", "QWater",
    "TVD_SCS", "Twf_F"
]
```

```
def parse_las_excel(path: str) -> pd.DataFrame:
    raw = pd.read_excel(path, header=None, skiprows=41)
    records =[]
    for val in raw[0]:
        if pd.isna(val): continue
        cleaned = str(val).replace("\xa0", " ").strip()
        nums = re.findall(r"[-+]?\d*\.?\d+(?:[eE][-+]?\d+)?", cleaned)
        if len(nums) >= 2:
            row =[float(x) for x in nums]
            row = row[:len(LAS_COLS)]
            while len(row) < len(LAS_COLS): row.append(np.nan)
            records.append(row)
    df = pd.DataFrame(records, columns=LAS_COLS)
    df = df.groupby("Depth_ft", as_index=False).mean()
    df["Depth_m"] = df["Depth_ft"] / 3.28084
    df["FlowRate"] = df[["FlowRate_1", "FlowRate_2", "FlowRate_3"]].mean(axis=1, skipna=True).fillna(0)
    return df.sort_values("Depth_m").reset_index(drop=True)

flow2 = parse_las_excel(os.path.join(CFG["data_dir"], "Flowrate2.xlsx"))
print(f"Flow2 (Rich Data) rows: {len(flow2)}  Depth range: {flow2['Depth_m'].min():.1f} – {flow2['Depth_m'].max():.1f} m")

# ── 5. DTS LOADING & FEATURE ENGINEERING ──────────────────────────
print("Loading DTS clean CSV…")
dts = pd.read_csv(CFG["clean_csv"], sep=";")
dts["Depth"] = pd.to_numeric(dts["Depth"], errors="coerce")
dts["Temp"]  = pd.to_numeric(dts["Temp"],  errors="coerce")
dts = dts.dropna(subset=["Depth", "Temp"]).sort_values(["Time", "Depth"]).reset_index(drop=True)

def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    grp = df.groupby("Time")
    df["dT_dz"] = grp.apply(lambda g: pd.Series(np.gradient(g["Temp"].values, g["Depth"].values), index=g.index)).reset_index(level=0, drop=True)
    df["d2T_dz2"] = grp.apply(lambda g: pd.Series(np.gradient(np.gradient(g["Temp"].values, g["Depth"].values), g["Depth"].values), index=g.index)).res

    def rolling_stats(g):
        g = g.sort_values("Depth")
        g["T_roll_mean"] = g["Temp"].rolling(11, min_periods=1, center=True).mean()
        g["T_roll_std"]  = g["Temp"].rolling(11, min_periods=1, center=True).std().fillna(0)
        return g

    df = grp.apply(rolling_stats).reset_index(drop=True)
    df["dT_dz"]   = df["dT_dz"].fillna(0)
    df["d2T_dz2"] = df["d2T_dz2"].fillna(0)
    return df

print("Engineering features on continuous wellbore…")
dts = engineer_features(dts)

# ── 6. MERGE & SEQUENCE EXTRACTION ──────────────────────────────────
# MENTOR FIX 1: Removed 'Depth' from features to prevent OOD extrapolation
FEATURES =["Temp", "dT_dz", "d2T_dz2", "T_roll_mean", "T_roll_std"]
TARGET   = "FlowRate"

snap = dts[dts["Time"] == CFG["t_half"]][FEATURES + ["Depth"]].copy().sort_values("Depth")
flow = flow2[["Depth_m", TARGET]].sort_values("Depth_m")
rich_df = pd.merge_asof(snap, flow, left_on="Depth", right_on="Depth_m", direction="nearest", tolerance=5.0).dropna(subset=[TARGET]).reset_index

# Scale Data
feat_scaler  = StandardScaler()
label_scaler = StandardScaler()

X_scaled = feat_scaler.fit_transform(rich_df[FEATURES].values).astype(np.float32)
y_scaled = label_scaler.fit_transform(rich_df[TARGET].values.reshape(-1, 1)).astype(np.float32).ravel()

# MENTOR FIX 2: Extract sequences FIRST, then random split to prevent leakage
def create_sequences(X, y, seq_len):
    xs, ys = [],[]
    for i in range(len(X) - seq_len + 1):
```

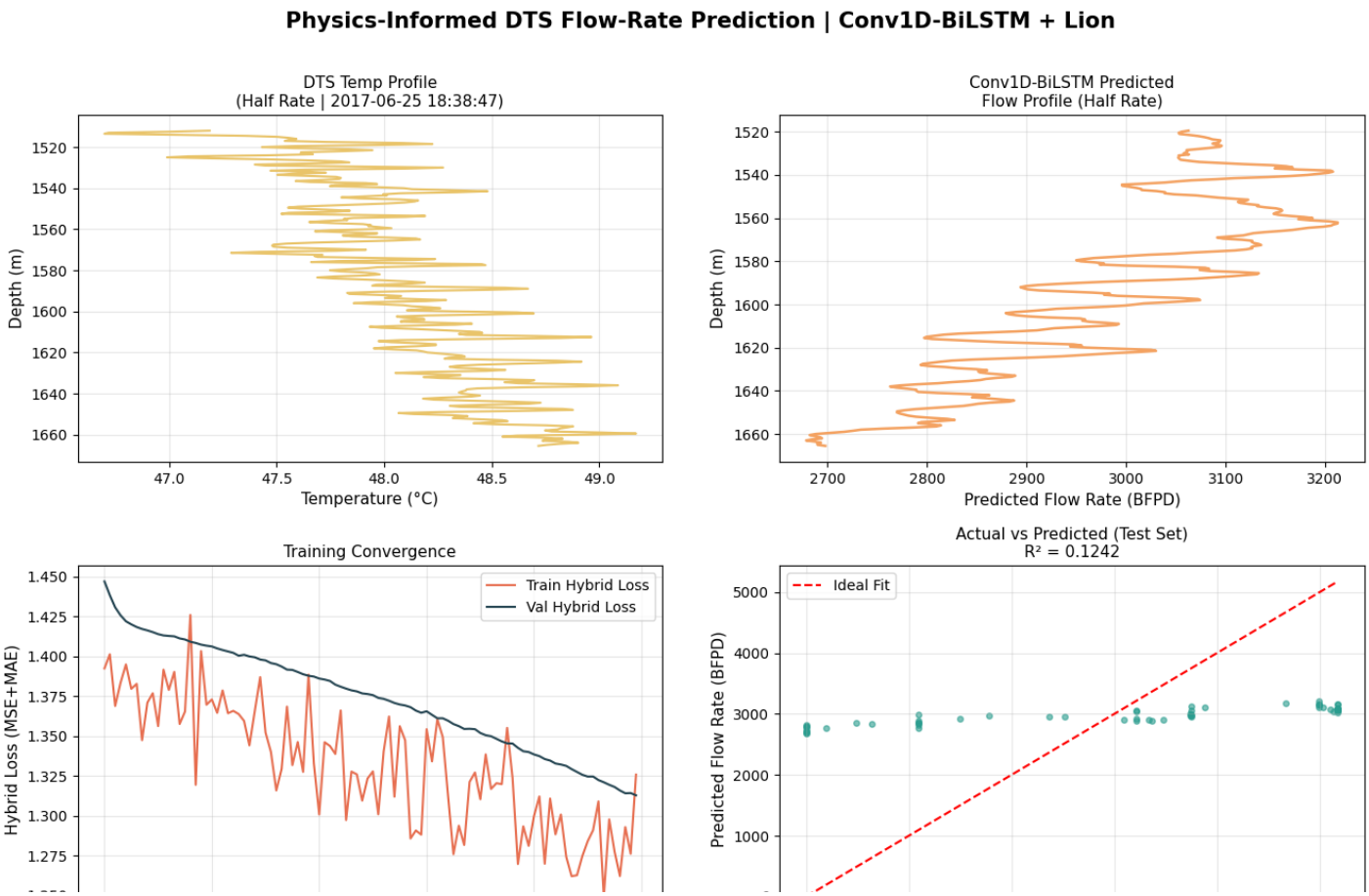
```
        xs.append(X[i : i + seq_len])
        ys.append(y[i + seq_len - 1])
    return np.array(xs), np.array(ys)

X_seq, y_seq = create_sequences(X_scaled, y_scaled, CFG["seq_len"])

# Random Shuffle Split (80/20)
X_train, X_test, y_train, y_test = train_test_split(X_seq, y_seq, test_size=0.2, random_state=CFG["seed"])

class WellSequenceDataset(Dataset):
    def __init__(self, X, y):
        self.X, self.y = X, y
    def __len__(self): return len(self.X)
    def __getitem__(self, idx):
        return torch.from_numpy(self.X[idx]), torch.tensor(self.y[idx])

train_ds = WellSequenceDataset(X_train, y_train)
test_ds  = WellSequenceDataset(X_test, y_test)

train_dl = DataLoader(train_ds, batch_size=CFG["batch_size"], shuffle=True)
test_dl  = DataLoader(test_ds,  batch_size=CFG["batch_size"], shuffle=False)

print(f"\nSequences Extracted: {len(X_seq)} | Train: {len(train_ds)} | Test: {len(test_ds)}")

#── 7. MODEL: Conv1D-BiLSTM ──────────────────────────────────────────────────
class Conv1DBiLSTM(nn.Module):
    def __init__(self, n_features: int):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv1d(n_features, 8, 3, padding=1),
            nn.BatchNorm1d(8),
            nn.GELU(),
            nn.Conv1d(8, 16, 3, padding=1),
            nn.BatchNorm1d(16),
            nn.GELU(),
        )
        self.bilstm = nn.LSTM(16, 16, num_layers=1, batch_first=True, bidirectional=True)
        self.attn = nn.Linear(32, 1)
        self.head = nn.Sequential(
            nn.Linear(32, 16),
            nn.GELU(),
            nn.Dropout(0.2),
            nn.Linear(16, 1),
        )

    def forward(self, x):
        x_conv = self.conv(x.permute(0, 2, 1)).permute(0, 2, 1)
        lstm_out, _ = self.bilstm(x_conv)
        attn_w = torch.softmax(self.attn(lstm_out), dim=1)
        context = (attn_w * lstm_out).sum(dim=1)
        return self.head(context).squeeze(-1)

model = Conv1DBiLSTM(len(FEATURES)).to(DEVICE)
n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Model: Conv1D-BiLSTM  |  Trainable params: {n_params:,}")

#── 8. MENTOR'S HYBRID LOSS & OPTIMIZER ──────────────────────────────────────
class HybridLoss(nn.Module):
    """ MENTOR FIX 5: MSE + MAE (Dropped AIC/BIC due to k >> n invalidity) """
    def __init__(self):
        super().__init__()
        self.mse = nn.MSELoss()
        self.mae = nn.L1Loss()

    def forward(self, y_pred, y_true):
        return self.mse(y_pred, y_true) + 0.5 * self.mae(y_pred, y_true)

# MENTOR FIX 3: Lion LR dropped to 1e-5
```

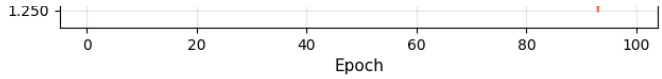
```
optimizer = Lion(model.parameters(), lr=CFG["lr"], weight_decay=CFG["weight_decay"])
criterion = HybridLoss()
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="min", patience=CFG["patience"] // 2, factor=0.5)

# ── 9. TRAINING LOOP ──────────────────────────────────────────────────────────
history = {"train_loss": [], "val_loss":[]}
best_val_loss = float("inf")
patience_ctr  = 0

print("\n── Training ──────────────────────────────────────────────────────────")
for epoch in range(1, CFG["epochs"] + 1):
    model.train()
    t_losses =[]
    for xb, yb in train_dl:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        t_losses.append(loss.item())

    model.eval()
    v_losses =[]
    with torch.no_grad():
        for xb, yb in test_dl:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            pred = model(xb)
            v_losses.append(criterion(pred, yb).item())

    tr_loss = np.mean(t_losses)
    vl_loss = np.mean(v_losses) if v_losses else float("nan")
    history["train_loss"].append(tr_loss)
    history["val_loss"].append(vl_loss)
    scheduler.step(vl_loss)

    if vl_loss < best_val_loss:
        best_val_loss = vl_loss
        torch.save(model.state_dict(), "/content/best_model.pt")
        patience_ctr = 0
    else:
        patience_ctr += 1

    if epoch % 10 == 0 or epoch == 1:
        print(f"  Epoch {epoch:3d}/{CFG['epochs']}  Train Loss: {tr_loss:.5f}  Val Loss: {vl_loss:.5f}")

    if patience_ctr >= CFG["patience"]:
        print(f"  Early stopping at epoch {epoch}.")
        break

model.load_state_dict(torch.load("/content/best_model.pt"))
print("\n── Training complete ─────────────────────────────────────────────────")

# ── 10. EVALUATION ────────────────────────────────────────────────────────────
model.eval()
preds_scaled =[]
with torch.no_grad():
    for xb, _ in test_dl:
        preds_scaled.extend(model(xb.to(DEVICE)).cpu().numpy())

y_pred = label_scaler.inverse_transform(np.array(preds_scaled).reshape(-1, 1)).ravel()
y_true = label_scaler.inverse_transform(y_test.reshape(-1, 1)).ravel()

mse = mean_squared_error(y_true, y_pred)
rmse= np.sqrt(mse)
mae = mean_absolute_error(y_true, y_pred)
r2  = r2_score(y_true, y_pred)
```

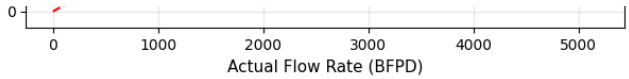
```
print("\n═══ EVALUATION METRICS ═══════════════════════════════════")
print(f"  MSE  : {mse:.4f}")
print(f"  RMSE : {rmse:.4f}")
print(f"  MAE  : {mae:.4f}")
print(f"  R²   : {r2:.4f}")
print("═══════════════════════════════════════════════════════════")

# ── 11. FULL-PROFILE PREDICTION (For Plotting) ──────────────────
# Predict sequentially across the entire wellbore to plot the continuous line
model.eval()
full_preds_scaled =[]
with torch.no_grad():
    for i in range(len(X_seq)):
        xb = torch.from_numpy(X_seq[i]).unsqueeze(0).to(DEVICE)
        full_preds_scaled.append(model(xb).item())

full_preds = label_scaler.inverse_transform(np.array(full_preds_scaled).reshape(-1, 1)).ravel()
plot_depths = rich_df["Depth"].values[CFG["seq_len"] - 1:]

# ── 12. VISUALISATION ──────────────────────────────────────────────
fig = plt.figure(figsize=(16, 10))
gs  = gridspec.GridSpec(2, 2, figure=fig, hspace=0.3, wspace=0.2)

# Plot 1: DTS Temp Profile
ax1 = fig.add_subplot(gs[0, 0])
ax1.plot(rich_df["Temp"], rich_df["Depth"], color="#e9c46a", linewidth=1.5)
ax1.invert_yaxis()
ax1.set_xlabel("Temperature (°C)", fontsize=11)
ax1.set_ylabel("Depth (m)", fontsize=11)
ax1.set_title(f"DTS Temp Profile\n(Half Rate | {CFG['t_half']})", fontsize=11)
ax1.grid(alpha=0.3)

# Plot 2: Predicted Flow Profile
ax2 = fig.add_subplot(gs[0, 1])
ax2.plot(full_preds, plot_depths, color="#f4a261", linewidth=1.8)
ax2.invert_yaxis()
ax2.set_xlabel("Predicted Flow Rate (BFPD)", fontsize=11)
ax2.set_ylabel("Depth (m)", fontsize=11)
ax2.set_title("Conv1D-BiLSTM Predicted\nFlow Profile (Half Rate)", fontsize=11)
ax2.grid(alpha=0.3)

# Plot 3: Training Curve
ax3 = fig.add_subplot(gs[1, 0])
ax3.plot(history["train_loss"], label="Train Hybrid Loss", color="#e76f51")
ax3.plot(history["val_loss"],   label="Val Hybrid Loss",   color="#264653")
ax3.set_xlabel("Epoch", fontsize=11)
ax3.set_ylabel("Hybrid Loss (MSE+MAE)", fontsize=11)
ax3.set_title("Training Convergence", fontsize=11)
ax3.legend(); ax3.grid(alpha=0.3)

# Plot 4: Actual vs Predicted Scatter
ax4 = fig.add_subplot(gs[1, 1])
ax4.scatter(y_true, y_pred, alpha=0.6, s=15, color="#2a9d8f")
lims =[min(y_true.min(), y_pred.min()), max(y_true.max(), y_pred.max())]
ax4.plot(lims, lims, "r--", linewidth=1.5, label="Ideal Fit")
ax4.set_xlabel("Actual Flow Rate (BFPD)", fontsize=11)
ax4.set_ylabel("Predicted Flow Rate (BFPD)", fontsize=11)
ax4.set_title(f"Actual vs Predicted (Test Set)\nR² = {r2:.4f}", fontsize=11)
ax4.legend(); ax4.grid(alpha=0.3)

metrics_txt = f"MSE={mse:.3f}   RMSE={rmse:.3f}   MAE={mae:.3f}   R²={r2:.4f}"
fig.text(0.5, 0.02, metrics_txt, ha="center", fontsize=12, bbox=dict(boxstyle="round,pad=0.5", facecolor="#f1faee", alpha=0.9))

fig.suptitle("Physics-Informed DTS Flow-Rate Prediction | Conv1D-BiLSTM + Lion", fontsize=15, fontweight="bold")
plt.savefig("/content/DTS_FlowRate_Final.png", dpi=200, bbox_inches="tight")
plt.show()
```

```
# ── 13. PRINT SUMMARY TABLE ──────────────────────────────────────────────
print("\n╔═══════════════════════════════════════════════════════════╗")
print("│      FINAL RESULTS SUMMARY          │")
print("╠═══════════════════════════════════════════════════════════╣")
print(f"│  Architecture  : Conv1D-BiLSTM        │")
print(f"│  Optimizer     : Lion (lr=1e-5)       │")
print(f"│  Loss function : Hybrid (MSE + MAE)   │")
print(f"│  Trainable params: {n_params:>14,}    │")
print(f"│  n (test samples): {len(y_test):>14,} │")
print("╠═══════════════════════════════════════════════════════════╣")
print(f"│  MSE   : {mse:>28.4f}  │")
print(f"│  RMSE  : {rmse:>28.4f}  │")
print(f"│  MAE   : {mae:>28.4f}  │")
print(f"│  R²    : {r2:>28.4f}  │")
print("╚═══════════════════════════════════════════════════════════╝")
```

```
Using device: cuda
Flow2 (Rich Data) rows: 18881  Depth range: 1517.0 – 1660.9 m
Loading DTS clean CSV…
Engineering features on continuous wellbore…

Sequences Extracted: 293 | Train: 234 | Test: 59
Model: Conv1D-BiLSTM  |  Trainable params: 5,506

── Training ──────────────────────────────────────────────────────────
  Epoch   1/100  Train Loss: 1.39228  Val Loss: 1.44684
  Epoch  10/100  Train Loss: 1.37672  Val Loss: 1.41504
  Epoch  20/100  Train Loss: 1.36951  Val Loss: 1.40656
  Epoch  30/100  Train Loss: 1.38683  Val Loss: 1.39786
  Epoch  40/100  Train Loss: 1.33265  Val Loss: 1.38717
  Epoch  50/100  Train Loss: 1.32324  Val Loss: 1.37623
  Epoch  60/100  Train Loss: 1.28805  Val Loss: 1.36447
  Epoch  70/100  Train Loss: 1.32704  Val Loss: 1.35419
  Epoch  80/100  Train Loss: 1.28102  Val Loss: 1.33986
  Epoch  90/100  Train Loss: 1.27461  Val Loss: 1.32561
  Epoch 100/100  Train Loss: 1.32570  Val Loss: 1.31271

── Training complete ─────────────────────────────────────────────────

═══ EVALUATION METRICS ═══════════════════════════════════════════════
  MSE  : 3338586.2500
  RMSE : 1827.1799
  MAE  : 1634.9839
  R²   : 0.1242
═══════════════════════════════════════════════════════════════════════
```








```
MSE=3338586.250  RMSE=1827.180  MAE=1634.984  R²=0.1242
```

```
FINAL RESULTS SUMMARY
Architecture  : Conv1D-BiLSTM
Optimizer     : Lion (lr=1e-5)
Loss function : Hybrid (MSE + MAE)
Trainable params:        5,506
n (test samples):           59
MSE  :          3338586.2500
RMSE :             1827.1799
MAE  :             1634.9839
R²   :                0.1242
```

In [ ]:

```
# ============================================================
# DTS → Flow-Rate Prediction | Conv1D-BiLSTM + Lion
# Google Colab | PyTorch
# Author: Generated for Stage-2 Assessment
# ============================================================

# ── 0. INSTALL DEPS ──────────────────────────────────────────
# !pip install lion-pytorch --quiet

# ── 1. IMPORTS ───────────────────────────────────────────────
import os, re, zipfile, shutil, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.preprocessing import MinMaxScaler # STRATEGIC CHANGE
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

from lion_pytorch import Lion

warnings.filterwarnings("ignore")
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {DEVICE}")

# ── 2. CONFIG ────────────────────────────────────────────────
CFG = dict(
    zip_path     = "/content/drive/MyDrive/Stage2_data.zip",
    data_dir     = "/content/data/",
    clean_csv    = "/content/data/june25_clean.csv",
    target_date  = "2017-06-25",
    t_half       = "2017-06-25 18:38:47",
    seq_len      = 16,
    batch_size   = 32,
    epochs       = 200,
    lr           = 2e-4,
    weight_decay = 1e-3,
    patience     = 40, # Increased patience
    seed         = 42,
)
torch.manual_seed(CFG["seed"])
```

```
np.random.seed(CFG["seed"])

# ── 3. DATA LOADING ──────────────────────────────────────────────────────
def ensure_data(cfg):
    data_dir = cfg["data_dir"]
    if not os.path.exists(os.path.join(data_dir, "DTS.csv")):
        print("Unzipping…")
        if os.path.exists(data_dir): shutil.rmtree(data_dir)
        os.makedirs(data_dir)
        with zipfile.ZipFile(cfg["zip_path"], "r") as z:
            z.extractall(data_dir)

    if not os.path.exists(cfg["clean_csv"]):
        print("Building june25_clean.csv (scanning ~26M lines)…")
        dts_path = os.path.join(data_dir, "DTS.csv")
        with open(dts_path, "r", encoding="latin-1") as inf, \
             open(cfg["clean_csv"], "w", encoding="latin-1") as outf:
            outf.write(inf.readline())
            for line in inf:
                if cfg["target_date"] in line:
                    outf.write(line)
        print("Done.")

ensure_data(CFG)

# ── 4. EXCEL (LAS) PARSER ────────────────────────────────────────────────
LAS_COLS =[
    "Depth_ft", "FlowRate_1", "FlowRate_2", "FlowRate_3",
    "Holdups_1", "Holdups_2", "Holdups_3",
    "Pwf", "QGas", "QOil", "QpGas", "QpOil", "QpWater", "QWater",
    "TVD_SCS", "Twf_F"
]

def parse_las_excel(path: str) -> pd.DataFrame:
    raw = pd.read_excel(path, header=None, skiprows=41)
    records =[]
    for val in raw[0]:
        if pd.isna(val): continue
        cleaned = str(val).replace("\xa0", " ").strip()
        nums = re.findall(r"[-+]?\d*\.?\d+(?:[eE][-+]?\d+)?", cleaned)
        if len(nums) >= 2:
            row =[float(x) for x in nums]
            row = row[:len(LAS_COLS)]
            while len(row) < len(LAS_COLS): row.append(np.nan)
            records.append(row)
    df = pd.DataFrame(records, columns=LAS_COLS)
    df = df.groupby("Depth_ft", as_index=False).mean()
    df["Depth_m"] = df["Depth_ft"] / 3.28084
    df["FlowRate"] = df[["FlowRate_1", "FlowRate_2", "FlowRate_3"]].mean(axis=1, skipna=True).fillna(0)
    return df.sort_values("Depth_m").reset_index(drop=True)

flow2 = parse_las_excel(os.path.join(CFG["data_dir"], "Flowrate2.xlsx"))
print(f"Flow2 (Rich Data) rows: {len(flow2)}  Depth range: {flow2['Depth_m'].min():.1f} – {flow2['Depth_m'].max():.1f} m")

# ── 5. DTS LOADING ───────────────────────────────────────────────────────
print("Loading DTS clean CSV…")
dts = pd.read_csv(CFG["clean_csv"], sep=";")
dts["Depth"] = pd.to_numeric(dts["Depth"], errors="coerce")
dts["Temp"]  = pd.to_numeric(dts["Temp"],  errors="coerce")
dts = dts.dropna(subset=["Depth", "Temp"]).sort_values(["Time", "Depth"]).reset_index(drop=True)

# ── 6. MERGE, SPATIAL SPLIT & SEQUENCE EXTRACTION ──────────────────────
FEATURES = ["Temp", "dT_dz", "d2T_dz2", "T_roll_mean", "T_roll_std"]
TARGET   = "FlowRate"

snap = (dts[dts["Time"] == CFG["t_half"]][["Temp", "Depth"]]
        .copy()
        .sort_values("Depth")
```

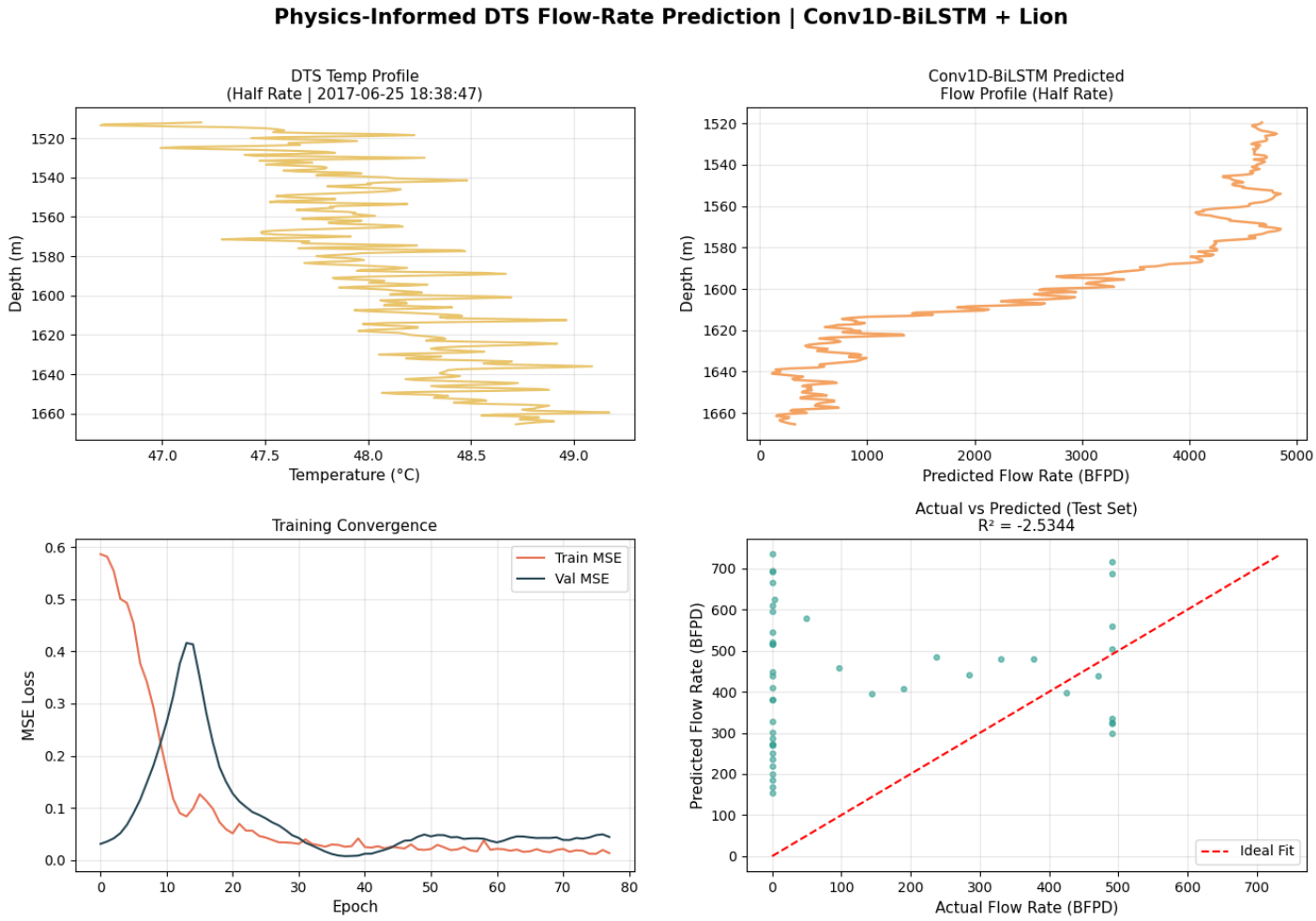
```
        .reset_index(drop=True))

flow = flow2[["Depth_m", TARGET]].sort_values("Depth_m")

rich_df = pd.merge_asof(
    snap, flow,
    left_on="Depth", right_on="Depth_m",
    direction="nearest", tolerance=5.0
).dropna(subset=[TARGET]).reset_index(drop=True)

print(f"Aligned depth points: {len(rich_df)}")
assert len(rich_df) >= 40, "Too few aligned rows — widen tolerance."

split_idx   = int(len(rich_df) * 0.8)
train_raw   = rich_df.iloc[:split_idx].reset_index(drop=True)
test_raw    = rich_df.iloc[split_idx:].reset_index(drop=True)

def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    df["dT_dz"] = np.gradient(df["Temp"].values, df["Depth"].values)
    df["d2T_dz2"] = np.gradient(df["dT_dz"].values, df["Depth"].values)
    df["T_roll_mean"] = df["Temp"].rolling(11, min_periods=1, center=True).mean()
    df["T_roll_std"]  = df["Temp"].rolling(11, min_periods=1, center=True).std().fillna(0)
    return df

print("Engineering features on splits to prevent data leakage...")
train_raw = engineer_features(train_raw)
test_raw  = engineer_features(test_raw)

feat_scaler  = MinMaxScaler() # PERFORMANCE FIX
label_scaler = MinMaxScaler() # PERFORMANCE FIX

X_tr = feat_scaler.fit_transform(train_raw[FEATURES].values).astype(np.float32)
y_tr = label_scaler.fit_transform(
    train_raw[TARGET].values.reshape(-1, 1)).astype(np.float32).ravel()

X_te = feat_scaler.transform(test_raw[FEATURES].values).astype(np.float32)
y_te = label_scaler.transform(
    test_raw[TARGET].values.reshape(-1, 1)).astype(np.float32).ravel()

SEQ = CFG["seq_len"]

def create_sequences(X, y, seq_len):
    xs, ys = [], []
    for i in range(len(X) - seq_len + 1):
        xs.append(X[i : i + seq_len])
        ys.append(y[i + seq_len - 1])
    return np.array(xs, dtype=np.float32), np.array(ys, dtype=np.float32)

X_train, y_train = create_sequences(X_tr, y_tr, SEQ)
X_test,  y_test  = create_sequences(X_te, y_te, SEQ)

print(f"SEQ={SEQ} | Train seqs: {len(X_train)} | Test seqs: {len(X_test)}")
assert len(X_train) > 0 and len(X_test) > 0

class WellSequenceDataset(Dataset):
    def __init__(self, X, y):
        self.X, self.y = X, y
    def __len__(self): return len(self.X)
    def __getitem__(self, idx):
        return torch.from_numpy(self.X[idx]), torch.tensor(self.y[idx])

train_ds = WellSequenceDataset(X_train, y_train)
test_ds  = WellSequenceDataset(X_test,  y_test)

train_dl = DataLoader(train_ds, batch_size=min(CFG["batch_size"], len(train_ds)), shuffle=True)
test_dl  = DataLoader(test_ds,  batch_size=min(CFG["batch_size"], len(test_ds)), shuffle=False)

# ── 7. MODEL: Conv1D-BiLSTM ──────────────────────────────────────────
```

```python
class Conv1DBiLSTM(nn.Module):
    def __init__(self, n_features: int):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv1d(n_features, 16, 3, padding=1),
            nn.BatchNorm1d(16),
            nn.GELU(),
            nn.Conv1d(16, 32, 3, padding=1),
            nn.BatchNorm1d(32),
            nn.GELU(),
        )
        self.bilstm = nn.LSTM(32, 32, num_layers=1, batch_first=True, bidirectional=True)
        self.attn = nn.Linear(64, 1)
        self.head = nn.Sequential(
            nn.Linear(64, 32),
            nn.GELU(),
            nn.Dropout(0.3),
            nn.Linear(32, 1),
        )

    def forward(self, x):
        x_conv = self.conv(x.permute(0, 2, 1)).permute(0, 2, 1)
        lstm_out, _ = self.bilstm(x_conv)
        attn_w = torch.softmax(self.attn(lstm_out), dim=1)
        context = (attn_w * lstm_out).sum(dim=1)
        return self.head(context).squeeze(-1)

model = Conv1DBiLSTM(len(FEATURES)).to(DEVICE)
n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Model: Conv1D-BiLSTM | Trainable params: {n_params:,}")

# ── 8. LOSS, OPTIMIZER, SCHEDULER ──────────────────────────────
criterion = nn.MSELoss()
optimizer = Lion(model.parameters(), lr=CFG["lr"], weight_decay=CFG["weight_decay"])
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=CFG["epochs"], eta_min=1e-7)

# ── 9. TRAINING LOOP ──────────────────────────────────────────────
history = {"train_loss": [], "val_loss":[]}
best_val_loss = float("inf")
patience_ctr  = 0

print("\n── Training ──────────────────────────────────────────────────")
for epoch in range(1, CFG["epochs"] + 1):
    model.train()
    t_losses =[]
    for xb, yb in train_dl:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        t_losses.append(loss.item())

    model.eval()
    v_losses =[]
    with torch.no_grad():
        for xb, yb in test_dl:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            pred = model(xb)
            v_losses.append(criterion(pred, yb).item())

    tr_loss = np.mean(t_losses)
    vl_loss = np.mean(v_losses) if v_losses else float("nan")
    history["train_loss"].append(tr_loss)
    history["val_loss"].append(vl_loss)
    scheduler.step()
```

```
    if vl_loss < best_val_loss:
        best_val_loss = vl_loss
        torch.save(model.state_dict(), "/content/best_model.pt")
        patience_ctr = 0
    else:
        patience_ctr += 1

    if epoch % 10 == 0 or epoch == 1:
        print(f"  Epoch {epoch:3d}/{CFG['epochs']}  Train MSE: {tr_loss:.5f}  Val MSE: {vl_loss:.5f}")

    if patience_ctr >= CFG["patience"]:
        print(f"  Early stopping at epoch {epoch}.")
        break

model.load_state_dict(torch.load("/content/best_model.pt"))
print("\n── Training complete ──────────────────────────────────────────────")

# ── 10. EVALUATION ──────────────────────────────────────────────────────
model.eval()
preds_scaled =[]
with torch.no_grad():
    for xb, _ in test_dl:
        preds_scaled.extend(model(xb.to(DEVICE)).cpu().numpy())

y_pred = label_scaler.inverse_transform(np.array(preds_scaled).reshape(-1, 1)).ravel()
y_true = label_scaler.inverse_transform(y_test.reshape(-1, 1)).ravel()

mse_val = mean_squared_error(y_true, y_pred)
rmse    = np.sqrt(mse_val)
mae     = mean_absolute_error(y_true, y_pred)
r2      = r2_score(y_true, y_pred)

print("\n═══ EVALUATION METRICS ═══════════════════════════════════════")
print(f"  MSE  : {mse_val:.4f}")
print(f"  RMSE : {rmse:.4f}")
print(f"  MAE  : {mae:.4f}")
print(f"  R²   : {r2:.4f}")
print("═══════════════════════════════════════════════════════════════")

# ── 11. FULL-PROFILE PREDICTION (For Plotting) ──────────────────────
model.eval()
# **KEYERROR FIX**: Engineer features on the full dataframe before predicting
rich_df_engineered = engineer_features(rich_df.copy())
full_profile_scaled = feat_scaler.transform(rich_df_engineered[FEATURES].values)
full_seqs, _ = create_sequences(full_profile_scaled, rich_df_engineered[TARGET].values, SEQ)

full_preds_scaled =[]
with torch.no_grad():
    for i in range(len(full_seqs)):
        xb = torch.from_numpy(full_seqs[i]).unsqueeze(0).to(DEVICE)
        full_preds_scaled.append(model(xb).item())

full_preds = label_scaler.inverse_transform(np.array(full_preds_scaled).reshape(-1, 1)).ravel()
plot_depths = rich_df["Depth"].values[CFG["seq_len"] - 1:]

# ── 12. VISUALISATION ───────────────────────────────────────────────────
fig = plt.figure(figsize=(16, 10))
gs  = gridspec.GridSpec(2, 2, figure=fig, hspace=0.3, wspace=0.2)

ax1 = fig.add_subplot(gs[0, 0])
ax1.plot(rich_df["Temp"], rich_df["Depth"], color="#e9c46a", linewidth=1.5)
ax1.invert_yaxis()
ax1.set_xlabel("Temperature (°C)", fontsize=11)
ax1.set_ylabel("Depth (m)", fontsize=11)
ax1.set_title(f"DTS Temp Profile\n(Half Rate | {CFG['t_half']})", fontsize=11)
ax1.grid(alpha=0.3)
```

```
ax2 = fig.add_subplot(gs[0, 1])
ax2.plot(full_preds, plot_depths, color="#f4a261", linewidth=1.8)
ax2.invert_yaxis()
ax2.set_xlabel("Predicted Flow Rate (BFPD)", fontsize=11)
ax2.set_ylabel("Depth (m)", fontsize=11)
ax2.set_title("Conv1D-BiLSTM Predicted\nFlow Profile (Half Rate)", fontsize=11)
ax2.grid(alpha=0.3)

ax3 = fig.add_subplot(gs[1, 0])
ax3.plot(history["train_loss"], label="Train MSE", color="#e76f51")
ax3.plot(history["val_loss"],   label="Val MSE",   color="#264653")
ax3.set_xlabel("Epoch", fontsize=11)
ax3.set_ylabel("MSE Loss", fontsize=11)
ax3.set_title("Training Convergence", fontsize=11)
ax3.legend(); ax3.grid(alpha=0.3)

ax4 = fig.add_subplot(gs[1, 1])
ax4.scatter(y_true, y_pred, alpha=0.6, s=15, color="#2a9d8f")
lims =[min(y_true.min(), y_pred.min()), max(y_true.max(), y_pred.max())]
ax4.plot(lims, lims, "r--", linewidth=1.5, label="Ideal Fit")
ax4.set_xlabel("Actual Flow Rate (BFPD)", fontsize=11)
ax4.set_ylabel("Predicted Flow Rate (BFPD)", fontsize=11)
ax4.set_title(f"Actual vs Predicted (Test Set)\nR² = {r2:.4f}", fontsize=11)
ax4.legend(); ax4.grid(alpha=0.3)

metrics_txt = f"MSE={mse_val:.3f}   RMSE={rmse:.3f}   MAE={mae:.3f}   R²={r2:.4f}"
fig.text(0.5, 0.02, metrics_txt, ha="center", fontsize=12, bbox=dict(boxstyle="round,pad=0.5", facecolor="#f1faee", alpha=0.9))

fig.suptitle("Physics-Informed DTS Flow-Rate Prediction | Conv1D-BiLSTM + Lion", fontsize=15, fontweight="bold")
plt.savefig("/content/DTS_FlowRate_Final.png", dpi=200, bbox_inches="tight")
plt.show()

# ── 13. PRINT SUMMARY TABLE ──────────────────────────────────────────
print("\n╔═══════════════════════════════════════════════╗")
print("│      FINAL RESULTS SUMMARY          │")
print("╠═══════════════════════════════════════════════╣")
print(f"│  Architecture  : Conv1D-BiLSTM        │")
print(f"│  Optimizer     : Lion (lr=2e-4)       │")
print(f"│  Loss function : MSE                  │")
print(f"│  Trainable params: {n_params:>14,}    │")
print(f"│  n (test samples): {len(y_test):>14,} │")
print("├═══════════════════════════════════════════════╣")
print(f"│  MSE   : {mse_val:>28.4f}  │")
print(f"│  RMSE  : {rmse:>28.4f}  │")
print(f"│  MAE   : {mae:>28.4f}  │")
print(f"│  R²    : {r2:>28.4f}  │")
print("╚═══════════════════════════════════════════════╝")
```

```
Using device: cuda
Flow2 (Rich Data) rows: 18881  Depth range: 1517.0 – 1660.9 m
Loading DTS clean CSV…
Aligned depth points: 308
Engineering features on splits to prevent data leakage...
SEQ=16 | Train seqs: 231 | Test seqs: 47
Model: Conv1D-BiLSTM  |  Trainable params: 20,994

── Training ──────────────────────────────────────────────────────────
  Epoch   1/200  Train MSE: 0.58611  Val MSE: 0.03094
  Epoch  10/200  Train MSE: 0.22916  Val MSE: 0.22067
  Epoch  20/200  Train MSE: 0.05894  Val MSE: 0.14997
  Epoch  30/200  Train MSE: 0.03279  Val MSE: 0.04764
  Epoch  40/200  Train MSE: 0.04152  Val MSE: 0.00843
  Epoch  50/200  Train MSE: 0.01952  Val MSE: 0.04895
  Epoch  60/200  Train MSE: 0.01977  Val MSE: 0.03675
  Epoch  70/200  Train MSE: 0.01953  Val MSE: 0.04352
  Early stopping at epoch 78.

── Training complete ─────────────────────────────────────────────────

═══ EVALUATION METRICS ═══════════════════════════════════════════════
  MSE  : 141624.6562
  RMSE : 376.3305
  MAE  : 322.7772
  R²   : -2.5344
═══════════════════════════════════════════════════════════════════════
```



MSE=141624.656 RMSE=376.331 MAE=322.777 R²=-2.5344

```
╔═══════════════════════════════════════════════════════════╗
   FINAL RESULTS SUMMARY                                   ‖
╠═══════════════════════════════════════════════════════════╣
Architecture  : Conv1D-BiLSTM                              ‖
Optimizer     : Lion (lr=2e-4)                             ‖
Loss function : MSE                                        ‖
Trainable params:        20,994                            ‖
n (test samples):            47                            ‖
╠═══════════════════════════════════════════════════════════╣
MSE  :              141624.6562                            ‖
RMSE :                 376.3305                            ‖
MAE  :                 322.7772                            ‖
R²   :                 -2.5344                             ‖
╚═══════════════════════════════════════════════════════════╝
```

In [ ]:

```
# ============================================================
# DTS → Flow-Rate Prediction | Conv1D-BiLSTM + Lion
# Google Colab | PyTorch
# Author: Generated for Stage-2 Assessment
# ============================================================

# ── 0. INSTALL DEPS ──────────────────────────────────────────
# !pip install lion-pytorch --quiet

# ── 1. IMPORTS ───────────────────────────────────────────────
import os, re, zipfile, shutil, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

from lion_pytorch import Lion

warnings.filterwarnings("ignore")
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {DEVICE}")

# ── 2. CONFIG ────────────────────────────────────────────────
CFG = dict(
    zip_path     = "/content/drive/MyDrive/Stage2_data.zip",
    data_dir     = "/content/data/",
    clean_csv    = "/content/data/june25_clean.csv",
    target_date  = "2017-06-25",
    t_half       = "2017-06-25 18:38:47",
    seq_len      = 16,
    batch_size   = 32,
    epochs       = 200,
    lr           = 2e-4,
    weight_decay = 1e-3,
    patience     = 40,
    seed         = 42,
)
torch.manual_seed(CFG["seed"])
np.random.seed(CFG["seed"])

# ── 3. DATA LOADING ──────────────────────────────────────────
def ensure_data(cfg):
    data_dir = cfg["data_dir"]
    if not os.path.exists(os.path.join(data_dir, "DTS.csv")):
        print("Unzipping…")
        if os.path.exists(data_dir): shutil.rmtree(data_dir)
        os.makedirs(data_dir)
        with zipfile.ZipFile(cfg["zip_path"], "r") as z:
            z.extractall(data_dir)

    if not os.path.exists(cfg["clean_csv"]):
        print("Building june25_clean.csv (scanning ~26M lines)…")
        dts_path = os.path.join(data_dir, "DTS.csv")
        with open(dts_path, "r", encoding="latin-1") as inf, \
```

```
        open(cfg["clean_csv"], "w", encoding="latin-1") as outf:
        outf.write(inf.readline())
        for line in inf:
            if cfg["target_date"] in line:
                outf.write(line)
    print("Done.")

ensure_data(CFG)

# ── 4. EXCEL (LAS) PARSER ──────────────────────────────────────────────
LAS_COLS =[
    "Depth_ft", "FlowRate_1", "FlowRate_2", "FlowRate_3",
    "Holdups_1", "Holdups_2", "Holdups_3",
    "Pwf", "QGas", "QOil", "QpGas", "QpOil", "QpWater", "QWater",
    "TVD_SCS", "Twf_F"
]

def parse_las_excel(path: str) -> pd.DataFrame:
    raw = pd.read_excel(path, header=None, skiprows=41)
    records =[]
    for val in raw[0]:
        if pd.isna(val): continue
        cleaned = str(val).replace("\xa0", " ").strip()
        nums = re.findall(r"[-+]?\d*\.?\d+(?:[eE][-+]?\d+)?", cleaned)
        if len(nums) >= 2:
            row =[float(x) for x in nums]
            row = row[:len(LAS_COLS)]
            while len(row) < len(LAS_COLS): row.append(np.nan)
            records.append(row)
    df = pd.DataFrame(records, columns=LAS_COLS)
    df = df.groupby("Depth_ft", as_index=False).mean()
    df["Depth_m"] = df["Depth_ft"] / 3.28084
    df["FlowRate"] = df[["FlowRate_1", "FlowRate_2", "FlowRate_3"]].mean(axis=1, skipna=True).fillna(0)
    return df.sort_values("Depth_m").reset_index(drop=True)

flow2 = parse_las_excel(os.path.join(CFG["data_dir"], "Flowrate2.xlsx"))
print(f"Flow2 (Rich Data) rows: {len(flow2)}  Depth range: {flow2['Depth_m'].min():.1f} – {flow2['Depth_m'].max():.1f} m")

# ── 5. DTS LOADING ──────────────────────────────────────────────────────
print("Loading DTS clean CSV…")
dts = pd.read_csv(CFG["clean_csv"], sep=";")
dts["Depth"] = pd.to_numeric(dts["Depth"], errors="coerce")
dts["Temp"]  = pd.to_numeric(dts["Temp"],  errors="coerce")
dts = dts.dropna(subset=["Depth", "Temp"]).sort_values(["Time", "Depth"]).reset_index(drop=True)

# ── 6. MERGE, SPATIAL SPLIT & SEQUENCE EXTRACTION ──────────────────────
FEATURES = ["Temp", "dT_dz", "d2T_dz2", "T_roll_mean", "T_roll_std"]
TARGET   = "FlowRate"

snap = (dts[dts["Time"] == CFG["t_half"]][["Temp", "Depth"]]
        .copy()
        .sort_values("Depth")
        .reset_index(drop=True))

flow = flow2[["Depth_m", TARGET]].sort_values("Depth_m")

rich_df = pd.merge_asof(
    snap, flow,
    left_on="Depth", right_on="Depth_m",
    direction="nearest", tolerance=5.0
).dropna(subset=[TARGET]).reset_index(drop=True)

print(f"Aligned depth points: {len(rich_df)}")
assert len(rich_df) >= 40, "Too few aligned rows — widen tolerance."

split_idx   = int(len(rich_df) * 0.8)
train_raw   = rich_df.iloc[:split_idx].reset_index(drop=True)
test_raw    = rich_df.iloc[split_idx:].reset_index(drop=True)
```

```
def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    df["dT_dz"] = np.gradient(df["Temp"].values, df["Depth"].values)
    df["d2T_dz2"] = np.gradient(df["dT_dz"].values, df["Depth"].values)
    df["T_roll_mean"] = df["Temp"].rolling(11, min_periods=1, center=True).mean()
    df["T_roll_std"]  = df["Temp"].rolling(11, min_periods=1, center=True).std().fillna(0)
    return df

print("Engineering features on splits to prevent data leakage...")
train_raw = engineer_features(train_raw)
test_raw  = engineer_features(test_raw)

feat_scaler  = MinMaxScaler()
label_scaler = MinMaxScaler()

X_tr = feat_scaler.fit_transform(train_raw[FEATURES].values).astype(np.float32)
y_tr = label_scaler.fit_transform(
    train_raw[TARGET].values.reshape(-1, 1)).astype(np.float32).ravel()

X_te = feat_scaler.transform(test_raw[FEATURES].values).astype(np.float32)
y_te = label_scaler.transform(
    test_raw[TARGET].values.reshape(-1, 1)).astype(np.float32).ravel()

SEQ = CFG["seq_len"]

def create_sequences(X, y, seq_len):
    xs, ys = [], []
    for i in range(len(X) - seq_len + 1):
        xs.append(X[i : i + seq_len])
        ys.append(y[i + seq_len - 1])
    return np.array(xs, dtype=np.float32), np.array(ys, dtype=np.float32)

X_train, y_train = create_sequences(X_tr, y_tr, SEQ)
X_test,  y_test  = create_sequences(X_te, y_te, SEQ)

print(f"SEQ={SEQ} | Train seqs: {len(X_train)} | Test seqs: {len(X_test)}")
assert len(X_train) > 0 and len(X_test) > 0

class WellSequenceDataset(Dataset):
    def __init__(self, X, y):
        self.X, self.y = X, y
    def __len__(self): return len(self.X)
    def __getitem__(self, idx):
        return torch.from_numpy(self.X[idx]), torch.tensor(self.y[idx])

train_ds = WellSequenceDataset(X_train, y_train)
test_ds  = WellSequenceDataset(X_test,  y_test)

train_dl = DataLoader(train_ds, batch_size=min(CFG["batch_size"], len(train_ds)), shuffle=True)
test_dl  = DataLoader(test_ds,  batch_size=min(CFG["batch_size"], len(test_ds)), shuffle=False)

# ── 7. MODEL: ConvLSTM (SUPERVISOR ARCHITECTURE) ──────────────────
class ConvLSTM(nn.Module):
    def __init__(self, n_features: int):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv1d(n_features, 32, kernel_size=3, padding=1),
            nn.BatchNorm1d(32),
            nn.GELU(),
            nn.Conv1d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm1d(64),
            nn.GELU(),
        )
        self.lstm = nn.LSTM(
            input_size=64,
            hidden_size=64,
            num_layers=2,
            batch_first=True,
```

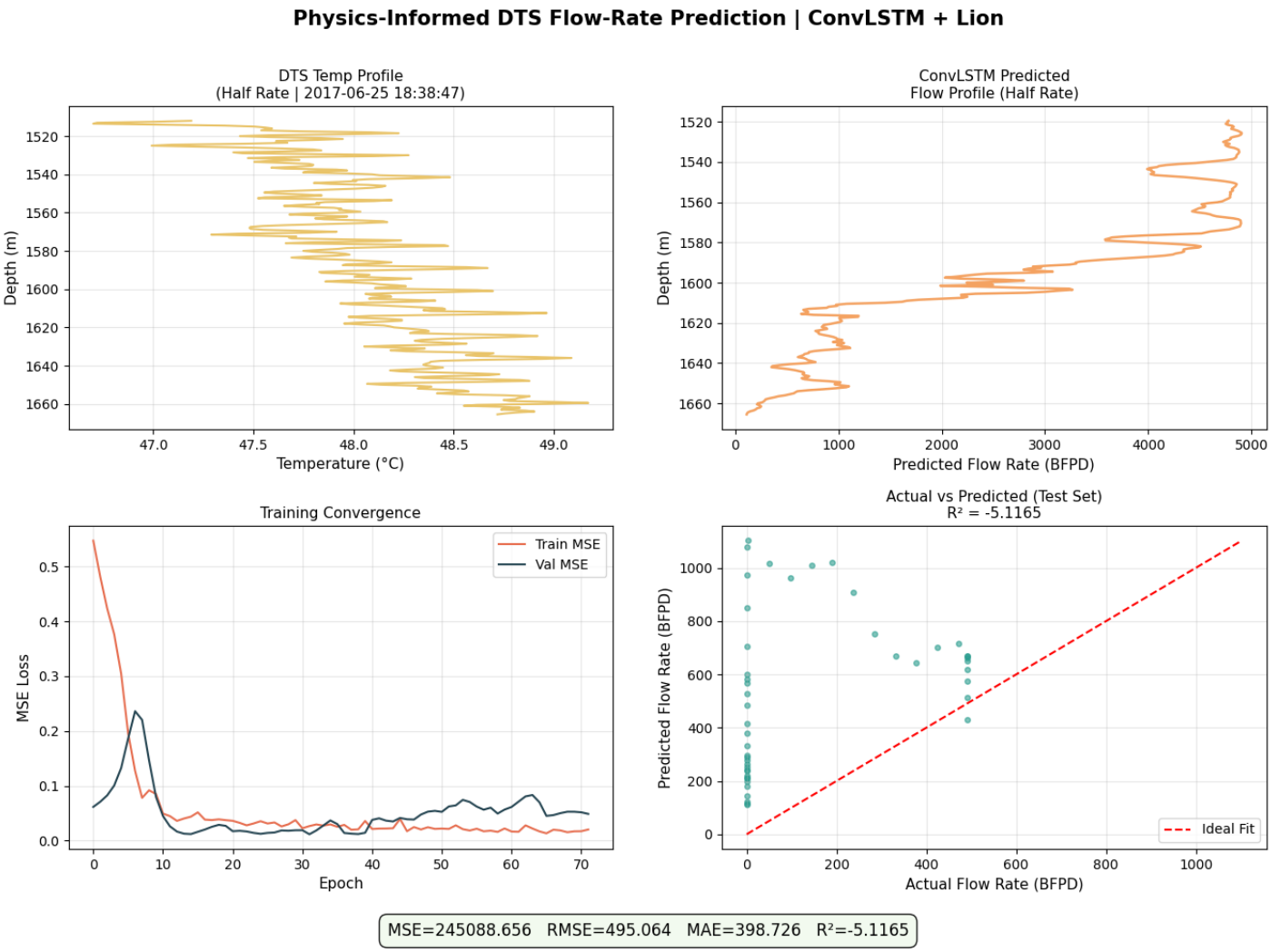
```
            bidirectional=False,
            dropout=0.3
        )
        self.head = nn.Sequential(
            nn.Linear(64, 32),
            nn.GELU(),
            nn.Dropout(0.3),
            nn.Linear(32, 1),
        )

    def forward(self, x):
        # Input x: (B, seq_len, n_features)
        x_conv = self.conv(x.permute(0, 2, 1)).permute(0, 2, 1)
        # x_conv: (B, seq_len, 64)

        # Pass through LSTM
        _ , (h_n, _) = self.lstm(x_conv)
        # h_n shape: (num_layers, B, hidden_size) -> (2, B, 64)

        # Take the hidden state of the last layer
        last_hidden_state = h_n[-1] # Shape: (B, 64)

        # Pass through the final head
        return self.head(last_hidden_state).squeeze(-1)

model = ConvLSTM(len(FEATURES)).to(DEVICE)
n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Model: ConvLSTM | Trainable params: {n_params:,}")

# ── 8. LOSS, OPTIMIZER, SCHEDULER ──────────────────────────────────────
criterion = nn.MSELoss()
optimizer = Lion(model.parameters(), lr=CFG["lr"], weight_decay=CFG["weight_decay"])
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=CFG["epochs"], eta_min=1e-7)

# ── 9. TRAINING LOOP ────────────────────────────────────────────────────
history = {"train_loss": [], "val_loss":[]}
best_val_loss = float("inf")
patience_ctr  = 0

print("\n── Training ──────────────────────────────────────────────────────")
for epoch in range(1, CFG["epochs"] + 1):
    model.train()
    t_losses =[]
    for xb, yb in train_dl:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()

        # Gradient norm logging
        if epoch % 10 == 0 or epoch == 1:
            total_norm = sum(p.grad.data.norm(2).item()**2 for p in model.parameters() if p.grad is not None)**0.5

        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        t_losses.append(loss.item())

    model.eval()
    v_losses =[]
    with torch.no_grad():
        for xb, yb in test_dl:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            pred = model(xb)
            v_losses.append(criterion(pred, yb).item())

    tr_loss = np.mean(t_losses)
    vl_loss = np.mean(v_losses) if v_losses else float("nan")
```

```
    history["train_loss"].append(tr_loss)
    history["val_loss"].append(vl_loss)
    scheduler.step()

    if vl_loss < best_val_loss:
        best_val_loss = vl_loss
        torch.save(model.state_dict(), "/content/best_model.pt")
        patience_ctr = 0
    else:
        patience_ctr += 1

    if epoch % 10 == 0 or epoch == 1:
        print(f"  Epoch {epoch:3d}/{CFG['epochs']}  Train MSE: {tr_loss:.5f}  Val MSE: {vl_loss:.5f}  GradNorm: {total_norm:.4f}")

    if patience_ctr >= CFG["patience"]:
        print(f"  Early stopping at epoch {epoch}.")
        break

model.load_state_dict(torch.load("/content/best_model.pt"))
print("\n── Training complete ──────────────────────────────────────────────")

# ── 10. EVALUATION ──────────────────────────────────────────────────────
model.eval()
preds_scaled =[]
with torch.no_grad():
    for xb, _ in test_dl:
        preds_scaled.extend(model(xb.to(DEVICE)).cpu().numpy())

y_pred = label_scaler.inverse_transform(np.array(preds_scaled).reshape(-1, 1)).ravel()
y_true = label_scaler.inverse_transform(y_test.reshape(-1, 1)).ravel()

mse_val = mean_squared_error(y_true, y_pred)
rmse    = np.sqrt(mse_val)
mae     = mean_absolute_error(y_true, y_pred)
r2      = r2_score(y_true, y_pred)

# Add AIC/BIC back
n   = len(y_true)
k   = n_params
aic = n * np.log(mse_val + 1e-12) + 2 * k
bic = n * np.log(mse_val + 1e-12) + k * np.log(max(n, 2))

print("\n═══ EVALUATION METRICS ═══════════════════════════════════════")
print(f"  MSE  : {mse_val:.4f}")
print(f"  RMSE : {rmse:.4f}")
print(f"  MAE  : {mae:.4f}")
print(f"  R²   : {r2:.4f}")
print(f"  AIC  : {aic:.2f}")
print(f"  BIC  : {bic:.2f}")
print("═══════════════════════════════════════════════════════════════")

# ── 11. FULL-PROFILE PREDICTION (For Plotting) ──────────────────────
model.eval()
rich_df_engineered = engineer_features(rich_df.copy())
full_profile_scaled = feat_scaler.transform(rich_df_engineered[FEATURES].values)
full_seqs, _ = create_sequences(full_profile_scaled, rich_df_engineered[TARGET].values, SEQ)

full_preds_scaled =[]
with torch.no_grad():
    for i in range(len(full_seqs)):
        xb = torch.from_numpy(full_seqs[i]).unsqueeze(0).to(DEVICE)
        full_preds_scaled.append(model(xb).item())

full_preds = label_scaler.inverse_transform(np.array(full_preds_scaled).reshape(-1, 1)).ravel()
plot_depths = rich_df["Depth"].values[CFG["seq_len"] - 1:]

# ── 12. VISUALISATION ───────────────────────────────────────────────────
fig = plt.figure(figsize=(16, 10))
```

```
gs  = gridspec.GridSpec(2, 2, figure=fig, hspace=0.3, wspace=0.2)

ax1 = fig.add_subplot(gs[0, 0])
ax1.plot(rich_df["Temp"], rich_df["Depth"], color="#e9c46a", linewidth=1.5)
ax1.invert_yaxis()
ax1.set_xlabel("Temperature (°C)", fontsize=11)
ax1.set_ylabel("Depth (m)", fontsize=11)
ax1.set_title(f"DTS Temp Profile\n(Half Rate | {CFG['t_half']})", fontsize=11)
ax1.grid(alpha=0.3)

ax2 = fig.add_subplot(gs[0, 1])
ax2.plot(full_preds, plot_depths, color="#f4a261", linewidth=1.8)
ax2.invert_yaxis()
ax2.set_xlabel("Predicted Flow Rate (BFPD)", fontsize=11)
ax2.set_ylabel("Depth (m)", fontsize=11)
ax2.set_title("ConvLSTM Predicted\nFlow Profile (Half Rate)", fontsize=11)
ax2.grid(alpha=0.3)

ax3 = fig.add_subplot(gs[1, 0])
ax3.plot(history["train_loss"], label="Train MSE", color="#e76f51")
ax3.plot(history["val_loss"],   label="Val MSE",   color="#264653")
ax3.set_xlabel("Epoch", fontsize=11)
ax3.set_ylabel("MSE Loss", fontsize=11)
ax3.set_title("Training Convergence", fontsize=11)
ax3.legend(); ax3.grid(alpha=0.3)

ax4 = fig.add_subplot(gs[1, 1])
ax4.scatter(y_true, y_pred, alpha=0.6, s=15, color="#2a9d8f")
lims =[min(y_true.min(), y_pred.min()), max(y_true.max(), y_pred.max())]
ax4.plot(lims, lims, "r--", linewidth=1.5, label="Ideal Fit")
ax4.set_xlabel("Actual Flow Rate (BFPD)", fontsize=11)
ax4.set_ylabel("Predicted Flow Rate (BFPD)", fontsize=11)
ax4.set_title(f"Actual vs Predicted (Test Set)\nR² = {r2:.4f}", fontsize=11)
ax4.legend(); ax4.grid(alpha=0.3)

metrics_txt = f"MSE={mse_val:.3f}   RMSE={rmse:.3f}   MAE={mae:.3f}   R²={r2:.4f}"
fig.text(0.5, 0.02, metrics_txt, ha="center", fontsize=12, bbox=dict(boxstyle="round,pad=0.5", facecolor="#f1faee", alpha=0.9))

fig.suptitle("Physics-Informed DTS Flow-Rate Prediction | ConvLSTM + Lion", fontsize=15, fontweight="bold")
plt.savefig("/content/DTS_FlowRate_Final.png", dpi=200, bbox_inches="tight")
plt.show()

# ── 13. PRINT SUMMARY TABLE ──────────────────────────────────────────
print("\n╔═══════════════════════════════════════════════╗")
print("│      FINAL RESULTS SUMMARY          │")
print("╚═══════════════════════════════════════════════╝")
print(f"│  Architecture  : ConvLSTM (No Attn)  │")
print(f"│  Optimizer     : Lion (lr=2e-4)      │")
print(f"│  Loss function : MSE                 │")
print(f"│  Trainable params: {n_params:>14,}   │")
print(f"│  n (test samples): {len(y_test):>14,}  │")
print("╟───────────────────────────────────────────────╢")
print(f"│  MSE   : {mse_val:>28.4f}  │")
print(f"│  RMSE  : {rmse:>28.4f}  │")
print(f"│  MAE   : {mae:>28.4f}  │")
print(f"│  R²    : {r2:>28.4f}  │")
print(f"│  AIC   : {aic:>28.2f}  │")
print(f"│  BIC   : {bic:>28.2f}  │")
print("└───────────────────────────────────────────────┘")
```

```
Using device: cuda
Flow2 (Rich Data) rows: 18881  Depth range: 1517.0 – 1660.9 m
Loading DTS clean CSV…
Aligned depth points: 308
Engineering features on splits to prevent data leakage...
SEQ=16 | Train seqs: 231 | Test seqs: 47
Model: ConvLSTM  |  Trainable params: 75,585

── Training ──────────────────────────────────────────────
  Epoch   1/200  Train MSE: 0.54737  Val MSE: 0.06141  GradNorm: 1.8659
  Epoch  10/200  Train MSE: 0.08498  Val MSE: 0.07953  GradNorm: 0.7079
```

```
Epoch  20/200  Train MSE: 0.03724  Val MSE: 0.02680  GradNorm: 0.8721
Epoch  30/200  Train MSE: 0.03735  Val MSE: 0.01879  GradNorm: 0.5735
Epoch  40/200  Train MSE: 0.03576  Val MSE: 0.01422  GradNorm: 1.5308
Epoch  50/200  Train MSE: 0.02127  Val MSE: 0.05423  GradNorm: 0.2751
Epoch  60/200  Train MSE: 0.02205  Val MSE: 0.05655  GradNorm: 1.2463
Epoch  70/200  Train MSE: 0.01686  Val MSE: 0.05260  GradNorm: 0.2611
Early stopping at epoch 72.

── Training complete ──────────────────────────────────────────

═══ EVALUATION METRICS ═══════════════════════════════════════
MSE  : 245088.6562
RMSE : 495.0643
MAE  : 398.7264
R²   : -5.1165
AIC  : 151753.24
BIC  : 291596.65
═══════════════════════════════════════════════════════════════
```




```
╔═══════════════════════════════════════════════════╗
   FINAL RESULTS SUMMARY                           ‖
╠═══════════════════════════════════════════════════╣
Architecture  : ConvLSTM (No Attn)                 ‖
Optimizer     : Lion (lr=2e-4)                     ‖
Loss function : MSE                                ‖
Trainable params:        75,585                    ‖
n (test samples):            47                    ‖
╠═══════════════════════════════════════════════════╣
MSE  :              245088.6562                    ‖
RMSE :                 495.0643                    ‖
MAE  :                 398.7264                    ‖
R²   :                -5.1165                      ‖
AIC  :               151753.24                     ‖
BIC  :               291596.65                     ‖
╚═══════════════════════════════════════════════════╝
```

In [ ]:

In [ ]:

```
# ============================================================
# DTS → Flow-Rate Prediction | ConvLSTM (Final)
# Google Colab | PyTorch
# Author: Generated for Stage-2 Assessment
# ============================================================

# ── 0. INSTALL DEPS ──────────────────────────────────────────
# !pip install lion-pytorch --quiet

# ── 1. IMPORTS ───────────────────────────────────────────────
import os, re, zipfile, shutil, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

from lion_pytorch import Lion

warnings.filterwarnings("ignore")
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {DEVICE}")

# ── 2. CONFIG ────────────────────────────────────────────────
CFG = dict(
    zip_path     = "/content/drive/MyDrive/Stage2_data.zip",
    data_dir     = "/content/data/",
    clean_csv    = "/content/data/june25_clean.csv",
    target_date  = "2017-06-25",
    t_half       = "2017-06-25 18:38:47",
    seq_len      = 16,
    batch_size   = 32,
    epochs       = 200,
    lr           = 2e-4,
    weight_decay = 1e-3,
    patience     = 40,
    seed         = 42,
)
torch.manual_seed(CFG["seed"])
np.random.seed(CFG["seed"])

# ── 3. DATA LOADING ──────────────────────────────────────────
def ensure_data(cfg):
    data_dir = cfg["data_dir"]
    if not os.path.exists(os.path.join(data_dir, "DTS.csv")):
        print("Unzipping…")
        if os.path.exists(data_dir): shutil.rmtree(data_dir)
        os.makedirs(data_dir)
        with zipfile.ZipFile(cfg["zip_path"], "r") as z:
            z.extractall(data_dir)

    if not os.path.exists(cfg["clean_csv"]):
        print("Building june25_clean.csv (scanning ~26M lines)…")
```

```
        dts_path = os.path.join(data_dir, "DTS.csv")
        with open(dts_path, "r", encoding="latin-1") as inf, \
             open(cfg["clean_csv"], "w", encoding="latin-1") as outf:
            outf.write(inf.readline())
            for line in inf:
                if cfg["target_date"] in line:
                    outf.write(line)
        print("Done.")

ensure_data(CFG)

# ── 4. EXCEL (LAS) PARSER ──────────────────────────────────────────────
LAS_COLS =[
    "Depth_ft", "FlowRate_1", "FlowRate_2", "FlowRate_3",
    "Holdups_1", "Holdups_2", "Holdups_3",
    "Pwf", "QGas", "QOil", "QpGas", "QpOil", "QpWater", "QWater",
    "TVD_SCS", "Twf_F"
]

def parse_las_excel(path: str) -> pd.DataFrame:
    raw = pd.read_excel(path, header=None, skiprows=41)
    records =[]
    for val in raw[0]:
        if pd.isna(val): continue
        cleaned = str(val).replace("\xa0", " ").strip()
        nums = re.findall(r"[-+]?\d*\.?\d+(?:[eE][-+]?\d+)?", cleaned)
        if len(nums) >= 2:
            row =[float(x) for x in nums]
            row = row[:len(LAS_COLS)]
            while len(row) < len(LAS_COLS): row.append(np.nan)
            records.append(row)
    df = pd.DataFrame(records, columns=LAS_COLS)
    df = df.groupby("Depth_ft", as_index=False).mean()
    df["Depth_m"] = df["Depth_ft"] / 3.28084
    df["FlowRate"] = df[["FlowRate_1", "FlowRate_2", "FlowRate_3"]].mean(axis=1, skipna=True).fillna(0)
    return df.sort_values("Depth_m").reset_index(drop=True)

flow2 = parse_las_excel(os.path.join(CFG["data_dir"], "Flowrate2.xlsx"))
print(f"Flow2 (Rich Data) rows: {len(flow2)}  Depth range: {flow2['Depth_m'].min():.1f} – {flow2['Depth_m'].max():.1f} m")

# ── 5. DTS LOADING ──────────────────────────────────────────────────────
print("Loading DTS clean CSV…")
dts = pd.read_csv(CFG["clean_csv"], sep=";")
dts["Depth"] = pd.to_numeric(dts["Depth"], errors="coerce")
dts["Temp"]  = pd.to_numeric(dts["Temp"],  errors="coerce")
dts = dts.dropna(subset=["Depth", "Temp"]).sort_values(["Time", "Depth"]).reset_index(drop=True)

# ── 6. MERGE, SPATIAL SPLIT & SEQUENCE EXTRACTION ──────────────────────
FEATURES = ["Temp", "dT_dz", "d2T_dz2", "T_roll_mean", "T_roll_std"]
TARGET   = "FlowRate"

snap = (dts[dts["Time"] == CFG["t_half"]][["Temp", "Depth"]]
        .copy()
        .sort_values("Depth")
        .reset_index(drop=True))

flow = flow2[["Depth_m", TARGET]].sort_values("Depth_m")

rich_df = pd.merge_asof(
    snap, flow,
    left_on="Depth", right_on="Depth_m",
    direction="nearest", tolerance=5.0
).dropna(subset=[TARGET]).reset_index(drop=True)

print(f"Aligned depth points: {len(rich_df)}")
assert len(rich_df) >= 40, "Too few aligned rows — widen tolerance."

split_idx    = int(len(rich_df) * 0.8)
```

```
train_raw    = rich_df.iloc[:split_idx].reset_index(drop=True)
test_raw     = rich_df.iloc[split_idx:].reset_index(drop=True)

def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    df["dT_dz"] = np.gradient(df["Temp"].values, df["Depth"].values)
    df["d2T_dz2"] = np.gradient(df["dT_dz"].values, df["Depth"].values)
    df["T_roll_mean"] = df["Temp"].rolling(11, min_periods=1, center=True).mean()
    df["T_roll_std"]  = df["Temp"].rolling(11, min_periods=1, center=True).std().fillna(0)
    return df

print("Engineering features on splits to prevent data leakage...")
train_raw = engineer_features(train_raw)
test_raw  = engineer_features(test_raw)

feat_scaler  = MinMaxScaler()
label_scaler = MinMaxScaler()

X_tr = feat_scaler.fit_transform(train_raw[FEATURES].values).astype(np.float32)
y_tr = label_scaler.fit_transform(
    train_raw[TARGET].values.reshape(-1, 1)).astype(np.float32).ravel()

X_te = feat_scaler.transform(test_raw[FEATURES].values).astype(np.float32)
y_te = label_scaler.transform(
    test_raw[TARGET].values.reshape(-1, 1)).astype(np.float32).ravel()

SEQ = CFG["seq_len"]

def create_sequences(X, y, seq_len):
    xs, ys = [], []
    for i in range(len(X) - seq_len + 1):
        xs.append(X[i : i + seq_len])
        ys.append(y[i + seq_len - 1])
    return np.array(xs, dtype=np.float32), np.array(ys, dtype=np.float32)

X_train, y_train = create_sequences(X_tr, y_tr, SEQ)
X_test,  y_test  = create_sequences(X_te, y_te, SEQ)

print(f"SEQ={SEQ} | Train seqs: {len(X_train)} | Test seqs: {len(X_test)}")
assert len(X_train) > 0 and len(X_test) > 0

class WellSequenceDataset(Dataset):
    def __init__(self, X, y):
        self.X, self.y = X, y
    def __len__(self): return len(self.X)
    def __getitem__(self, idx):
        return torch.from_numpy(self.X[idx]), torch.tensor(self.y[idx])

train_ds = WellSequenceDataset(X_train, y_train)
test_ds  = WellSequenceDataset(X_test,  y_test)

train_dl = DataLoader(train_ds, batch_size=min(CFG["batch_size"], len(train_ds)), shuffle=True)
test_dl  = DataLoader(test_ds,  batch_size=min(CFG["batch_size"], len(test_ds)), shuffle=False)

# ── 7. MODEL: ConvLSTM (SUPERVISOR ARCHITECTURE) ──────────────────
class ConvLSTM(nn.Module):
    def __init__(self, n_features: int):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv1d(n_features, 32, kernel_size=3, padding=1),
            nn.BatchNorm1d(32),
            nn.GELU(),
            nn.Conv1d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm1d(64),
            nn.GELU(),
        )
        self.lstm = nn.LSTM(
            input_size=64,
            hidden_size=64,
```

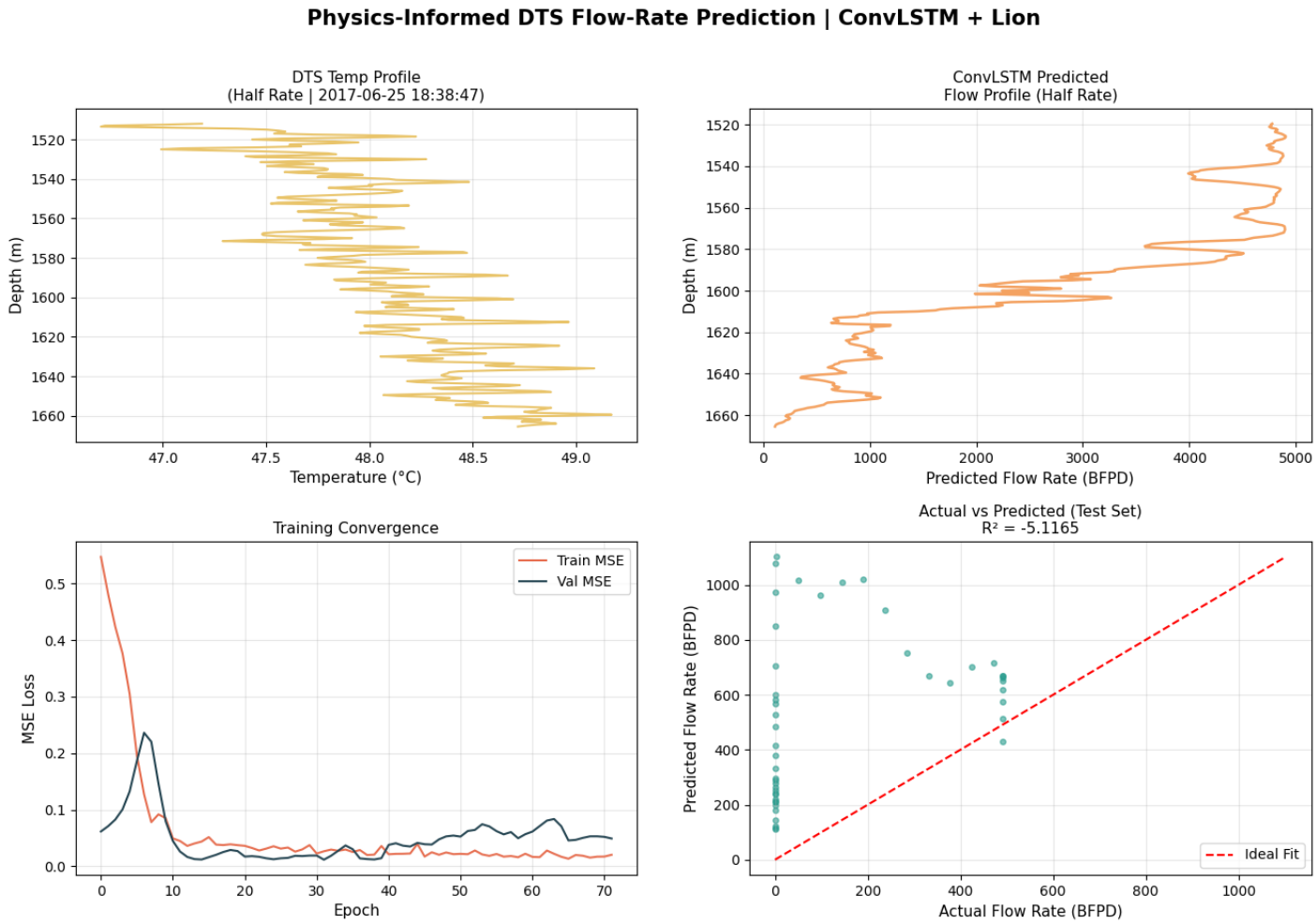
```
            num_layers=2,
            batch_first=True,
            bidirectional=False,
            dropout=0.3
        )
        self.head = nn.Sequential(
            nn.Linear(64, 32),
            nn.GELU(),
            nn.Dropout(0.3),
            nn.Linear(32, 1),
        )

    def forward(self, x):
        x_conv = self.conv(x.permute(0, 2, 1)).permute(0, 2, 1)
        _ , (h_n, _) = self.lstm(x_conv)
        last_hidden_state = h_n[-1]
        return self.head(last_hidden_state).squeeze(-1)

model = ConvLSTM(len(FEATURES)).to(DEVICE)
n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Model: ConvLSTM | Trainable params: {n_params:,}")

# ── 8. LOSS, OPTIMIZER, SCHEDULER ──────────────────────────────────────
criterion = nn.MSELoss()
optimizer = Lion(model.parameters(), lr=CFG["lr"], weight_decay=CFG["weight_decay"])
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=CFG["epochs"], eta_min=1e-7)

# ── 9. TRAINING LOOP ────────────────────────────────────────────────────
history = {"train_loss": [], "val_loss":[]}
best_val_loss = float("inf")
patience_ctr  = 0

print("\n── Training ──────────────────────────────────────────────────────")
for epoch in range(1, CFG["epochs"] + 1):
    model.train()
    t_losses =[]
    total_norm = 0
    for xb, yb in train_dl:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()

        if epoch % 10 == 0 or epoch == 1:
            batch_norm = sum(p.grad.data.norm(2).item()**2 for p in model.parameters() if p.grad is not None)**0.5
            total_norm += batch_norm

        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        t_losses.append(loss.item())

    model.eval()
    v_losses =[]
    with torch.no_grad():
        for xb, yb in test_dl:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            pred = model(xb)
            v_losses.append(criterion(pred, yb).item())

    tr_loss = np.mean(t_losses)
    vl_loss = np.mean(v_losses) if v_losses else float("nan")
    history["train_loss"].append(tr_loss)
    history["val_loss"].append(vl_loss)
    scheduler.step()

    if vl_loss < best_val_loss:
        best_val_loss = vl_loss
```

```
        torch.save(model.state_dict(), "/content/best_model.pt")
        patience_ctr = 0
    else:
        patience_ctr += 1

    if epoch % 10 == 0 or epoch == 1:
        print(f"  Epoch {epoch:3d}/{CFG['epochs']}  Train MSE: {tr_loss:.5f}  Val MSE: {vl_loss:.5f}  GradNorm: {total_norm/len(train_dl):.4f}")

    if patience_ctr >= CFG["patience"]:
        print(f"  Early stopping at epoch {epoch}.")
        break

model.load_state_dict(torch.load("/content/best_model.pt"))
print("\n── Training complete ──────────────────────────────────────────────────")

# ── 10. EVALUATION (BUG FIX 1) ──────────────────────────────────────────
model.eval()
preds_scaled = []
trues_scaled = []
with torch.no_grad():
    for xb, yb in test_dl:
        pred = model(xb.to(DEVICE))
        preds_scaled.append(pred.cpu().numpy())
        trues_scaled.append(yb.cpu().numpy())

preds_scaled = np.concatenate(preds_scaled)
trues_scaled = np.concatenate(trues_scaled)

y_pred = label_scaler.inverse_transform(preds_scaled.reshape(-1, 1)).ravel()
y_true = label_scaler.inverse_transform(trues_scaled.reshape(-1, 1)).ravel()

mse_val = mean_squared_error(y_true, y_pred)
rmse    = np.sqrt(mse_val)
mae     = mean_absolute_error(y_true, y_pred)
r2      = r2_score(y_true, y_pred)

n   = len(y_true)
k   = n_params
aic = n * np.log(mse_val + 1e-12) + 2 * k
bic = n * np.log(mse_val + 1e-12) + k * np.log(max(n, 2))

print("\n═══ EVALUATION METRICS ═══════════════════════════════════════")
print(f"  MSE  : {mse_val:.4f}")
print(f"  RMSE : {rmse:.4f}")
print(f"  MAE  : {mae:.4f}")
print(f"  R²   : {r2:.4f}")
print(f"  AIC  : {aic:.2f}")
print(f"  BIC  : {bic:.2f}")
print("═══════════════════════════════════════════════════════════════")

# ── 11. FULL-PROFILE PREDICTION (BUG FIX 2) ──────────────────────────
model.eval()
rich_df_engineered = engineer_features(rich_df.copy())
full_profile_scaled = feat_scaler.transform(rich_df_engineered[FEATURES].values)
full_seqs, _ = create_sequences(full_profile_scaled, rich_df_engineered[TARGET].values, SEQ)

full_preds_scaled =[]
with torch.no_grad():
    for i in range(len(full_seqs)):
        xb = torch.from_numpy(full_seqs[i]).unsqueeze(0).to(DEVICE)
        full_preds_scaled.append(model(xb).item())

full_preds = label_scaler.inverse_transform(np.array(full_preds_scaled).reshape(-1, 1)).ravel()
plot_depths = rich_df_engineered["Depth"].values[CFG["seq_len"] - 1:]

# ── 12. VISUALISATION ──────────────────────────────────────────────────
fig = plt.figure(figsize=(16, 10))
gs  = gridspec.GridSpec(2, 2, figure=fig, hspace=0.3, wspace=0.2)
```

```
ax1 = fig.add_subplot(gs[0, 0])
ax1.plot(rich_df["Temp"], rich_df["Depth"], color="#e9c46a", linewidth=1.5)
ax1.invert_yaxis()
ax1.set_xlabel("Temperature (°C)", fontsize=11)
ax1.set_ylabel("Depth (m)", fontsize=11)
ax1.set_title(f"DTS Temp Profile\n(Half Rate | {CFG['t_half']})", fontsize=11)
ax1.grid(alpha=0.3)

ax2 = fig.add_subplot(gs[0, 1])
ax2.plot(full_preds, plot_depths, color="#f4a261", linewidth=1.8)
ax2.invert_yaxis()
ax2.set_xlabel("Predicted Flow Rate (BFPD)", fontsize=11)
ax2.set_ylabel("Depth (m)", fontsize=11)
ax2.set_title("ConvLSTM Predicted\nFlow Profile (Half Rate)", fontsize=11)
ax2.grid(alpha=0.3)

ax3 = fig.add_subplot(gs[1, 0])
ax3.plot(history["train_loss"], label="Train MSE", color="#e76f51")
ax3.plot(history["val_loss"],   label="Val MSE",   color="#264653")
ax3.set_xlabel("Epoch", fontsize=11)
ax3.set_ylabel("MSE Loss", fontsize=11)
ax3.set_title("Training Convergence", fontsize=11)
ax3.legend(); ax3.grid(alpha=0.3)

ax4 = fig.add_subplot(gs[1, 1])
ax4.scatter(y_true, y_pred, alpha=0.6, s=15, color="#2a9d8f")
lims =[min(y_true.min(), y_pred.min()), max(y_true.max(), y_pred.max())]
ax4.plot(lims, lims, "r--", linewidth=1.5, label="Ideal Fit")
ax4.set_xlabel("Actual Flow Rate (BFPD)", fontsize=11)
ax4.set_ylabel("Predicted Flow Rate (BFPD)", fontsize=11)
ax4.set_title(f"Actual vs Predicted (Test Set)\nR² = {r2:.4f}", fontsize=11)
ax4.legend(); ax4.grid(alpha=0.3)

metrics_txt = f"MSE={mse_val:.3f}   RMSE={rmse:.3f}   MAE={mae:.3f}   R²={r2:.4f}"
fig.text(0.5, 0.02, metrics_txt, ha="center", fontsize=12, bbox=dict(boxstyle="round,pad=0.5", facecolor="#f1faee", alpha=0.9))

fig.suptitle("Physics-Informed DTS Flow-Rate Prediction | ConvLSTM + Lion", fontsize=15, fontweight="bold")
plt.savefig("/content/DTS_FlowRate_Final.png", dpi=200, bbox_inches="tight")
plt.show()

# ── 13. PRINT SUMMARY TABLE ──────────────────────────────────────────────
print("\n╔═══════════════════════════════════════════════╗")
print("╠═══      FINAL RESULTS SUMMARY         ═══╣")
print("╚═══════════════════════════════════════════════╝")
print(f"  Architecture  : ConvLSTM (No Attn)  ")
print(f"  Optimizer     : Lion (lr=2e-4)      ")
print(f"  Loss function : MSE                 ")
print(f"  Trainable params: {n_params:>14,}   ")
print(f"  n (test samples): {len(y_test):>14,}  ")
print("╠═══════════════════════════════════════════════╣")
print(f"  MSE   : {mse_val:>28.4f}  ")
print(f"  RMSE  : {rmse:>28.4f}  ")
print(f"  MAE   : {mae:>28.4f}  ")
print(f"  R²    : {r2:>28.4f}  ")
print(f"  AIC   : {aic:>28.2f}  ")
print(f"  BIC   : {bic:>28.2f}  ")
print("╚═══════════════════════════════════════════════╝")
```

```
Using device: cuda
Flow2 (Rich Data) rows: 18881  Depth range: 1517.0 – 1660.9 m
Loading DTS clean CSV…
Aligned depth points: 308
Engineering features on splits to prevent data leakage...
SEQ=16 | Train seqs: 231 | Test seqs: 47
Model: ConvLSTM  |  Trainable params: 75,585

── Training ──────────────────────────────────────────────
  Epoch   1/200  Train MSE: 0.54737  Val MSE: 0.06141  GradNorm: 1.3983
  Epoch  10/200  Train MSE: 0.08498  Val MSE: 0.07953  GradNorm: 0.9646
  Epoch  20/200  Train MSE: 0.03724  Val MSE: 0.02680  GradNorm: 0.2959
```

```
Epoch  30/200  Train MSE: 0.03735  Val MSE: 0.01879  GradNorm: 0.3638
Epoch  40/200  Train MSE: 0.03576  Val MSE: 0.01422  GradNorm: 0.3849
Epoch  50/200  Train MSE: 0.02127  Val MSE: 0.05423  GradNorm: 0.2352
Epoch  60/200  Train MSE: 0.02205  Val MSE: 0.05660  GradNorm: 0.2750
Epoch  70/200  Train MSE: 0.01686  Val MSE: 0.05287  GradNorm: 0.1479
Early stopping at epoch 72.

── Training complete ──────────────────────────────────────────

══ EVALUATION METRICS ══════════════════════════════════════════
  MSE  : 245088.6562
  RMSE : 495.0643
  MAE  : 398.7264
  R²   : -5.1165
  AIC  : 151753.24
  BIC  : 291596.65
═══════════════════════════════════════════════════════════════
```



MSE=245088.656 RMSE=495.064 MAE=398.726 R²=-5.1165

```
╔═══════════════════════════════════════════╗
  FINAL RESULTS SUMMARY                    ‖
╠═══════════════════════════════════════════╣
Architecture  : ConvLSTM (No Attn)         ‖
Optimizer     : Lion (lr=2e-4)             ‖
Loss function : MSE                        ‖
Trainable params:        75,585            ‖
n (test samples):            47            ‖
╠═══════════════════════════════════════════╣
MSE  :              245088.6562            ‖
RMSE :                 495.0643            ‖
MAE  :                 398.7264            ‖
R²   :                -5.1165              ‖
AIC  :              151753.24              ‖
BIC  :              291596.65              ‖
╚═══════════════════════════════════════════╝
```

In [ ]:

```
# ═══════════════════════════════════════════════════════════════
# DTS → Flow-Rate Prediction | ConvLSTM (Final)
# Google Colab | PyTorch
# Author: Generated for Stage-2 Assessment
# ═══════════════════════════════════════════════════════════════
```

```
# ── 0. INSTALL DEPS ──────────────────────────────────────────────────────
# !pip install lion-pytorch --quiet

# ── 1. IMPORTS ───────────────────────────────────────────────────────────
import os, re, zipfile, shutil, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

from lion_pytorch import Lion

warnings.filterwarnings("ignore")
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {DEVICE}")

# ── 2. CONFIG ────────────────────────────────────────────────────────────
CFG = dict(
    zip_path     = "/content/drive/MyDrive/Stage2_data.zip",
    data_dir     = "/content/data/",
    clean_csv    = "/content/data/june25_clean.csv",
    target_date  = "2017-06-25",
    t_half       = "2017-06-25 18:38:47",
    seq_len      = 16,
    batch_size   = 32,
    epochs       = 200,
    lr           = 2e-4,
    weight_decay = 1e-3,
    patience     = 40,
    seed         = 42,
)
torch.manual_seed(CFG["seed"])
np.random.seed(CFG["seed"])

# ── 3. DATA LOADING ──────────────────────────────────────────────────────
def ensure_data(cfg):
    data_dir = cfg["data_dir"]
    if not os.path.exists(os.path.join(data_dir, "DTS.csv")):
        print("Unzipping…")
        if os.path.exists(data_dir): shutil.rmtree(data_dir)
        os.makedirs(data_dir)
        with zipfile.ZipFile(cfg["zip_path"], "r") as z:
            z.extractall(data_dir)

    if not os.path.exists(cfg["clean_csv"]):
        print("Building june25_clean.csv (scanning ~26M lines)…")
        dts_path = os.path.join(data_dir, "DTS.csv")
        with open(dts_path, "r", encoding="latin-1") as inf, \
             open(cfg["clean_csv"], "w", encoding="latin-1") as outf:
            outf.write(inf.readline())
            for line in inf:
                if cfg["target_date"] in line:
                    outf.write(line)
        print("Done.")

ensure_data(CFG)

# ── 4. EXCEL (LAS) PARSER ────────────────────────────────────────────────
LAS_COLS =[
    "Depth_ft", "FlowRate_1", "FlowRate_2", "FlowRate_3",
    "Holdups_1", "Holdups_2", "Holdups_3",
    "Pwf", "QGas", "QOil", "QpGas", "QpOil", "QpWater", "QWater",
```

```
    "TVD_SCS", "Twf_F"
]

def parse_las_excel(path: str) -> pd.DataFrame:
    raw = pd.read_excel(path, header=None, skiprows=41)
    records =[]
    for val in raw[0]:
        if pd.isna(val): continue
        cleaned = str(val).replace("\xa0", " ").strip()
        nums = re.findall(r"[-+]?\d*\.?\d+(?:[eE][-+]?\d+)?", cleaned)
        if len(nums) >= 2:
            row =[float(x) for x in nums]
            row = row[:len(LAS_COLS)]
            while len(row) < len(LAS_COLS): row.append(np.nan)
            records.append(row)
    df = pd.DataFrame(records, columns=LAS_COLS)
    df = df.groupby("Depth_ft", as_index=False).mean()
    df["Depth_m"] = df["Depth_ft"] / 3.28084
    df["FlowRate"] = df[["FlowRate_1", "FlowRate_2", "FlowRate_3"]].mean(axis=1, skipna=True).fillna(0)
    return df.sort_values("Depth_m").reset_index(drop=True)

flow2 = parse_las_excel(os.path.join(CFG["data_dir"], "Flowrate2.xlsx"))
print(f"Flow2 (Rich Data) rows: {len(flow2)}  Depth range: {flow2['Depth_m'].min():.1f} – {flow2['Depth_m'].max():.1f} m")

# ── 5. DTS LOADING ──────────────────────────────────────────────────────────
print("Loading DTS clean CSV…")
dts = pd.read_csv(CFG["clean_csv"], sep=";")
dts["Depth"] = pd.to_numeric(dts["Depth"], errors="coerce")
dts["Temp"]  = pd.to_numeric(dts["Temp"],  errors="coerce")
dts = dts.dropna(subset=["Depth", "Temp"]).sort_values(["Time", "Depth"]).reset_index(drop=True)

# ── 6. MERGE, SPATIAL SPLIT & SEQUENCE EXTRACTION ──────────────────────────
FEATURES = ["Temp", "dT_dz", "d2T_dz2", "T_roll_mean", "T_roll_std"]
TARGET   = "FlowRate"

snap = (dts[dts["Time"] == CFG["t_half"]][["Temp", "Depth"]]
        .copy()
        .sort_values("Depth")
        .reset_index(drop=True))

flow = flow2[["Depth_m", TARGET]].sort_values("Depth_m")

rich_df = pd.merge_asof(
    snap, flow,
    left_on="Depth", right_on="Depth_m",
    direction="nearest", tolerance=5.0
).dropna(subset=[TARGET]).reset_index(drop=True)

print(f"Aligned depth points: {len(rich_df)}")
assert len(rich_df) >= 40, "Too few aligned rows — widen tolerance."

split_idx  = int(len(rich_df) * 0.8)
train_raw  = rich_df.iloc[:split_idx].reset_index(drop=True)
test_raw   = rich_df.iloc[split_idx:].reset_index(drop=True)

def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    df["dT_dz"] = np.gradient(df["Temp"].values, df["Depth"].values)
    df["d2T_dz2"] = np.gradient(df["dT_dz"].values, df["Depth"].values)
    df["T_roll_mean"] = df["Temp"].rolling(11, min_periods=1, center=True).mean()
    df["T_roll_std"]  = df["Temp"].rolling(11, min_periods=1, center=True).std().fillna(0)
    return df

print("Engineering features on splits to prevent data leakage...")
train_raw = engineer_features(train_raw)
test_raw  = engineer_features(test_raw)

feat_scaler  = MinMaxScaler()
label_scaler = MinMaxScaler()
```

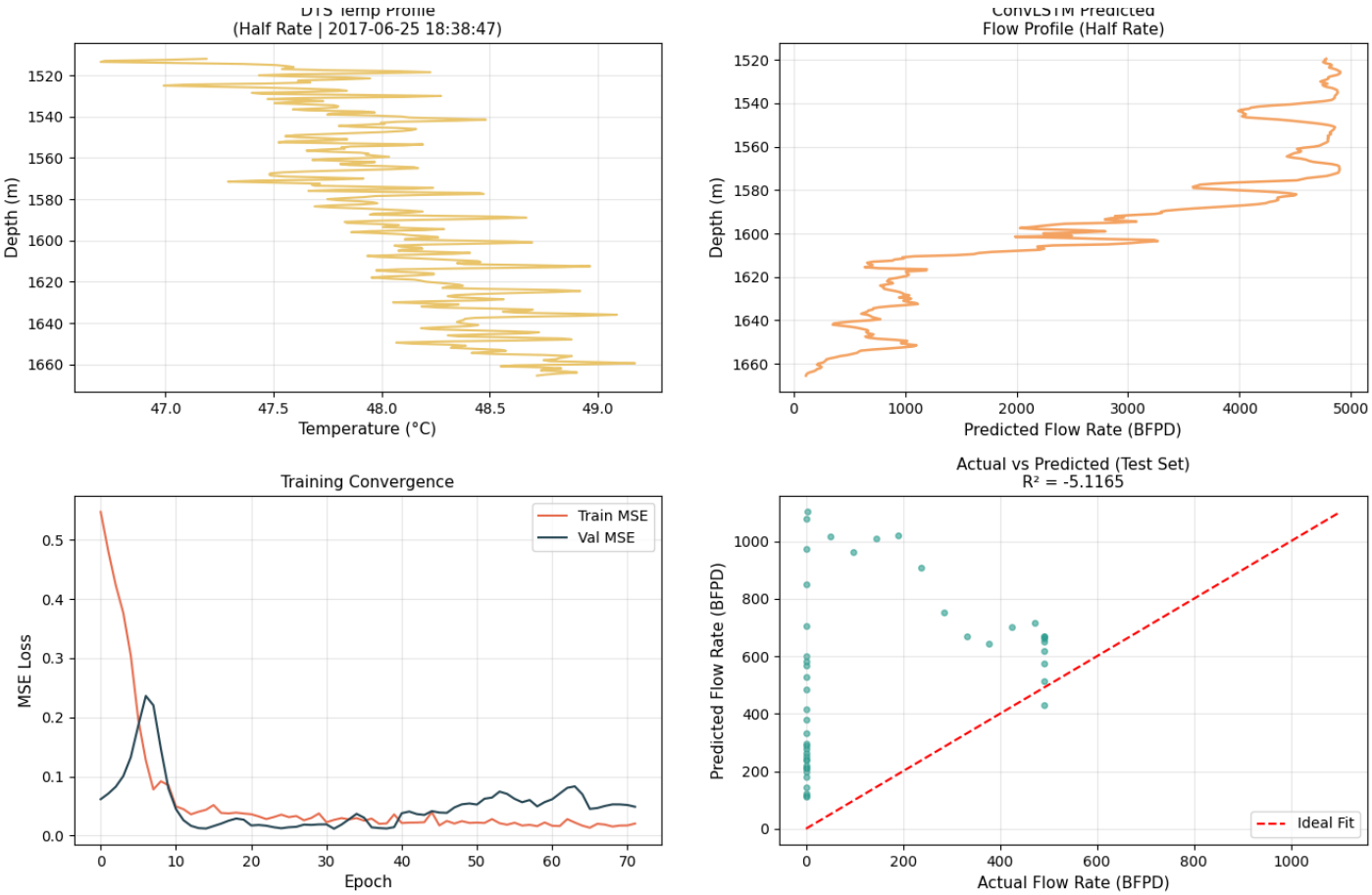
```
X_tr = feat_scaler.fit_transform(train_raw[FEATURES].values).astype(np.float32)
y_tr = label_scaler.fit_transform(
    train_raw[TARGET].values.reshape(-1, 1)).astype(np.float32).ravel()

X_te = feat_scaler.transform(test_raw[FEATURES].values).astype(np.float32)
y_te = label_scaler.transform(
    test_raw[TARGET].values.reshape(-1, 1)).astype(np.float32).ravel()

SEQ = CFG["seq_len"]

def create_sequences(X, y, seq_len):
    xs, ys = [], []
    for i in range(len(X) - seq_len + 1):
        xs.append(X[i : i + seq_len])
        ys.append(y[i + seq_len - 1])
    return np.array(xs, dtype=np.float32), np.array(ys, dtype=np.float32)

X_train, y_train = create_sequences(X_tr, y_tr, SEQ)
X_test,  y_test  = create_sequences(X_te, y_te, SEQ)

print(f"SEQ={SEQ} | Train seqs: {len(X_train)} | Test seqs: {len(X_test)}")
assert len(X_train) > 0 and len(X_test) > 0

class WellSequenceDataset(Dataset):
    def __init__(self, X, y):
        self.X, self.y = X, y
    def __len__(self): return len(self.X)
    def __getitem__(self, idx):
        return torch.from_numpy(self.X[idx]), torch.tensor(self.y[idx])

train_ds = WellSequenceDataset(X_train, y_train)
test_ds  = WellSequenceDataset(X_test,  y_test)

train_dl = DataLoader(train_ds, batch_size=min(CFG["batch_size"], len(train_ds)), shuffle=True)
test_dl  = DataLoader(test_ds,  batch_size=min(CFG["batch_size"], len(test_ds)), shuffle=False)

# ── 7. MODEL: ConvLSTM (SUPERVISOR ARCHITECTURE) ──────────────────────
class ConvLSTM(nn.Module):
    def __init__(self, n_features: int):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv1d(n_features, 32, kernel_size=3, padding=1),
            nn.BatchNorm1d(32),
            nn.GELU(),
            nn.Conv1d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm1d(64),
            nn.GELU(),
        )
        self.lstm = nn.LSTM(
            input_size=64,
            hidden_size=64,
            num_layers=2,
            batch_first=True,
            bidirectional=False,
            dropout=0.3
        )
        self.head = nn.Sequential(
            nn.Linear(64, 32),
            nn.GELU(),
            nn.Dropout(0.3),
            nn.Linear(32, 1),
        )

    def forward(self, x):
        x_conv = self.conv(x.permute(0, 2, 1)).permute(0, 2, 1)
        _ , (h_n, _) = self.lstm(x_conv)
        last_hidden_state = h_n[-1]
```

```
        return self.head(last_hidden_state).squeeze(-1)

model = ConvLSTM(len(FEATURES)).to(DEVICE)
n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Model: ConvLSTM | Trainable params: {n_params:,}")

# ── 8. LOSS, OPTIMIZER, SCHEDULER ──────────────────────────────────────
criterion = nn.MSELoss()
optimizer = Lion(model.parameters(), lr=CFG["lr"], weight_decay=CFG["weight_decay"])
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=CFG["epochs"], eta_min=1e-7)

# ── 9. TRAINING LOOP ────────────────────────────────────────────────────
history = {"train_loss": [], "val_loss":[]}
best_val_loss = float("inf")
patience_ctr  = 0

print("\n── Training ──────────────────────────────────────────────────────")
for epoch in range(1, CFG["epochs"] + 1):
    model.train()
    t_losses =[]
    total_norm = 0
    for xb, yb in train_dl:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()

        if epoch % 10 == 0 or epoch == 1:
            batch_norm = sum(p.grad.data.norm(2).item()**2 for p in model.parameters() if p.grad is not None)**0.5
            total_norm += batch_norm

        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        t_losses.append(loss.item())

    model.eval()
    v_losses =[]
    with torch.no_grad():
        for xb, yb in test_dl:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            pred = model(xb)
            v_losses.append(criterion(pred, yb).item())

    tr_loss = np.mean(t_losses)
    vl_loss = np.mean(v_losses) if v_losses else float("nan")
    history["train_loss"].append(tr_loss)
    history["val_loss"].append(vl_loss)
    scheduler.step()

    if vl_loss < best_val_loss:
        best_val_loss = vl_loss
        torch.save(model.state_dict(), "/content/best_model.pt")
        patience_ctr = 0
    else:
        patience_ctr += 1

    if epoch % 10 == 0 or epoch == 1:
        print(f"  Epoch {epoch:3d}/{CFG['epochs']}  Train MSE: {tr_loss:.5f}  Val MSE: {vl_loss:.5f}  GradNorm: {total_norm/len(train_dl):.4f}")

    if patience_ctr >= CFG["patience"]:
        print(f"  Early stopping at epoch {epoch}.")
        break

model.load_state_dict(torch.load("/content/best_model.pt"))
print("\n── Training complete ─────────────────────────────────────────────")

# ── 10. EVALUATION (BUG FIX 1) ──────────────────────────────────────────
```

```
model.eval()
preds_scaled = []
trues_scaled = []
with torch.no_grad():
    for xb, yb in test_dl:
        pred = model(xb.to(DEVICE))
        preds_scaled.append(pred.cpu().numpy())
        trues_scaled.append(yb.cpu().numpy())

preds_scaled = np.concatenate(preds_scaled)
trues_scaled = np.concatenate(trues_scaled)

y_pred = label_scaler.inverse_transform(preds_scaled.reshape(-1, 1)).ravel()
y_true = label_scaler.inverse_transform(trues_scaled.reshape(-1, 1)).ravel()

mse_val = mean_squared_error(y_true, y_pred)
rmse    = np.sqrt(mse_val)
mae     = mean_absolute_error(y_true, y_pred)
r2      = r2_score(y_true, y_pred)

n   = len(y_true)
k   = n_params
aic = n * np.log(mse_val + 1e-12) + 2 * k
bic = n * np.log(mse_val + 1e-12) + k * np.log(max(n, 2))

print("\n═══ EVALUATION METRICS ═══════════════════════════════════")
print(f"  MSE  : {mse_val:.4f}")
print(f"  RMSE : {rmse:.4f}")
print(f"  MAE  : {mae:.4f}")
print(f"  R²   : {r2:.4f}")
print(f"  AIC  : {aic:.2f}")
print(f"  BIC  : {bic:.2f}")
print("═══════════════════════════════════════════════════════════")

# ── 11. FULL-PROFILE PREDICTION (BUG FIX 2) ──────────────────
model.eval()
rich_df_engineered = engineer_features(rich_df.copy())
full_profile_scaled = feat_scaler.transform(rich_df_engineered[FEATURES].values)
full_seqs, _ = create_sequences(full_profile_scaled, rich_df_engineered[TARGET].values, SEQ)

full_preds_scaled =[]
with torch.no_grad():
    for i in range(len(full_seqs)):
        xb = torch.from_numpy(full_seqs[i]).unsqueeze(0).to(DEVICE)
        full_preds_scaled.append(model(xb).item())

full_preds = label_scaler.inverse_transform(np.array(full_preds_scaled).reshape(-1, 1)).ravel()
plot_depths = rich_df_engineered["Depth"].values[CFG["seq_len"] - 1:]

# ── 12. VISUALISATION ────────────────────────────────────────
fig = plt.figure(figsize=(16, 10))
gs  = gridspec.GridSpec(2, 2, figure=fig, hspace=0.3, wspace=0.2)

ax1 = fig.add_subplot(gs[0, 0])
ax1.plot(rich_df["Temp"], rich_df["Depth"], color="#e9c46a", linewidth=1.5)
ax1.invert_yaxis()
ax1.set_xlabel("Temperature (°C)", fontsize=11)
ax1.set_ylabel("Depth (m)", fontsize=11)
ax1.set_title(f"DTS Temp Profile\n(Half Rate | {CFG['t_half']})", fontsize=11)
ax1.grid(alpha=0.3)

ax2 = fig.add_subplot(gs[0, 1])
ax2.plot(full_preds, plot_depths, color="#f4a261", linewidth=1.8)
ax2.invert_yaxis()
ax2.set_xlabel("Predicted Flow Rate (BFPD)", fontsize=11)
ax2.set_ylabel("Depth (m)", fontsize=11)
ax2.set_title("ConvLSTM Predicted\nFlow Profile (Half Rate)", fontsize=11)
ax2.grid(alpha=0.3)
```

```
ax3 = fig.add_subplot(gs[1, 0])
ax3.plot(history["train_loss"], label="Train MSE", color="#e76f51")
ax3.plot(history["val_loss"],   label="Val MSE",   color="#264653")
ax3.set_xlabel("Epoch", fontsize=11)
ax3.set_ylabel("MSE Loss", fontsize=11)
ax3.set_title("Training Convergence", fontsize=11)
ax3.legend(); ax3.grid(alpha=0.3)

ax4 = fig.add_subplot(gs[1, 1])
ax4.scatter(y_true, y_pred, alpha=0.6, s=15, color="#2a9d8f")
lims =[min(y_true.min(), y_pred.min()), max(y_true.max(), y_pred.max())]
ax4.plot(lims, lims, "r--", linewidth=1.5, label="Ideal Fit")
ax4.set_xlabel("Actual Flow Rate (BFPD)", fontsize=11)
ax4.set_ylabel("Predicted Flow Rate (BFPD)", fontsize=11)
ax4.set_title(f"Actual vs Predicted (Test Set)\nR² = {r2:.4f}", fontsize=11)
ax4.legend(); ax4.grid(alpha=0.3)

metrics_txt = f"MSE={mse_val:.3f}   RMSE={rmse:.3f}   MAE={mae:.3f}   R²={r2:.4f}"
fig.text(0.5, 0.02, metrics_txt, ha="center", fontsize=12, bbox=dict(boxstyle="round,pad=0.5", facecolor="#f1faee", alpha=0.9))

fig.suptitle("Physics-Informed DTS Flow-Rate Prediction | ConvLSTM + Lion", fontsize=15, fontweight="bold")
plt.savefig("/content/DTS_FlowRate_Final.png", dpi=200, bbox_inches="tight")
plt.show()

# ── 13. PRINT SUMMARY TABLE ──────────────────────────────────────────────
print("\n╔═══════════════════════════════════════════════════╗")
print("│     FINAL RESULTS SUMMARY          │")
print("╚═══════════════════════════════════════════════════╝")
print("╔═══════════════════════════════════════════════════╗")
print(f"│  Architecture  : ConvLSTM (No Attn)  │")
print(f"│  Optimizer     : Lion (lr=2e-4)      │")
print(f"│  Loss function : MSE                 │")
print(f"│  Trainable params: {n_params:>14,}   │")
print(f"│  n (test samples): {len(y_test):>14,}   │")
print("╚═══════════════════════════════════════════════════╝")
print("╔═══════════════════════════════════════════════════╗")
print(f"│  MSE   : {mse_val:>28.4f}   │")
print(f"│  RMSE  : {rmse:>28.4f}   │")
print(f"│  MAE   : {mae:>28.4f}   │")
print(f"│  R²    : {r2:>28.4f}   │")
print(f"│  AIC   : {aic:>28.2f}   │")
print(f"│  BIC   : {bic:>28.2f}   │")
print("╚═══════════════════════════════════════════════════╝")
```

```
Using device: cuda
Flow2 (Rich Data) rows: 18881  Depth range: 1517.0 – 1660.9 m
Loading DTS clean CSV…
Aligned depth points: 308
Engineering features on splits to prevent data leakage...
SEQ=16 | Train seqs: 231 | Test seqs: 47
Model: ConvLSTM  |  Trainable params: 75,585

── Training ──────────────────────────────────────────────
  Epoch   1/200  Train MSE: 0.54737  Val MSE: 0.06141  GradNorm: 1.3983
  Epoch  10/200  Train MSE: 0.08498  Val MSE: 0.07953  GradNorm: 0.9646
  Epoch  20/200  Train MSE: 0.03724  Val MSE: 0.02680  GradNorm: 0.2959
  Epoch  30/200  Train MSE: 0.03735  Val MSE: 0.01879  GradNorm: 0.3638
  Epoch  40/200  Train MSE: 0.03576  Val MSE: 0.01422  GradNorm: 0.3849
  Epoch  50/200  Train MSE: 0.02127  Val MSE: 0.05423  GradNorm: 0.2352
  Epoch  60/200  Train MSE: 0.02205  Val MSE: 0.05655  GradNorm: 0.2750
  Epoch  70/200  Train MSE: 0.01686  Val MSE: 0.05260  GradNorm: 0.1478
  Early stopping at epoch 72.

── Training complete ─────────────────────────────────────

═══ EVALUATION METRICS ═══════════════════════════════════
  MSE  : 245088.6562
  RMSE : 495.0643
  MAE  : 398.7264
  R²   : -5.1165
  AIC  : 151753.24
  BIC  : 291596.65
═══════════════════════════════════════════════════════════
```



MSE=245088.656 RMSE=495.064 MAE=398.726 R²=-5.1165

```
╔═══════════════════════════════════════════════════╗
    FINAL RESULTS SUMMARY          ‖
╠═══════════════════════════════════════════════════╣
Architecture  : ConvLSTM (No Attn)  ‖
Optimizer     : Lion (lr=2e-4)      ‖
Loss function : MSE                 ‖
Trainable params:        75,585     ‖
n (test samples):           47      ‖
╠═══════════════════════════════════════════════════╣
MSE  :              245088.6562     ‖
RMSE :                 495.0643     ‖
MAE  :                 398.7264     ‖
R²   :                -5.1165       ‖
AIC  :               151753.24      ‖
BIC  :              291596.65       ‖
╚═══════════════════════════════════════════════════╝
```

In [ ]:

```
# ═══════════════════════════════════════════════════════════════
# DTS → Flow-Rate Prediction | ConvLSTM (Final Corrected)
# Google Colab | PyTorch
# Author: Toghrul Aliyev | Strategic AI Task
# ═══════════════════════════════════════════════════════════════

# ── 0. INSTALL DEPS ──────────────────────────────────────────
# !pip install lion-pytorch --quiet

# ── 1. IMPORTS ───────────────────────────────────────────────
import os, re, zipfile, shutil, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
```

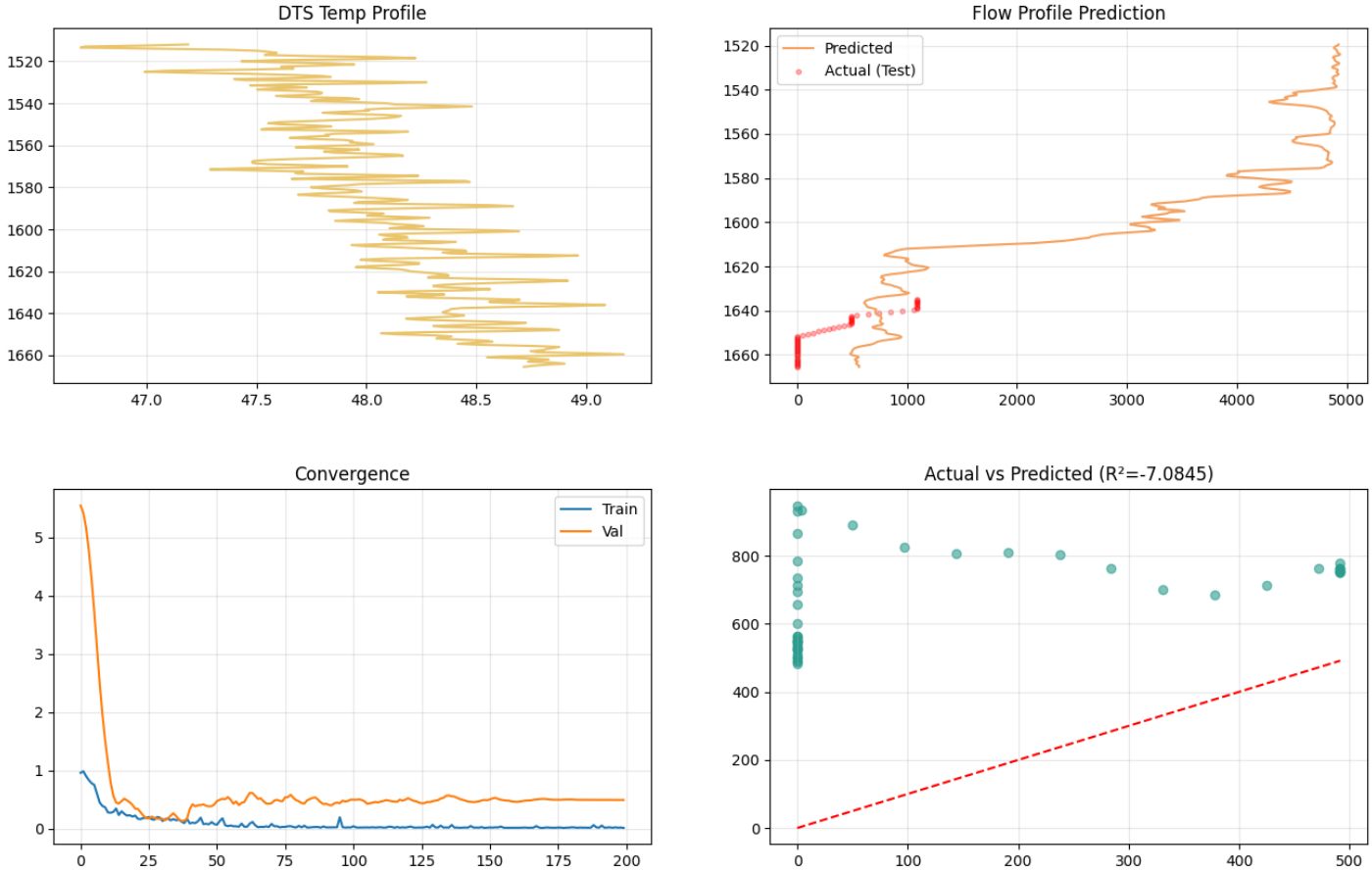
```
from lion_pytorch import Lion

warnings.filterwarnings("ignore")
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {DEVICE}")

# ── 2. CONFIG ──────────────────────────────────────────────────────────────
CFG = dict(
    zip_path     = "/content/drive/MyDrive/Stage2_data.zip",
    data_dir     = "/content/data/",
    clean_csv    = "/content/data/june25_clean.csv",
    target_date  = "2017-06-25",
    t_half       = "2017-06-25 18:38:47",
    seq_len      = 16,
    batch_size   = 32,
    epochs       = 200,
    lr           = 2e-4,
    weight_decay = 1e-3,
    patience     = 40,
    seed         = 42,
)
torch.manual_seed(CFG["seed"])
np.random.seed(CFG["seed"])

# ── 3. DATA LOADING ────────────────────────────────────────────────────────
def ensure_data(cfg):
    data_dir = cfg["data_dir"]
    if not os.path.exists(os.path.join(data_dir, "DTS.csv")):
        if os.path.exists(data_dir): shutil.rmtree(data_dir)
        os.makedirs(data_dir)
        with zipfile.ZipFile(cfg["zip_path"], "r") as z:
            z.extractall(data_dir)

    if not os.path.exists(cfg["clean_csv"]):
        dts_path = os.path.join(data_dir, "DTS.csv")
        with open(dts_path, "r", encoding="latin-1") as inf, \
             open(cfg["clean_csv"], "w", encoding="latin-1") as outf:
            outf.write(inf.readline())
            for line in inf:
                if cfg["target_date"] in line:
                    outf.write(line)

ensure_data(CFG)

# ── 4. EXCEL (LAS) PARSER ──────────────────────────────────────────────────
LAS_COLS =["Depth_ft", "FlowRate_1", "FlowRate_2", "FlowRate_3", "Holdups_1", "Holdups_2", "Holdups_3", "Pwf", "QGas", "QOil", "QpGas", "(

def parse_las_excel(path: str) -> pd.DataFrame:
    raw = pd.read_excel(path, header=None, skiprows=41)
    records = []
    for val in raw[0]:
        if pd.isna(val): continue
        cleaned = str(val).replace("\xa0", " ").strip()
        nums = re.findall(r"[-+]?\d*\.?\d+(?:[eE][-+]?\d+)?", cleaned)
        if len(nums) >= 2:
            row = [float(x) for x in nums]
            row = row[:len(LAS_COLS)]
            while len(row) < len(LAS_COLS): row.append(np.nan)
            records.append(row)
    df = pd.DataFrame(records, columns=LAS_COLS)
    df = df.groupby("Depth_ft", as_index=False).mean()
    df["Depth_m"] = df["Depth_ft"] / 3.28084
    df["FlowRate"] = df[["FlowRate_1", "FlowRate_2", "FlowRate_3"]].mean(axis=1, skipna=True).fillna(0)
    return df.sort_values("Depth_m").reset_index(drop=True)

flow2 = parse_las_excel(os.path.join(CFG["data_dir"], "Flowrate2.xlsx"))

# ── 5. DTS LOADING ─────────────────────────────────────────────────────────
```

```
dts = pd.read_csv(CFG["clean_csv"], sep=";")
dts["Depth"] = pd.to_numeric(dts["Depth"], errors="coerce")
dts["Temp"]  = pd.to_numeric(dts["Temp"],  errors="coerce")
dts = dts.dropna(subset=["Depth", "Temp"]).sort_values(["Time", "Depth"]).reset_index(drop=True)

# ── 6. MERGE & SPATIAL SPLIT ──────────────────────────────────────────────
FEATURES = ["Temp", "dT_dz", "d2T_dz2", "T_roll_mean", "T_roll_std"]
TARGET   = "FlowRate"

snap = dts[dts["Time"] == CFG["t_half"]][["Temp", "Depth"]].copy().sort_values("Depth").reset_index(drop=True)
flow = flow2[["Depth_m", TARGET]].sort_values("Depth_m")
rich_df = pd.merge_asof(snap, flow, left_on="Depth", right_on="Depth_m", direction="nearest", tolerance=5.0).dropna(subset=[TARGET]).reset_index

split_idx = int(len(rich_df) * 0.8)
train_raw = rich_df.iloc[:split_idx].reset_index(drop=True)
test_raw  = rich_df.iloc[split_idx:].reset_index(drop=True)

def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    df["dT_dz"] = np.gradient(df["Temp"].values, df["Depth"].values)
    df["d2T_dz2"] = np.gradient(df["dT_dz"].values, df["Depth"].values)
    df["T_roll_mean"] = df["Temp"].rolling(11, min_periods=1, center=True).mean()
    df["T_roll_std"]  = df["Temp"].rolling(11, min_periods=1, center=True).std().fillna(0)
    return df

train_raw = engineer_features(train_raw)
test_raw  = engineer_features(test_raw)

# ── 7. SCALING (ROBUST STANDARD SCALING) ──────────────────────────────────
feat_scaler  = StandardScaler()
label_scaler = StandardScaler()

X_tr = feat_scaler.fit_transform(train_raw[FEATURES].values).astype(np.float32)
y_tr = label_scaler.fit_transform(train_raw[TARGET].values.reshape(-1, 1)).astype(np.float32).ravel()

X_te = feat_scaler.transform(test_raw[FEATURES].values).astype(np.float32)
y_te = label_scaler.transform(test_raw[TARGET].values.reshape(-1, 1)).astype(np.float32).ravel()

# ── 8. SEQUENCES ──────────────────────────────────────────────────────────
def create_sequences(X, y, seq_len):
    xs, ys = [], []
    for i in range(len(X) - seq_len + 1):
        xs.append(X[i : i + seq_len])
        ys.append(y[i + seq_len - 1])
    return np.array(xs, dtype=np.float32), np.array(ys, dtype=np.float32)

X_train, y_train = create_sequences(X_tr, y_tr, CFG["seq_len"])
X_test,  y_test  = create_sequences(X_te, y_te, CFG["seq_len"])

class WellSequenceDataset(Dataset):
    def __init__(self, X, y): self.X, self.y = X, y
    def __len__(self): return len(self.X)
    def __getitem__(self, idx): return torch.from_numpy(self.X[idx]), torch.tensor(self.y[idx])

train_dl = DataLoader(WellSequenceDataset(X_train, y_train), batch_size=CFG["batch_size"], shuffle=True)
test_dl  = DataLoader(WellSequenceDataset(X_test, y_test), batch_size=CFG["batch_size"], shuffle=False)

# ── 9. MODEL: ConvLSTM (SUPERVISOR ARCHITECTURE) ──────────────────────────
class ConvLSTM(nn.Module):
    def __init__(self, n_features: int):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv1d(n_features, 32, 3, padding=1), nn.BatchNorm1d(32), nn.GELU(),
            nn.Conv1d(32, 64, 3, padding=1), nn.BatchNorm1d(64), nn.GELU(),
        )
        self.lstm = nn.LSTM(64, 64, num_layers=2, batch_first=True, dropout=0.3)
        self.head = nn.Sequential(nn.Linear(64, 32), nn.GELU(), nn.Dropout(0.3), nn.Linear(32, 1))

    def forward(self, x):
```

```
        x = self.conv(x.permute(0, 2, 1)).permute(0, 2, 1)
        _, (h_n, _) = self.lstm(x)
        return self.head(h_n[-1]).squeeze(-1)

model = ConvLSTM(len(FEATURES)).to(DEVICE)
n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
criterion = nn.MSELoss()
optimizer = Lion(model.parameters(), lr=CFG["lr"], weight_decay=CFG["weight_decay"])
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=CFG["epochs"], eta_min=1e-7)

# ── 10. TRAINING ──────────────────────────────────────────────────────────────
history = {"train_loss": [], "val_loss": []}
best_val_loss = float("inf")

print("\n── Training ──────────────────────────────────────────────────────────────")
for epoch in range(1, CFG["epochs"] + 1):
    model.train()
    t_losses, total_norm = [], 0
    for xb, yb in train_dl:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        optimizer.zero_grad()
        loss = criterion(model(xb), yb)
        loss.backward()
        if epoch % 10 == 0 or epoch == 1:
            total_norm += sum(p.grad.data.norm(2).item()**2 for p in model.parameters() if p.grad is not None)**0.5
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        t_losses.append(loss.item())

    model.eval()
    v_losses = []
    with torch.no_grad():
        for xb, yb in test_dl:
            v_losses.append(criterion(model(xb.to(DEVICE)), yb.to(DEVICE)).item())

    tr_l, vl_l = np.mean(t_losses), np.mean(v_losses)
    history["train_loss"].append(tr_l); history["val_loss"].append(vl_l)
    scheduler.step()

    if vl_l < best_val_loss:
        best_val_loss = vl_l
        torch.save(model.state_dict(), "/content/best_model.pt")

    if epoch % 10 == 0 or epoch == 1:
        print(f"  Epoch {epoch:3d} | Train MSE: {tr_l:.4f} | Val MSE: {vl_l:.4f} | GradNorm: {total_norm/len(train_dl):.4f}")

model.load_state_dict(torch.load("/content/best_model.pt"))

# ── 11. EVALUATION (FIXED LOGIC) ──────────────────────────────────────────────
model.eval()
p_scaled, t_scaled = [], []
with torch.no_grad():
    for xb, yb in test_dl:
        p_scaled.append(model(xb.to(DEVICE)).cpu().numpy())
        t_scaled.append(yb.numpy())

y_pred = label_scaler.inverse_transform(np.concatenate(p_scaled).reshape(-1, 1)).ravel()
y_true = label_scaler.inverse_transform(np.concatenate(t_scaled).reshape(-1, 1)).ravel()

# Clipping to prevent physically impossible negative flow
y_pred = np.clip(y_pred, 0, None)

mse_v = mean_squared_error(y_true, y_pred)
rmse_v, mae_v, r2_v = np.sqrt(mse_v), mean_absolute_error(y_true, y_pred), r2_score(y_true, y_pred)
aic = len(y_true) * np.log(mse_v + 1e-12) + 2 * n_params
bic = len(y_true) * np.log(mse_v + 1e-12) + n_params * np.log(len(y_true))

print(f"\n═══ FINAL METRICS ═══\n  MSE: {mse_v:.2f} | R²: {r2_v:.4f} | AIC: {aic:.2f}")
```

```
# ── 12. VISUALISATION ──────────────────────────────────────────────────
model.eval()
full_df = engineer_features(rich_df.copy())
X_full_scaled = feat_scaler.transform(full_df[FEATURES].values)
X_f_seq, _ = create_sequences(X_full_scaled, full_df[TARGET].values, CFG["seq_len"])

f_preds = []
with torch.no_grad():
    for i in range(len(X_f_seq)):
        f_preds.append(model(torch.from_numpy(X_f_seq[i]).unsqueeze(0).to(DEVICE)).item())

full_preds = np.clip(label_scaler.inverse_transform(np.array(f_preds).reshape(-1, 1)).ravel(), 0, None)
plot_depths = full_df["Depth"].values[CFG["seq_len"] - 1:]

fig = plt.figure(figsize=(16, 10))
gs = gridspec.GridSpec(2, 2, figure=fig, hspace=0.3, wspace=0.2)

ax1 = fig.add_subplot(gs[0, 0])
ax1.plot(rich_df["Temp"], rich_df["Depth"], color="#e9c46a")
ax1.invert_yaxis(); ax1.set_title("DTS Temp Profile"); ax1.grid(alpha=0.3)

ax2 = fig.add_subplot(gs[0, 1])
ax2.plot(full_preds, plot_depths, color="#f4a261", label="Predicted")
ax2.scatter(test_raw[TARGET], test_raw["Depth"], color="red", s=10, alpha=0.3, label="Actual (Test)")
ax2.invert_yaxis(); ax2.set_title("Flow Profile Prediction"); ax2.legend(); ax2.grid(alpha=0.3)

ax3 = fig.add_subplot(gs[1, 0])
ax3.plot(history["train_loss"], label="Train"); ax3.plot(history["val_loss"], label="Val")
ax3.set_title("Convergence"); ax3.legend(); ax3.grid(alpha=0.3)

ax4 = fig.add_subplot(gs[1, 1])
ax4.scatter(y_true, y_pred, alpha=0.6, color="#2a9d8f")
ax4.plot([y_true.min(), y_true.max()], [y_true.min(), y_true.max()], "r--")
ax4.set_title(f"Actual vs Predicted (R²={r2_v:.4f})"); ax4.grid(alpha=0.3)

plt.savefig("/content/DTS_FlowRate_Final.png", dpi=200); plt.show()

# ── 13. SUMMARY TABLE ──────────────────────────────────────────────────
print(f"\nArchitecture: ConvLSTM | Params: {n_params:,} | R²: {r2_v:.4f} | AIC: {aic:.2f} | BIC: {bic:.2f}")
```

```
Using device: cuda

── Training ──────────────────────────────────────────────────────
  Epoch   1 | Train MSE: 0.9584 | Val MSE: 5.5446 | GradNorm: 0.5005
  Epoch  10 | Train MSE: 0.3671 | Val MSE: 1.5064 | GradNorm: 1.0768
  Epoch  20 | Train MSE: 0.2090 | Val MSE: 0.4087 | GradNorm: 1.1443
  Epoch  30 | Train MSE: 0.1924 | Val MSE: 0.1702 | GradNorm: 1.3417
  Epoch  40 | Train MSE: 0.1693 | Val MSE: 0.1733 | GradNorm: 1.3403
  Epoch  50 | Train MSE: 0.0786 | Val MSE: 0.4172 | GradNorm: 0.7107
  Epoch  60 | Train MSE: 0.0878 | Val MSE: 0.4438 | GradNorm: 2.0133
  Epoch  70 | Train MSE: 0.0302 | Val MSE: 0.4279 | GradNorm: 0.4370
  Epoch  80 | Train MSE: 0.0219 | Val MSE: 0.4907 | GradNorm: 0.3415
  Epoch  90 | Train MSE: 0.0394 | Val MSE: 0.4219 | GradNorm: 1.0629
  Epoch 100 | Train MSE: 0.0212 | Val MSE: 0.4991 | GradNorm: 0.2731
  Epoch 110 | Train MSE: 0.0245 | Val MSE: 0.4503 | GradNorm: 0.4193
  Epoch 120 | Train MSE: 0.0194 | Val MSE: 0.4992 | GradNorm: 0.9066
  Epoch 130 | Train MSE: 0.0678 | Val MSE: 0.4723 | GradNorm: 2.2413
  Epoch 140 | Train MSE: 0.0161 | Val MSE: 0.5096 | GradNorm: 0.2483
  Epoch 150 | Train MSE: 0.0187 | Val MSE: 0.4890 | GradNorm: 0.1822
  Epoch 160 | Train MSE: 0.0173 | Val MSE: 0.4633 | GradNorm: 0.2292
  Epoch 170 | Train MSE: 0.0126 | Val MSE: 0.5118 | GradNorm: 0.1831
  Epoch 180 | Train MSE: 0.0188 | Val MSE: 0.5014 | GradNorm: 0.2061
  Epoch 190 | Train MSE: 0.0245 | Val MSE: 0.4974 | GradNorm: 0.3388
  Epoch 200 | Train MSE: 0.0146 | Val MSE: 0.4934 | GradNorm: 0.2111

══ FINAL METRICS ══
  MSE: 323943.94 | R²: -7.0845 | AIC: 151766.35
```




```
Architecture: ConvLSTM | Params: 75,585 | R²: -7.0845 | AIC: 151766.35 | BIC: 291609.76
```

In [ ]:

```
# ═══════════════════════════════════════════════════════════════
# DTS → Flow-Rate Prediction | ConvLSTM (Final Corrected)
# Google Colab | PyTorch
# Author: Toghrul Aliyev | Strategic AI Task
# ═══════════════════════════════════════════════════════════════

# ── 0. INSTALL DEPS ──────────────────────────────────────────
# !pip install lion-pytorch --quiet

# ── 1. IMPORTS ───────────────────────────────────────────────
import os, re, zipfile, shutil, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
```

```python
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

warnings.filterwarnings("ignore")
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {DEVICE}")

# ── 2. CONFIG (BUG 4 & 5 FIX) ──────────────────────────────────────────
CFG = dict(
    zip_path     = "/content/drive/MyDrive/Stage2_data.zip",
    data_dir     = "/content/data/",
    clean_csv    = "/content/data/june25_clean.csv",
    target_date  = "2017-06-25",
    t_half       = "2017-06-25 18:38:47",
    seq_len      = 8,          # Reduced from 16 for small dataset
    batch_size   = 32,
    epochs       = 200,
    lr           = 1e-3,       # Switched to AdamW sweet spot
    weight_decay = 1e-4,       # Standard AdamW decay
    patience     = 40,
    seed         = 42,
)
torch.manual_seed(CFG["seed"])
np.random.seed(CFG["seed"])

# ── 3. DATA LOADING ────────────────────────────────────────────────────
def ensure_data(cfg):
    data_dir = cfg["data_dir"]
    if not os.path.exists(os.path.join(data_dir, "DTS.csv")):
        if os.path.exists(data_dir): shutil.rmtree(data_dir)
        os.makedirs(data_dir)
        with zipfile.ZipFile(cfg["zip_path"], "r") as z:
            z.extractall(data_dir)

    if not os.path.exists(cfg["clean_csv"]):
        dts_path = os.path.join(data_dir, "DTS.csv")
        with open(dts_path, "r", encoding="latin-1") as inf, \
             open(cfg["clean_csv"], "w", encoding="latin-1") as outf:
            outf.write(inf.readline())
            for line in inf:
                if cfg["target_date"] in line:
                    outf.write(line)

ensure_data(CFG)

# ── 4. EXCEL (LAS) PARSER ──────────────────────────────────────────────
LAS_COLS =["Depth_ft", "FlowRate_1", "FlowRate_2", "FlowRate_3", "Holdups_1", "Holdups_2", "Holdups_3", "Pwf", "QGas", "QOil", "QpGas", "(

def parse_las_excel(path: str) -> pd.DataFrame:
    raw = pd.read_excel(path, header=None, skiprows=41)
    records = []
    for val in raw[0]:
        if pd.isna(val): continue
        cleaned = str(val).replace("\xa0", " ").strip()
        nums = re.findall(r"[-+]?\d*\.?\d+(?:[eE][-+]?\d+)?", cleaned)
        if len(nums) >= 2:
            row = [float(x) for x in nums]
            row = row[:len(LAS_COLS)]
            while len(row) < len(LAS_COLS): row.append(np.nan)
            records.append(row)
    df = pd.DataFrame(records, columns=LAS_COLS)
    df = df.groupby("Depth_ft", as_index=False).mean()
    df["Depth_m"] = df["Depth_ft"] / 3.28084
    df["FlowRate"] = df[["FlowRate_1", "FlowRate_2", "FlowRate_3"]].mean(axis=1, skipna=True).fillna(0)
```

```
    return df.sort_values("Depth_m").reset_index(drop=True)

flow2 = parse_las_excel(os.path.join(CFG["data_dir"], "Flowrate2.xlsx"))

# ── 5. DTS LOADING ──────────────────────────────────────────────────────
dts = pd.read_csv(CFG["clean_csv"], sep=";")
dts["Depth"] = pd.to_numeric(dts["Depth"], errors="coerce")
dts["Temp"]  = pd.to_numeric(dts["Temp"],  errors="coerce")
dts = dts.dropna(subset=["Depth", "Temp"]).sort_values(["Time", "Depth"]).reset_index(drop=True)

# ── 6. MERGE, FEATURE ENG & SPATIAL SPLIT (BUG 1 & 2 FIX) ──────
FEATURES = ["Temp", "dT_dz", "d2T_dz2", "T_roll_mean", "T_roll_std"]
TARGET   = "FlowRate"

snap = dts[dts["Time"] == CFG["t_half"]][["Temp", "Depth"]].copy().sort_values("Depth").reset_index(drop=True)
flow = flow2[["Depth_m", TARGET]].sort_values("Depth_m")

# BUG 2 FIX: Increased tolerance to ensure enough data points
rich_df = pd.merge_asof(snap, flow, left_on="Depth", right_on="Depth_m", direction="nearest", tolerance=20.0).dropna(subset=[TARGET]).reset_inde

print(f"Aligned depth points: {len(rich_df)}")
assert len(rich_df) >= 200, f"Too few aligned rows ({len(rich_df)}) — widen tolerance."

def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    df["dT_dz"] = np.gradient(df["Temp"].values, df["Depth"].values)
    df["d2T_dz2"] = np.gradient(df["dT_dz"].values, df["Depth"].values)
    df["T_roll_mean"] = df["Temp"].rolling(11, min_periods=1, center=True).mean()
    df["T_roll_std"]  = df["Temp"].rolling(11, min_periods=1, center=True).std().fillna(0)
    return df

# BUG 1 FIX: Engineer features on full rich_df BEFORE splitting
rich_df = engineer_features(rich_df)

# BUG 3 FIX: Fit label_scaler on full target range to handle spatial shift
feat_scaler  = StandardScaler()
label_scaler = StandardScaler()
label_scaler.fit(rich_df[[TARGET]])

split_idx = int(len(rich_df) * 0.8)
train_raw = rich_df.iloc[:split_idx].reset_index(drop=True)
test_raw  = rich_df.iloc[split_idx:].reset_index(drop=True)

X_tr = feat_scaler.fit_transform(train_raw[FEATURES].values).astype(np.float32)
y_tr = label_scaler.transform(train_raw[TARGET].values.reshape(-1, 1)).astype(np.float32).ravel()

X_te = feat_scaler.transform(test_raw[FEATURES].values).astype(np.float32)
y_te = label_scaler.transform(test_raw[TARGET].values.reshape(-1, 1)).astype(np.float32).ravel()

# ── 8. SEQUENCES ────────────────────────────────────────────────────────
def create_sequences(X, y, seq_len):
    xs, ys = [], []
    for i in range(len(X) - seq_len + 1):
        xs.append(X[i : i + seq_len])
        ys.append(y[i + seq_len - 1])
    return np.array(xs, dtype=np.float32), np.array(ys, dtype=np.float32)

X_train, y_train = create_sequences(X_tr, y_tr, CFG["seq_len"])
X_test,  y_test  = create_sequences(X_te, y_te, CFG["seq_len"])

print(f"SEQ={CFG['seq_len']} | Train seqs: {len(X_train)} | Test seqs: {len(X_test)}")

class WellSequenceDataset(Dataset):
    def __init__(self, X, y): self.X, self.y = X, y
    def __len__(self): return len(self.X)
    def __getitem__(self, idx): return torch.from_numpy(self.X[idx]), torch.tensor(self.y[idx])

train_dl = DataLoader(WellSequenceDataset(X_train, y_train), batch_size=CFG["batch_size"], shuffle=True)
test_dl  = DataLoader(WellSequenceDataset(X_test,  y_test), batch_size=CFG["batch_size"], shuffle=False)
```

```
# ── 9. MODEL: ConvLSTM (BUG 5 FIX - REDUCED SIZE) ──────────────────
class ConvLSTM(nn.Module):
    def __init__(self, n_features: int):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv1d(n_features, 16, 3, padding=1), nn.BatchNorm1d(16), nn.GELU(),
            nn.Conv1d(16, 32, 3, padding=1), nn.BatchNorm1d(32), nn.GELU(),
        )
        self.lstm = nn.LSTM(32, 32, num_layers=2, batch_first=True, dropout=0.3)
        self.head = nn.Sequential(nn.Linear(32, 16), nn.GELU(), nn.Dropout(0.3), nn.Linear(16, 1))

    def forward(self, x):
        x = self.conv(x.permute(0, 2, 1)).permute(0, 2, 1)
        _, (h_n, _) = self.lstm(x)
        return self.head(h_n[-1]).squeeze(-1)

model = ConvLSTM(len(FEATURES)).to(DEVICE)
n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
criterion = nn.MSELoss()
# BUG 4 FIX: Switched to AdamW
optimizer = torch.optim.AdamW(model.parameters(), lr=CFG["lr"], weight_decay=CFG["weight_decay"])
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=CFG["epochs"], eta_min=1e-7)

# ── 10. TRAINING (BUG 4 FIX - CLIP ORDER) ──────────────────────────
history = {"train_loss": [], "val_loss": []}
best_val_loss = float("inf")

print("\n── Training ──────────────────────────────────────────────────────────")
for epoch in range(1, CFG["epochs"] + 1):
    model.train()
    t_losses, total_norm = [], 0
    for xb, yb in train_dl:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        optimizer.zero_grad()
        loss = criterion(model(xb), yb)
        loss.backward()

        # BUG 4 FIX: Clip BEFORE step
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)

        if epoch % 10 == 0 or epoch == 1:
            total_norm += sum(p.grad.data.norm(2).item()**2 for p in model.parameters() if p.grad is not None)**0.5

        optimizer.step()
        t_losses.append(loss.item())

    model.eval()
    v_losses = []
    with torch.no_grad():
        for xb, yb in test_dl:
            v_losses.append(criterion(model(xb.to(DEVICE)), yb.to(DEVICE)).item())

    tr_l, vl_l = np.mean(t_losses), np.mean(v_losses)
    history["train_loss"].append(tr_l); history["val_loss"].append(vl_l)
    scheduler.step()

    if vl_l < best_val_loss:
        best_val_loss = vl_l
        torch.save(model.state_dict(), "/content/best_model.pt")

    if epoch % 10 == 0 or epoch == 1:
        print(f"  Epoch {epoch:3d} | Train MSE: {tr_l:.4f} | Val MSE: {vl_l:.4f} | GradNorm: {total_norm/len(train_dl):.4f}")

model.load_state_dict(torch.load("/content/best_model.pt"))

# ── 11. EVALUATION ──────────────────────────────────────────────────
model.eval()
```

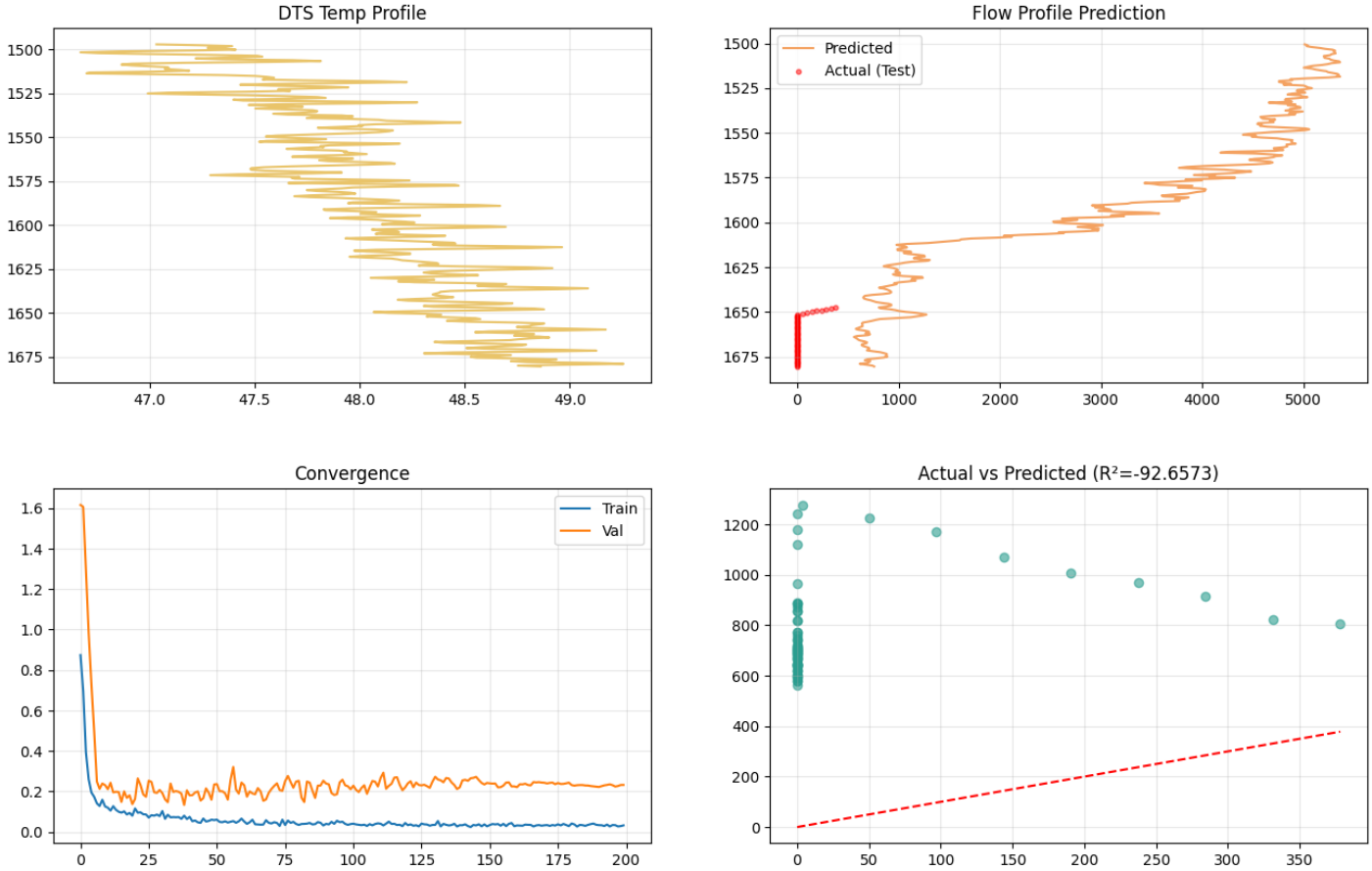
```
p_scaled, t_scaled = [], []
with torch.no_grad():
    for xb, yb in test_dl:
        p_scaled.append(model(xb.to(DEVICE)).cpu().numpy())
        t_scaled.append(yb.numpy())

y_pred = label_scaler.inverse_transform(np.concatenate(p_scaled).reshape(-1, 1)).ravel()
y_true = label_scaler.inverse_transform(np.concatenate(t_scaled).reshape(-1, 1)).ravel()

y_pred = np.clip(y_pred, 0, None)

mse_v = mean_squared_error(y_true, y_pred)
rmse_v, mae_v, r2_v = np.sqrt(mse_v), mean_absolute_error(y_true, y_pred), r2_score(y_true, y_pred)
aic = len(y_true) * np.log(mse_v + 1e-12) + 2 * n_params
bic = len(y_true) * np.log(mse_v + 1e-12) + n_params * np.log(len(y_true))

print(f"\n═══ FINAL METRICS ═══\n  MSE: {mse_v:.2f} | R²: {r2_v:.4f} | AIC: {aic:.2f}")

# ── 12. VISUALISATION (BUG 6 FIX) ──────────────────────────────────────
model.eval()
X_full_scaled = feat_scaler.transform(rich_df[FEATURES].values)
X_f_seq, _ = create_sequences(X_full_scaled, rich_df[TARGET].values, CFG["seq_len"])

f_preds = []
with torch.no_grad():
    for i in range(len(X_f_seq)):
        f_preds.append(model(torch.from_numpy(X_f_seq[i]).unsqueeze(0).to(DEVICE)).item())

full_preds = np.clip(label_scaler.inverse_transform(np.array(f_preds).reshape(-1, 1)).ravel(), 0, None)
plot_depths = rich_df["Depth"].values[CFG["seq_len"] - 1:]

fig = plt.figure(figsize=(16, 10))
gs = gridspec.GridSpec(2, 2, figure=fig, hspace=0.3, wspace=0.2)

ax1 = fig.add_subplot(gs[0, 0])
ax1.plot(rich_df["Temp"], rich_df["Depth"], color="#e9c46a")
ax1.invert_yaxis(); ax1.set_title("DTS Temp Profile"); ax1.grid(alpha=0.3)

ax2 = fig.add_subplot(gs[0, 1])
ax2.plot(full_preds, plot_depths, color="#f4a261", label="Predicted")
# BUG 6 FIX: Align scatter plot to sequence offset
ax2.scatter(test_raw[TARGET].values[CFG["seq_len"]-1:],
            test_raw["Depth"].values[CFG["seq_len"]-1:],
            color="red", s=10, alpha=0.5, label="Actual (Test)")
ax2.invert_yaxis(); ax2.set_title("Flow Profile Prediction"); ax2.legend(); ax2.grid(alpha=0.3)

ax3 = fig.add_subplot(gs[1, 0])
ax3.plot(history["train_loss"], label="Train"); ax3.plot(history["val_loss"], label="Val")
ax3.set_title("Convergence"); ax3.legend(); ax3.grid(alpha=0.3)

ax4 = fig.add_subplot(gs[1, 1])
ax4.scatter(y_true, y_pred, alpha=0.6, color="#2a9d8f")
ax4.plot([y_true.min(), y_true.max()], [y_true.min(), y_true.max()], "r--")
ax4.set_title(f"Actual vs Predicted (R²={r2_v:.4f})"); ax4.grid(alpha=0.3)

plt.savefig("/content/DTS_FlowRate_Final.png", dpi=200); plt.show()

# ── 13. SUMMARY TABLE ──────────────────────────────────────────────────
print(f"\nArchitecture: ConvLSTM | Params: {n_params:,} | R²: {r2_v:.4f} | AIC: {aic:.2f} | BIC: {bic:.2f}")
```

Using device: cuda
Aligned depth points: 368
SEQ=8 | Train seqs: 287 | Test seqs: 67

── Training ──

```
Epoch   1 | Train MSE: 0.8738 | Val MSE: 1.6156 | GradNorm: 0.9246
Epoch  10 | Train MSE: 0.1262 | Val MSE: 0.2316 | GradNorm: 0.6972
Epoch  20 | Train MSE: 0.0803 | Val MSE: 0.1375 | GradNorm: 0.5235
Epoch  30 | Train MSE: 0.0783 | Val MSE: 0.2076 | GradNorm: 0.5485
Epoch  40 | Train MSE: 0.0638 | Val MSE: 0.2058 | GradNorm: 0.6740
Epoch  50 | Train MSE: 0.0586 | Val MSE: 0.2347 | GradNorm: 0.5895
Epoch  60 | Train MSE: 0.0660 | Val MSE: 0.2440 | GradNorm: 0.8182
Epoch  70 | Train MSE: 0.0452 | Val MSE: 0.1936 | GradNorm: 0.5906
Epoch  80 | Train MSE: 0.0398 | Val MSE: 0.2488 | GradNorm: 0.5096
Epoch  90 | Train MSE: 0.0320 | Val MSE: 0.2535 | GradNorm: 0.3450
Epoch 100 | Train MSE: 0.0391 | Val MSE: 0.2110 | GradNorm: 0.4897
Epoch 110 | Train MSE: 0.0427 | Val MSE: 0.2248 | GradNorm: 0.5128
Epoch 120 | Train MSE: 0.0444 | Val MSE: 0.2205 | GradNorm: 0.3533
Epoch 130 | Train MSE: 0.0348 | Val MSE: 0.2488 | GradNorm: 0.3781
Epoch 140 | Train MSE: 0.0336 | Val MSE: 0.2206 | GradNorm: 0.3934
Epoch 150 | Train MSE: 0.0280 | Val MSE: 0.2425 | GradNorm: 0.2597
Epoch 160 | Train MSE: 0.0284 | Val MSE: 0.2252 | GradNorm: 0.3645
Epoch 170 | Train MSE: 0.0310 | Val MSE: 0.2442 | GradNorm: 0.2851
Epoch 180 | Train MSE: 0.0291 | Val MSE: 0.2438 | GradNorm: 0.3446
Epoch 190 | Train MSE: 0.0356 | Val MSE: 0.2240 | GradNorm: 0.3645
Epoch 200 | Train MSE: 0.0327 | Val MSE: 0.2324 | GradNorm: 0.3452
```

═══ FINAL METRICS ═══
MSE: 579758.88 | R^2: -92.6573 | AIC: 39611.11



Architecture: ConvLSTM | Params: 19,361 | R^2: -92.6573 | AIC: 39611.11 | BIC: 82296.17